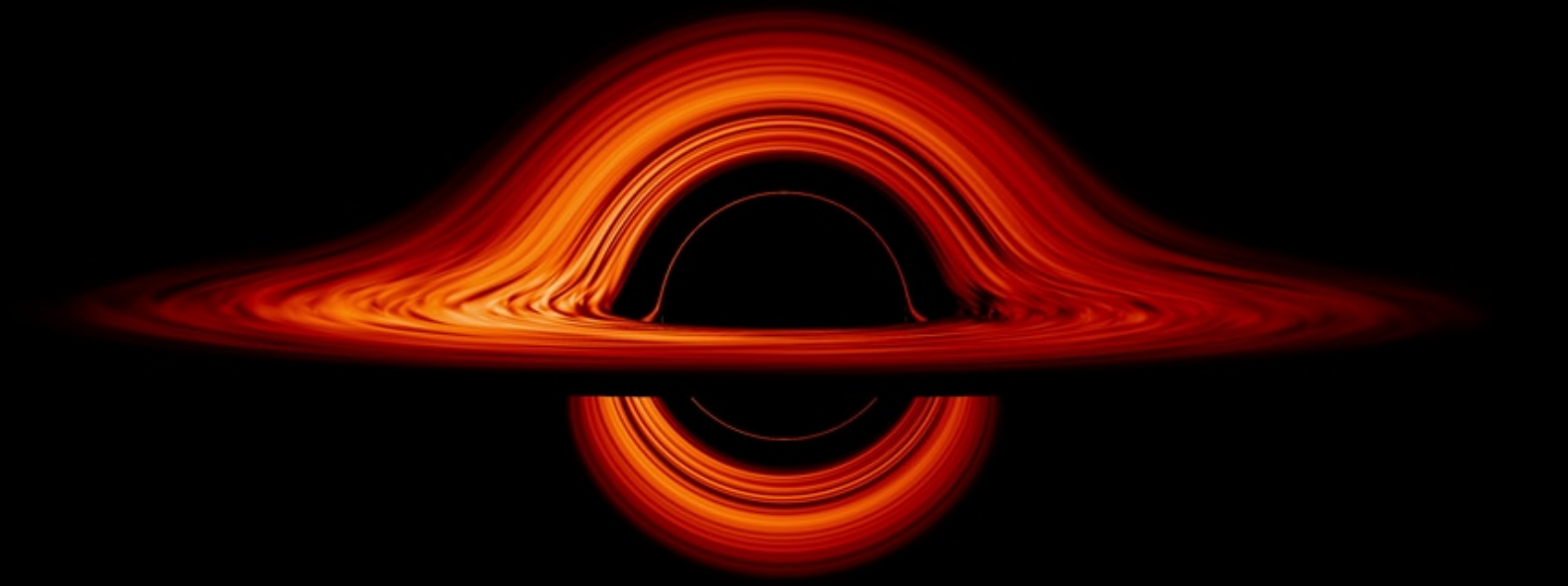
STUDENT EDITION

# *From* NLP *to* LLM

## The Evolution of Language Intelligence



**Mahmmoud A. Mahdi**

FIRST EDITION · 2026

**From NLP to LLM**
The Evolution of Language Intelligence

Copyright © 2026 Mahmmoud A. Mahdi

Computer Science Department, Faculty of Computers, Zagazig University
https://www.qursaan.com

First edition, 2026.

*To my dear father, may God forgive him.*

# Contents


How to use this book ix

I Foundations of Language Intelligence 1

1 NLP Foundations and Applications 3

1.1 Language as an engineering task 4
1.2 Historical approaches and their tradeoffs 6
1.3 Application tasks: inputs, outputs, and failures 9
1.4 Evaluation: choose evidence for the task 10
1.5 Modality and composition 16
1.6 Arabic requirements and a first project specification 18
1.7 Review and practice 22

2 Rules and Patterns 27

2.1 Regular expressions as written policies 28
2.2 Arabic patterns, step by step 43
2.3 ELIZA: dialogue from patterns 47
2.4 Review and practice 51

3 Counting and Comparing Words 57

3.1 Words and corpora: counting what is there 58
3.2 Minimum edit distance 64
3.3 Review and practice 77

4 Tokenization, Encoding, and Preprocessing 83

4.1 Tokenization and the vocabulary contract 84
4.2 Preprocessing without losing the task 95
4.3 An explicit English–Arabic preprocessing policy 98
4.4 From IDs back to text 100
4.5 Review and practice 103

**5 N-gram Language Models** 107

5.1 N-grams and count-based prediction . . . 108
5.2 Evaluating a language model . . . 112
5.3 Sparsity and smoothing . . . 114
5.4 Sampling and Arabic n-grams . . . 118
5.5 Review and practice . . . 121

# How to use this book

Language models begin with decisions about text and end with predictions that need evidence. Read with three questions in mind: what does the representation preserve, what does the model learn, and how can its behaviour be evaluated? A university help desk provides a recurring case for English, Arabic, and mixed-language applications.

## Before you begin

You should be comfortable with basic Python, lists and dictionaries, and simple probability. Chapters 1–4 need nothing more. From Chapter 5 onwards the calculations also use natural logarithms; Chapter 6's core route applies the logistic update numerically. Chapters 7 and 8 use matrix products, transposes, NumPy broadcasting, and $\tanh$. Partial derivatives and the chain rule are needed only on the challenge routes, from Chapter 6's gradient derivation to backpropagation through time in Chapter 8. Work through the small numerical examples before running the complete programs; they let you check each step.

**Math readiness check.** Try these before Chapter 5, without a calculator for the first three.

1. Simplify $\ln(1/2) + \ln(1/2)$ and $\exp(\ln 4)$.
2. If $P(A) = 2/3$ and $P(B \mid A) = 1/2$, what is $P(A, B)$?
3. A matrix of shape $(3, 11)$ is multiplied by one of shape $(11, 4)$. What is the result's shape? Can the product be taken in the other order?
4. For $y = \tanh(2x)$, write $dy/dx$ using the chain rule, given $\tanh'(u) = 1 - \tanh^2(u)$.
5. In NumPy, what shape does `np.zeros((3,4)) + np.ones(4)` have?

Answers: $\ln(1/4)$ and $4$; $1/3$; $(3, 4)$, and no, because $4 \neq 3$; $2(1 - \tanh^2(2x))$; $(3, 4)$, by broadcasting the row across all three rows. Items 1–2 are needed for Chapter 5; items 3 and 5 for Chapter 7; item 4 for the challenge routes from Chapter 6 on. If an item is unfamiliar, review it before that chapter. The core route below needs items 1–3 and 5.

**Software.** The labs need Python 3.9 or later and pytest, and the Sanad project Python 3.11; the neural chapters also need NumPy. Install it once with `python3 -m pip install numpy`, and record the version with your results.

## A route through each chapter

1. **Orient yourself.** Use the chapter map and learning objectives to identify the questions and skills the chapter develops.
2. **Work through the examples.** Trace a short input through the method. Check symbols and matrix dimensions, and predict code output before running it.
3. **Pause at the key points.** Explain the reasoning in your own words. Ask which distinctions the method preserves and where it can fail.
4. **Check your understanding.** Attempt the quiz and exercises, then revisit the sections behind any uncertain answers. Use the labs to turn the chapter's policies into tested code.

## Reading the visual cues

**Indigo highlights guide your reading.** Learning objectives describe the chapter's goals; notes draw attention to assumptions, limitations, and practical cautions. Highlighted terms introduce vocabulary used later.

**KEY POINT**

Copper panels collect key ideas for revision. Use them as prompts to explain the reasoning and connect it to an example.

Worked examples use a thin vertical rule. Code listings include line numbers for discussion, and numbered tables and figures support cross-references. Visual activities invite you to predict a result, write or sketch an answer, and change one condition to explore its effect. Work on paper or annotate the digital edition.

### Meet Sanad and Nour




From Chapter 2 on, two characters act out the help-desk case. Nour is a student at an Egyptian university who writes the way people really type: Arabizi, stretched letters, prices, and messages with no full stops. Sanad (*sanad*, "support") is the university's

help-desk bot. Its rules are only as good as the patterns behind them. Short comic strips show Sanad meeting a failure, and the text that follows explains why it happened. Boxes marked *Tips* give short practical advice.

## From foundations to experiments

Chapters 1 and 2 introduce language tasks, evaluation, regular expressions, and dialogue rules; Chapter 3 counts what a corpus contains and measures how far apart two strings are. Chapter 4 develops tokenization and preprocessing, and Chapter 5 builds n-gram language models from counts. Chapter 6 turns messages into vectors for retrieval, and classifies them with Naive Bayes and logistic regression. Chapter 7 builds a neural next-word model from a six-sentence corpus, with explicit forward and backward calculations. Chapter 8 extends the model to larger corpora and develops controlled experiments, held-out evaluation, and the route to LLM systems. Chapter 9 connects these foundations to pre-training, assistant adaptation, retrieval, tools, and evaluation of complete systems. Its practice uses paper traces rather than an API. Read these chapters in order on a first pass: later calculations reuse the earlier token policies, counts, and vocabulary.

## Core and challenge routes

The eight foundation chapters and the assistant bridge require several sessions. Choose a core or challenge route for each chapter.

| | Core route | Challenge route CHALLENGE |
|---|---|---|
| 1 | Task contracts, accuracy, precision, recall, F1, the per-class confusion matrix, modalities and pipelines; project level 1. | None: every section is on the core route. |
| 2 | Regex policies N1–N8, backreferences and lookaround, Arabic patterns W1–W3, Lab 1, ELIZA and Lab 2, scoring a rule; project level 2. | Arabic patterns W4–W6, which chain several policies. |
| 3 | Corpus data statements, token/type counts under a stated policy, lemmas and roots, basic Levenshtein distance, normalise-then-measure fuzzy search; project level 3. | Weighted and learned edit costs, backtraces, word error rate pooled over a test set, a swap move, a length filter; Lab 3, part D. |
| 4 | Tokenization units, weighted BPE, preprocessing policy; Lab 3, part C; project level 4. | Leading-boundary BPE, Unicode offsets, subword-algorithm comparison. |
| 5 | Counts and boundary markers, perplexity, add-one smoothing and <unk>, sampling, Arabic bigrams; Lab 3, G1–G6. | Interpolation and normalized backoff, with weights chosen on dev (Lab 3, G7). |
| 6 | Bag of words, TF-IDF, word-context cosine, Naive Bayes, the logistic update worked numerically; project level 5. | The logistic gradient derivation, lexical similarity, regularization, ranked-list measures, classifier audit with F-beta and macro/micro averages. |
| 7 | Shapes, softmax, cross-entropy, one training run, count baseline, the nine stages to generated output; Lab 4, P1–P2 and R1. | Backpropagation and gradient checks (Lab 4, R2–R3), SGNS and semantic evaluation (S1–S2), full decoding widget: temperature with top-$k$ and top-$p$; project level 6. |
| 8 | Train-first split, A1 and one of A2–A4; the recurrent forward pass (Lab 4, U1); project level 7. | Backpropagation through time, LSTM gates, and clipping (Lab 4, U2–U3); the other two studies from A2–A4, A5, and the Arabic extension A6. |
| 9 | Causal attention, architecture families, prediction versus adaptation, retrieval trace, assistant evidence; Lab 4, part T. | Response masking, preference methods, interpretability, judge audit. |

Lab tasks follow the same split: each lab chapter in the Lab Manual names its core and challenge tasks and the test command for each, so an unfinished challenge never affects a core submission. The Sanad project builds on the core route, but three levels also use a section from the challenge column: level 4, the leading word marker of Section 4.1.8; level 5, interpolation with a weight chosen on dev (Section 5.3.4); and level 6, backpropagation and negative sampling (Sections 7.2.9 and 7.4). Read that

section before the level, or continue from the next checkpoint, which contains the level solved.

## The corpora used in this book

Each worked calculation names its corpus. Check a number against the right one.

| Corpus | Contents and policy | First used |
|---|---|---|
| Running corpus | Six English sentences, 27 tokens, 11 types; lowercased, split on spaces (Listing 5.1). | Chapter 5 |
| Sam corpus | Three "I am Sam" sentences with <s> and </s>; casing kept. | Chapter 5 |
| Arabic three-request corpus | Three help-desk requests with boundary markers; $V = 6$ targets. | Chapter 5 |
| Lab 3 corpus | 40 Arabic requests, MSA with some Egyptian; 185 tokens, 105 types. | Chapter 3 |
| Feature documents D1–D3 | Three short English documents for bag-of-words and TF-IDF. | Chapter 6 |
| Embedding fixture | Four constructed English sentences, 12 tokens and 6 types; radius-one context pairs. | Chapter 6 |
| Urgent/routine tickets | Five labelled English help-desk tickets, three urgent and two routine; lowercased, split on spaces. | Chapter 6 |
| Larger corpus | Tiny Shakespeare or an identified replacement. | Chapter 8 |

## Notation

Symbols that look alike count different things. The book uses these meanings.

| | |
|---|---|
| $N$, $V$ (Chapter 3) | Corpus tokens and word types. |
| $V$ (Chapters 5–9) | Size of the model's target support or vocabulary, including any </s> or <unk>. |
| $N_{\text{tgt}}$ | Number of scored targets in a perplexity calculation. |
| $N$ (Chapters 7–8) | Number of training pairs, the rows of $X$ and $Y$. |
| $B$ | Mini-batch size. |
| $H$ | Hidden width of the neural model. |
| $D$, $\text{df}(w)$ | Number of documents, and documents containing $w$. |
| $C(a, b)$, $C_{\text{out}}(a)$ | Transition count and outgoing count of context $a$. |

## Practice and companion resources

The coding work is in a separate companion, the *Lab Manual*. It holds four skill labs, whose parts follow Chapters 2–9, and the Sanad project: one help-desk system that you extend at the end of every chapter, replacing a part when a chapter teaches a better one and keeping the change only if the evidence supports it. Each chapter ends with a box that names its lab and project level, and the Lab Manual opens with a table of what to do after each chapter. Every project level starts from a checkpoint that already contains working code for the earlier levels, so falling behind never blocks you. Chapter examples are supplied as `examples-chNN.zip`, lab starters as `labNN.zip`, and project checkpoints as `sanad-Lk.zip`. The student workbook provides printable practice for Chapters 1–9.

## Reading the labels

Short labels link objectives, questions, and practice across the book, the student workbook, and the Lab Manual. A label is local to its chapter unless the table says otherwise.

| Label | Meaning | Where |
|---|---|---|
| O1, O2, … | Learning objectives of the chapter. | Chapter opener |
| 1, 2, … | Quick-quiz items, one answer each. | Chapter review |
| Q1, Q2, … | *Check your understanding*: explain in a few sentences. | Chapter review |
| E1, E2, … | *Work it through*: calculate, implement, or report. | Chapter review |
| B1–B6 | Cumulative outcomes: define a language task; trace tokens and generation; justify preprocessing and representations; compute probability models; analyse neural training; design controlled evaluation. | Chapter 8 review |
| N1–N8 | Arabic normalisation and tokenization policies. | Chapter 2, Lab 1 |
| A1–A6 | Tasks of the larger-corpus assignment: core A1 and one study from A2–A4; A6 is the Arabic extension. | Chapter 8 |
| D1–D3 | The three feature documents. | Chapter 6 |
| C, G, D (Lab 3) | Lab 3 tasks: corpus statistics C1–C4 (after Chapter 4), n-grams G1–G6 (after Chapter 5); challenges G7 (interpolation) and D1–D6 (after Chapter 3: edit distance, weighted costs, backtrace, WER, and a swap move). | Lab Manual |
| P, R, K, S, U, T (Lab 4) | Lab 4 tasks: pooling P1–P2, the classifier R1–R3, decoding K1–K2, negative sampling S1–S2, recurrence U1–U3, causal attention T1–T2. Core: P, R1, K, U1, T; the rest are challenges. | Lab Manual |
| CHALLENGE | Challenge-route work: optional, beyond the core route, never needed for a complete submission. | Book, workbook, Lab Manual |
| Worksheet *n* | A sheet in the student workbook. | Workbook |
| Level *k* | A Sanad project level, started from checkpoint `sanad-Lk`. | Lab Manual |

In the *Check your progress* table at the end of each chapter, a row pairs one objective with the sections that prepare it and the labelled items that show you have met it.

PART

I

# Foundations of Language Intelligence

CHAPTER 1

# NLP Foundations and Applications

IN THIS CHAPTER

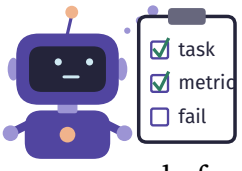

1.1 Language as an engineering task 4
1.2 Historical approaches and their tradeoffs 6
1.3 Application tasks: inputs, outputs, and failures 9
1.4 Evaluation: choose evidence for the task 10
1.5 Modality and composition 16
1.6 Arabic requirements and a first project specification 18
1.7 Review and practice 22


**LEARNING OBJECTIVES**

After working through this chapter, you should be able to:

- **O1.** distinguish NLU, NLG, and the contract of a language application;
- **O2.** analyse lexical, syntactic, referential, morphological, and pragmatic ambiguity;
- **O3.** explain the historical shift from rules to statistical, neural, and reusable models;
- **O4.** match the eight task families to their outputs and evaluation measures;
- **O5.** distinguish a task from its input/output modality, and compare a pipeline with an end-to-end system;
- **O6.** identify Arabic language requirements and turn an application idea into an evaluation plan.

Begin with two questions: what must a language application do, and what evidence would show that it works? Define the task before choosing a model. A university help desk will connect task definitions, ambiguity, evaluation, and system composition.




Follow one request through the chapter: "I cannot access the course page before tomorrow's quiz". We ask what the student needs, which tasks a system could perform, how to measure each one, and how to connect the parts. The chapter ends with a worked specification you can adapt in E1. No model training is needed yet: the goal is a problem clear enough to evaluate. Figure 1.1 shows the route.

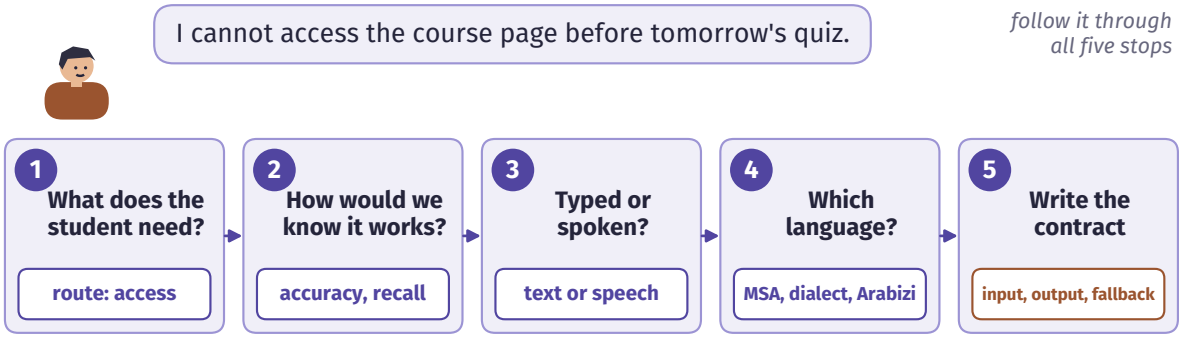



**Figure 1.1** One student message, five questions. Each stop names the question a section answers for this message: what the student needs (Sections 1.1–1.3), how to tell that a system works (Section 1.4), whether the message is typed or spoken (Section 1.5), which language it uses (Section 1.6), and the contract that ties them together (Section 1.6.1).

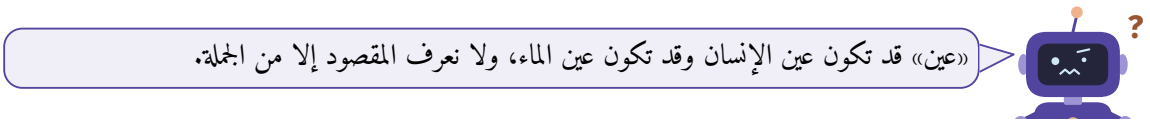
## 1.1 Language as an engineering task

**Natural language processing (NLP)** studies computational methods for analysing and producing human language. It combines ideas from linguistics, computer science, and machine learning. Language can arrive as text or as speech; an application may also connect it to images, structured records, or actions.

**Natural language understanding (NLU)** concerns tasks such as identifying intent, classifying a review, and extracting a date. **Natural language generation (NLG)** concerns producing a response, summary, or translation. These labels name the purpose of a task; they do not mean the system understands language as a person does. A generative model can answer a classification question, but its answer must still be checked like any classifier's.

### 1.1.1 Why context matters

Consider "I went to the bank". The situation decides whether *bank* means a financial institution or the side of a river. Arabic has the same problem: عين (*3ayn*) can name an eye or a water spring, and Figure 1.2 shows two sentences whose context selects each reading. In "I saw the man with the telescope", the phrase *with the telescope* can describe the observer's instrument or the man's possession. The first kind is lexical ambiguity (one word, several meanings); the second is syntactic ambiguity (one sentence, several structures).




Reference introduces a different problem. In "The trophy did not fit in the suitcase because it was too big", the likely referent of *it* is the trophy. Changing *big* to *small* changes the likely referent. Grammar alone cannot decide this; you need to know about the situation. Pragmatics adds another layer: "Great, my flight is delayed again" may express frustration despite its positive vocabulary.

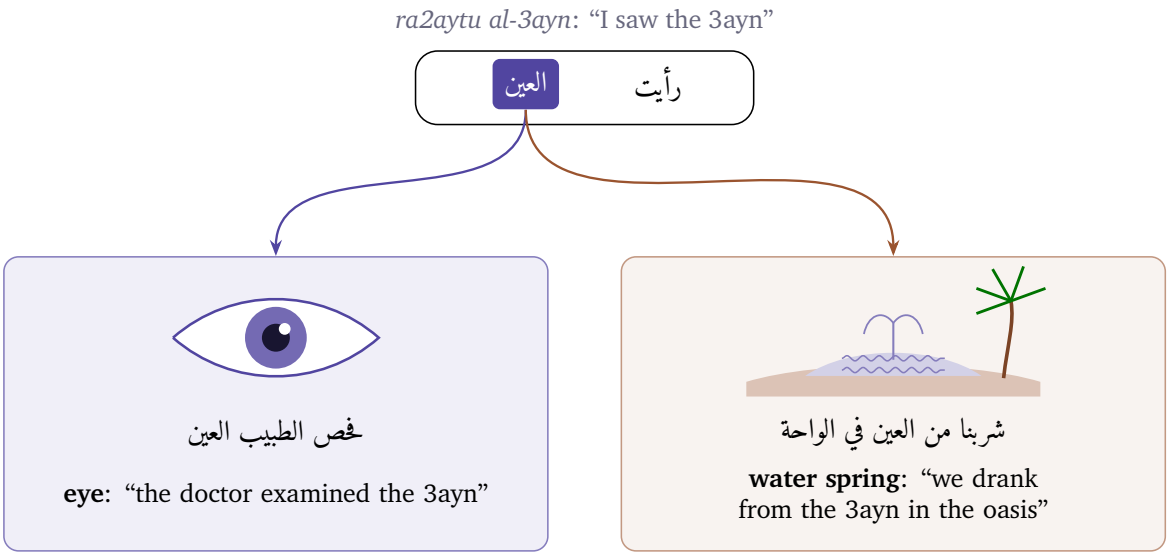
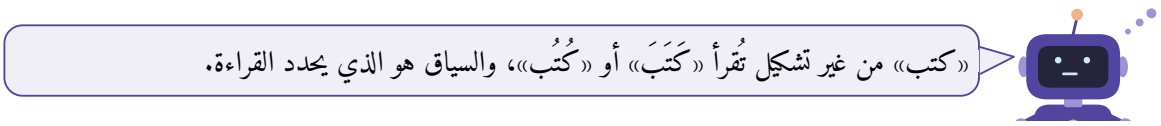



**Figure 1.2** The same word supports different interpretations. In رأيت العين (*ra2aytu al-3ayn*, “I saw the عين”), the word can name an eye or a water spring; only the surrounding context selects a reading. Token identity alone does not.

Arabic adds decisions that splitting on spaces cannot make. One written word can carry prefixes, suffixes, and clitics: وسيكتبونها (*wa-sa-yaktubunaha*) means “and they will write it”. *Morphological analysis* finds the parts of such a word and what each part means. Without vowel marks, كتب can be كَتَبَ (*kataba*, “he wrote”) or كُتُب (*kutub*, “books”); context decides. Modern Standard Arabic, dialects, Arabizi (Arabic typed in Latin letters and digits), and code-switching (mixing languages in one message) change the forms further. So look at Arabic data directly; do not reuse an English preprocessing recipe unchanged.




Throughout the book, a task’s **contract** means three things written down in advance: what the system receives, what it must return, and what would make its output wrong.

**Example 1.1**

**One message, two contracts.** A university help desk receives “I cannot access the course page before tomorrow’s quiz”. An intent classifier must pick an agreed label such as *access*. A response generator must give useful instructions based on the university’s real procedure. Accuracy measures only the first job; it says nothing about whether a generated instruction is correct, up to date, or allowed. The system may also need the course name, and “tomorrow” needs the date of the message. Asking for missing information is part of the task, not a failure. Figure 1.3 acts out both contracts.

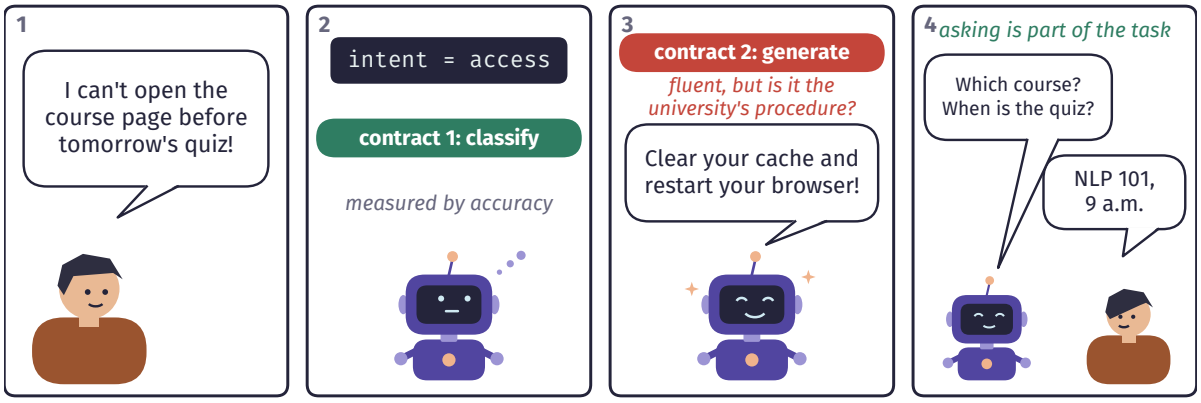



**Figure 1.3** One message, two contracts. Choosing the label *access* can be measured with accuracy; a fluent reply cannot be trusted until it is checked against the university's procedure, and asking for missing context is part of the task.

## 1.2 Historical approaches and their tradeoffs

Before building the help desk, look at what engineers have tried. The history of language processing is a story of where engineers put their effort: first rules written by hand, then patterns counted from corpora with hand-chosen features, then neural models that learn their own representations, and finally pre-trained models reused across many tasks by further training or prompting.

**Table 1.1** Four approaches and their engineering tradeoffs

| Approach | What engineers specify | Main limitation to investigate |
|---|---|---|
| Rules | Patterns, dictionaries, and grammar; ELIZA illustrates pattern-based dialogue. | Coverage and brittle behaviour outside anticipated cases. |
| Statistical | Features and probability estimates; examples include n-grams and text classifiers. | Sparse observations and dependence on feature choices. |
| Neural | Training objective, architecture, data, and learned representations. | Data needs, optimisation, and generalisation. |
| Pre-train and adapt | Reusable models, task data, prompts, and system integration. | Reliability, cost, bias, and domain mismatch. |

**KEY POINT**

A change in modelling approach changes the engineering work; it does not remove it. Data selection, evaluation, and the application contract remain necessary when a model can be prompted instead of trained from scratch.

### 1.2.1 Ten landmarks, and the problem each exposed

The milestones below show changes in what systems could do; no new method replaced the old ones overnight. Figure 1.4 puts the ten landmarks on one line, grouped into the four approaches of Table 1.1. The band under each group names what engineers had to specify in that era.

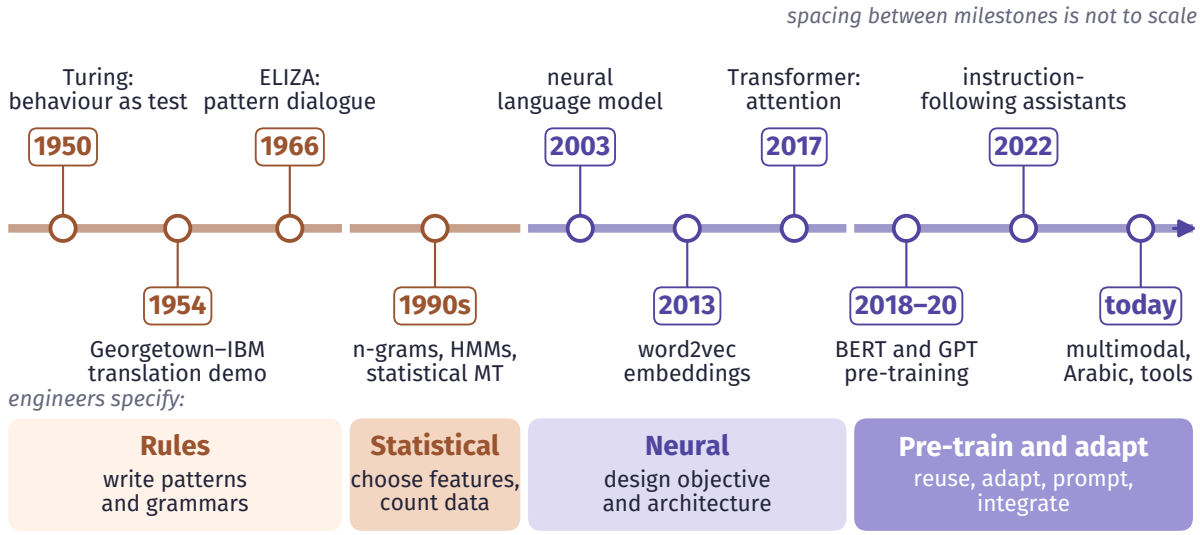



**Figure 1.4** Ten landmarks in the development of language systems. Each era moved engineering effort to a different place; earlier methods remain useful baselines and components.

1. **1950: conversational behaviour as a test.** Turing's discussion made a written exchange a possible way to investigate machine intelligence. An observer sees responses, not internal understanding. The enduring engineering question is which observable behaviour supports a particular claim.

2. **1954: the Georgetown–IBM demonstration.** A restricted translation demonstration made machine translation visible. Restricted vocabulary and selected sentences can make a compelling demonstration while leaving ordinary text unresolved. A held-out evaluation should expose the restrictions rather than inherit the demonstration's selection policy.

3. **1966: ELIZA.** Pattern-based dialogue could transform a user's phrase into a plausible follow-up. Fluency and responsiveness encouraged users to attribute understanding. This distinction matters whenever a generated answer sounds helpful but lacks evidence for its content.

4. **1990s: statistical modelling spreads to text.** N-grams and hidden Markov models, developed for speech recognition in the 1970s and 1980s, joined statistical translation in estimating patterns from data. Engineers still selected representations, states, or features. A count estimate replaces a manually written rule, but its reliability depends on which observations reached the counts.

5. **2003: neural probabilistic language modelling.** Bengio and colleagues jointly learned word representations and a sequence probability model, sharing parameters across contexts. Chapter 7 makes the representation and prediction connection concrete through a row lookup and an output distribution; its small network does not reproduce the full historical model.
6. **2013: word2vec.** Efficient embedding-learning methods made distributed word representations widely usable. Vector relationships illustrate this idea, but an analogy such as king–man+woman is an empirical pattern in some embeddings, not a law of language or a guarantee for our tiny data.
7. **2017: the Transformer.** This architecture uses a mechanism called *attention* to combine information from token positions. Which positions a token can attend to depends on the attention mask; a next-token model cannot look ahead at future tokens. It was first used for translation. Chapter 9 explains how it works.
8. **2018–2020: reusable pre-training.** BERT and GPT-style models showed that one model trained on a large text collection could be reused for many tasks. BERT learned by filling in hidden tokens using the context on both sides; GPT-style models learned by predicting the next token. (Tokens may be words or smaller units; see Chapter 4.) Reuse can mean more training on task data, or just writing examples into a prompt; a prompt does not change the model.
9. **2022: assistants become a public interface.** Instruction-following and preference-based adaptation helped make conversational generation accessible. A helpful interface changes user expectations: evaluation must include unsupported answers, task completion, and the ability to handle missing information.
10. **Multimodality, Arabic, and tools.** Models and products connect text to audio, images, retrieval, and external actions. Rather than memorising changing model sizes or product names, ask which representations, training objectives, and system interfaces each example uses.

The historical trend is not “more parameters solve every problem”. It is an expansion of what can be learned and reused, accompanied by new costs and evaluation needs. A regular expression can remain the most suitable component for a tightly specified date format even when a generator is available.

For the help desk, a rule can identify an exact course-code format, a count-based classifier can provide a routing baseline, and a reusable model can handle a wider range of wording. Compare them on the same labelled messages and important error types. Their place on the timeline is not evidence that one will serve this campus better.

## 1.3 Application tasks: inputs, outputs, and failures

The student's message could feed several tasks, and an application usually combines several. To name a task, say what it receives, what it returns, and what would make the result wrong. The help desk might route a message, find a procedure, and draft an answer: three contracts, even if one model does all three. Table 1.2 gives eight task families with examples; the next section explains how to evaluate them.

**Table 1.2** Eight task families: start with the required behaviour

| Task | Input → output | Example failure |
|---|---|---|
| Classification | Student message → access, assessment, registration, or other. | An access problem is routed to registration. |
| Entity extraction | Message → spans and types, such as a course code or date. | The extractor drops a digit from the course code. |
| Translation | Arabic request → English request with the same meaning. | "Cannot access" becomes "can access". |
| Question answering and retrieval | Question and documents → ranked passages or an answer. | An obsolete guide is retrieved, or the answer invents a deadline. |
| Summarisation | Ticket thread → selected sentences or a shorter account. | The summary omits that the quiz is tomorrow. |
| Speech recognition/synthesis | Audio → transcript; text → spoken response. | "Fifteen" becomes "fifty", or speech obscures the count. |
| Code generation | Specification → a program, such as a ticket router. | The program routes every ticket to the same queue. |
| Dialogue | Conversation and context → responses or tool requests. | The assistant forgets which course the student identified. |

Classification can also assign sentiment, topic, or spam labels. Define the unit being labelled: a whole review may have one sentiment label, while an aspect-level task labels the screen and battery separately. In the help desk, intent routing and urgency detection are separate classification tasks. An access request can be ordinary or urgent; an assessment request can also be urgent.

Entity extraction identifies spans and their types. In "Ahmed joined Zagazig University in October 2026", the organisation spans two words. **Named entity recognition (NER)** identifies entities such as people and organisations; **part-of-speech (POS) tagging** instead labels grammatical categories such as noun and verb. Correct POS tags do not establish that an organisation entity was found.

A translation should preserve meaning without copying the input's word order. An extractive summary selects source material; an abstractive summary produces new

wording. Retrieval ranks evidence, whereas answering uses or extracts it. These distinctions determine which output must be checked. A relevant passage may still lack the requested date.

Dialogue connects these tasks across turns. The help desk may ask for a course code, retrieve a procedure, and read instructions aloud. A plausible final sentence can conceal an earlier wrong record or tool failure. Evaluate the completed user task as well as each component.

## 1.4 Evaluation: choose evidence for the task

Once a task is named, ask how we would know it works. The measure follows from the task. Many measures compare the output with a **reference**: a label, transcript, or relevance judgement made under an agreed policy, without looking at the system's output. Other evidence, such as whether users finish their task or how fast the system answers, needs its own rule for success. A score means something only when its data and policy are stated.

### 1.4.1 What different tasks require

Each task family in Table 1.2 needs its own kind of evidence. Routing compares predicted labels with reference labels. Extraction checks whether the right span and type were found; a correct type with a missing word is still an error. Retrieval checks whether useful passages appear near the top of the list. Translations and summaries can be worded in many correct ways, so check for missing facts, changed numbers, and claims the source does not make. Speech recognition compares transcripts, generated code runs against independent tests, and dialogue checks whether the student's whole task was completed.

> **Reading route.** The rest of this section teaches accuracy, precision, recall, and F1 on the help-desk router. Chapter 3 adds word error rate for speech, and Chapter 6 adds measures for ranked lists and for combining scores across classes. Other names you may meet, such as BLEU, chrF, and COMET for translation, ROUGE for summaries, and pass@k for code, are signposts for later study, not formulas to learn now.

### 1.4.2 Measuring accuracy step by step

A score such as "accuracy 0.70" is the end of a procedure. If you cannot say how each step was done, you cannot tell what the number means. Follow the procedure once for the help desk's routing task, which assigns each message to *access*, *assessment*, *registration*, or *other*.

**Step 1. Fix the task and the label policy.** Write down the four labels, what each one covers, and what to do with a message that fits two of them. Without this policy, two people can label the same ticket differently and both feel correct.

**Step 2. Build a held-out test set.** Collect real tickets that the model has never seen and will not be tuned on; a model scored on its own training tickets is answering questions it has already seen. Annotators then give each ticket a **reference label** under the policy, *before* seeing any model output. If two annotators agree on 9 of 10 tickets, their agreement is 0.90 on this small sample, but also look at the disagreement and at how many tickets each label has. Settle any policy changes on training or development tickets before freezing the test labels.

**Step 3. Run the model once on every test ticket** and record each prediction. Do not rerun it until the score looks better.

**Step 4. Compare item by item.** A prediction is correct only if it equals the reference label exactly. Figure 1.5 marks each of ten tickets right or wrong.

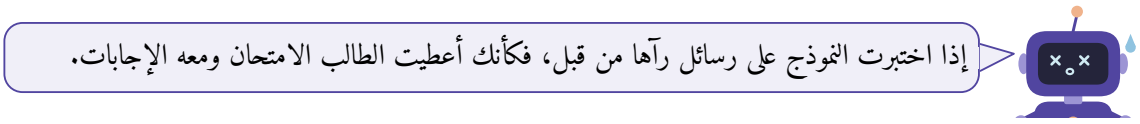
**Step 5. Count and divide. Accuracy** is the fraction of test items labelled correctly:

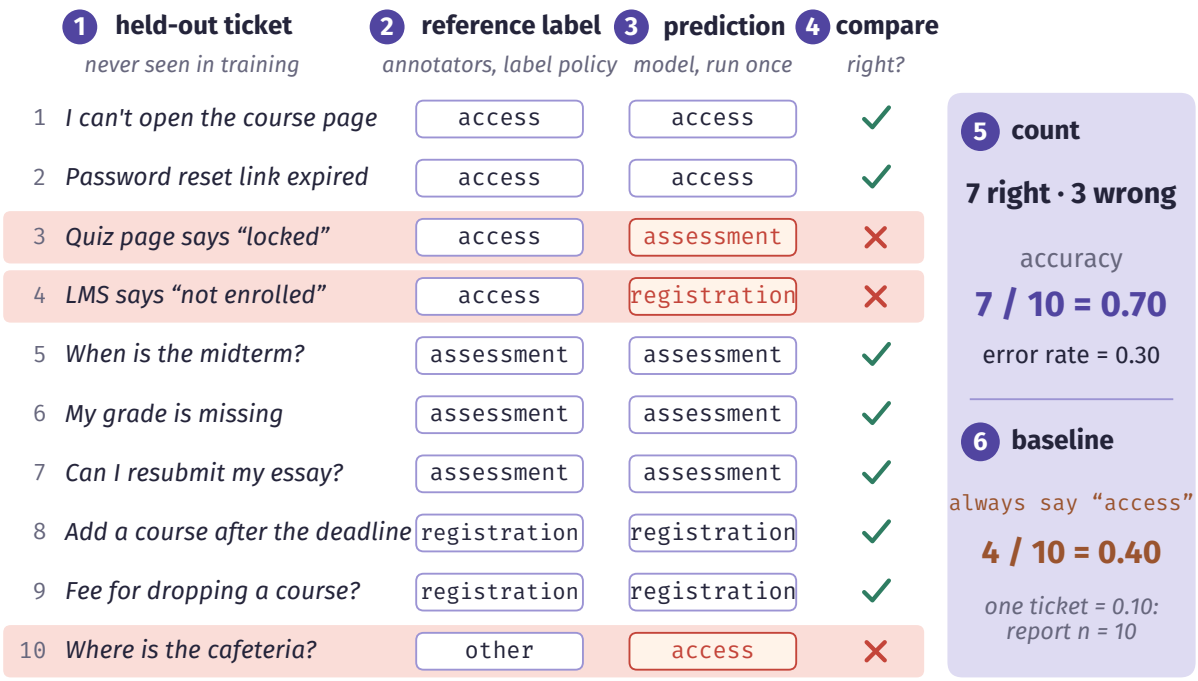
$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of test items}} = \frac{7}{10} = 0.70.$$

The **error rate** is $1 - \text{accuracy} = 0.30$.

**Step 6. Compare with a baseline.** A **baseline** is a simple strategy to compare the model against. For a majority-class baseline, pick the most common label in the training data and freeze it before testing. Suppose it is *access*, which appears in 4 of the 10 test tickets: always predicting it scores $4/10 = 0.40$. Picking the label by looking at the test set would leak test information. A score of 0.70 beats 0.40 on this small sample, but whether the model is useful also depends on which errors it makes.

**Step 7. Read the errors, not only the score.** Ticket 3 ("Quiz page says locked") is an access problem that mentions a quiz, so the error may come from an ambiguous policy rather than a weak model. Ticket 10 asks about the cafeteria. It is out of scope, yet the model forced it into *access*. Each error suggests a different fix.






*Shaded rows are the three errors; read them before trusting the score.*

**Figure 1.5** Measuring accuracy on ten held-out tickets. Each reference label is fixed before the model runs; every prediction is then marked right or wrong, counted, and compared with a majority-class baseline.

### 1.4.3 Counting a classifier's outcomes

Accuracy gives one number for all four labels. Some questions are simpler: a single yes/no decision. Urgency is one, and it cuts across the labels, since an access request and an assessment request can both be urgent. Suppose the help desk wants to find *urgent* tickets. Call urgent the **positive class**: the class whose detection the measure is about. "Positive" names that class; it does not say the outcome is good. Every other ticket is *negative*, here *ordinary*.

For each evaluated ticket, ask two independent questions (Figure 1.6). First, what does the reference label say: is the ticket really urgent? Second, what did the model predict: did it say urgent? Two yes/no answers give exactly four outcomes:

- **true positive (TP):** really urgent, and the model said urgent;
- **false negative (FN):** really urgent, but the model said ordinary (a missed ticket);
- **false positive (FP):** really ordinary, but the model said urgent (a false alarm);
- **true negative (TN):** really ordinary, and the model said ordinary.

Read each name in two parts. *True* or *false* records whether the prediction was right; *positive* or *negative* records what the model predicted. A false negative is therefore a wrong "ordinary" prediction.

الكلمة الثانية في الاسم هي ما قاله النموذج، والأولى تخبرك هل كان محقًا، فالسلبي الخاطئ تذكرة عاجلة ظنها النموذج عادية.

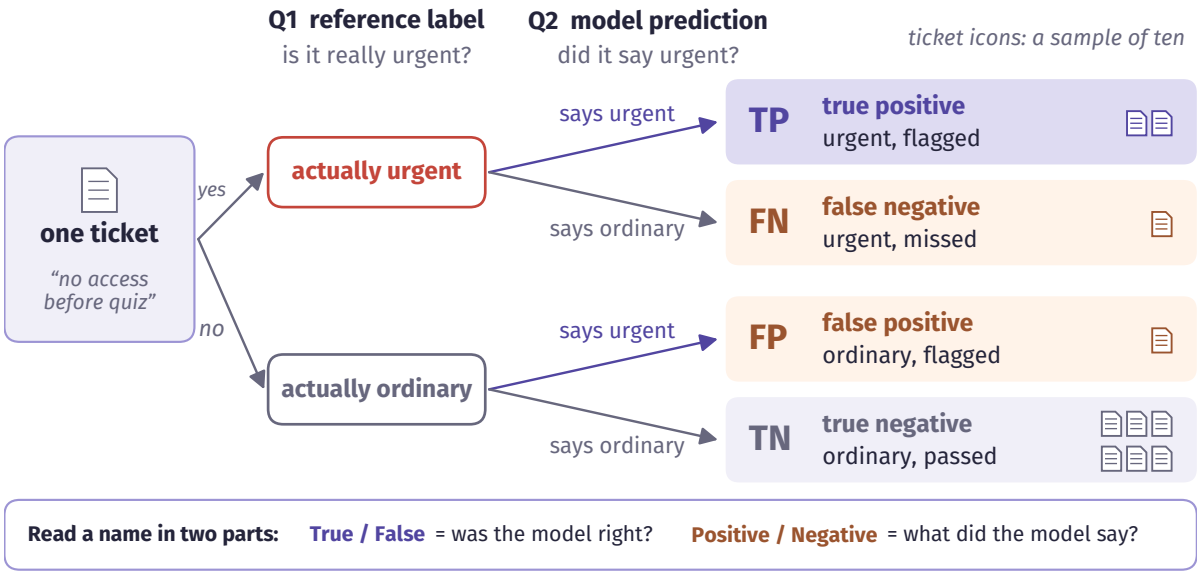



**Figure 1.6** Every evaluated ticket answers two questions. The reference label and the prediction together place it in exactly one of four outcomes; the icons show a sample of ten tickets.

Figure 1.6 sorts a sample of ten tickets: three are really urgent and seven are ordinary. The model finds two of the urgent tickets and raises one false alarm, so $TP = 2$, $FN = 1$, $FP = 1$, and $TN = 6$. Each measure below divides one group of these tickets by another, and each answers a different question:

- **Accuracy**: of all tickets, what fraction did the model label correctly? $(TP + TN)/\text{all} = (2 + 6)/10 = 0.80$.
- **Precision**: of the tickets the model *flagged* as urgent, what fraction really were urgent? $TP/(TP + FP) = 2/3 \approx 0.67$. Low precision means staff waste time on false alarms.
- **Recall**: of the tickets that *really were* urgent, what fraction did the model find? $TP/(TP + FN) = 2/3 \approx 0.67$. Low recall means urgent students wait unnoticed.
- **F1**: a single summary that is high only when precision and recall are both high, $F_1 = 2PR/(P + R)$. Here $F_1 = 0.67$.

The denominators are the key. Precision and recall share the numerator $TP$; precision divides by what the model *said*, recall by what was *true*. Arranged with reference rows and prediction columns, the four counts form a $2 \times 2$ confusion matrix. Precision then reads down the "predicted urgent" column and recall reads along the "actually urgent" row. If a denominator is zero, the ratio is undefined: say how you report it, and never count it as good performance.

الضبط يسأل: مما اختاره النموذج، كم كان عاجلًا فعلًا؟ والاستدعاء يسأل: من التذاكر العاجلة، كم استطاع أن يجد؟

**Example 1.2**

**Accuracy can hide the class that matters.** Scale the sample up. In 1000 tickets, 100 require urgent access help and 900 do not. A classifier finds 60 of the urgent tickets but also flags 40 ordinary tickets. Sorting every ticket as in Figure 1.6 gives $TP = 60$, $FN = 100 - 60 = 40$, $FP = 40$, and $TN = 900 - 40 = 860$. Figure 1.9 arranges these counts as a confusion matrix.

$$\text{accuracy} = \frac{TP + TN}{1000} = 0.92,$$

$$\text{precision} = \frac{TP}{TP + FP} = 0.60, \quad \text{recall} = \frac{TP}{TP + FN} = 0.60.$$

Its urgent-class $F_1 = 2PR/(P+R) = 0.60$. The majority baseline of Step 6, predicting every ticket as non-urgent, would reach 0.90 accuracy while finding none of the urgent tickets. Whether 0.60 recall is acceptable depends on the consequences and fallback procedure. These constructed counts illustrate a metric calculation, not a deployed system result. Figure 1.8 acts out the gap.

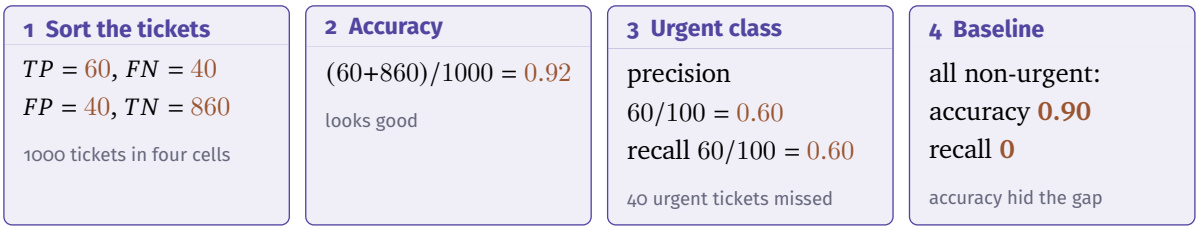



**Figure 1.7** Example 1.2 from counts to measures. Amber marks the new number in each frame.

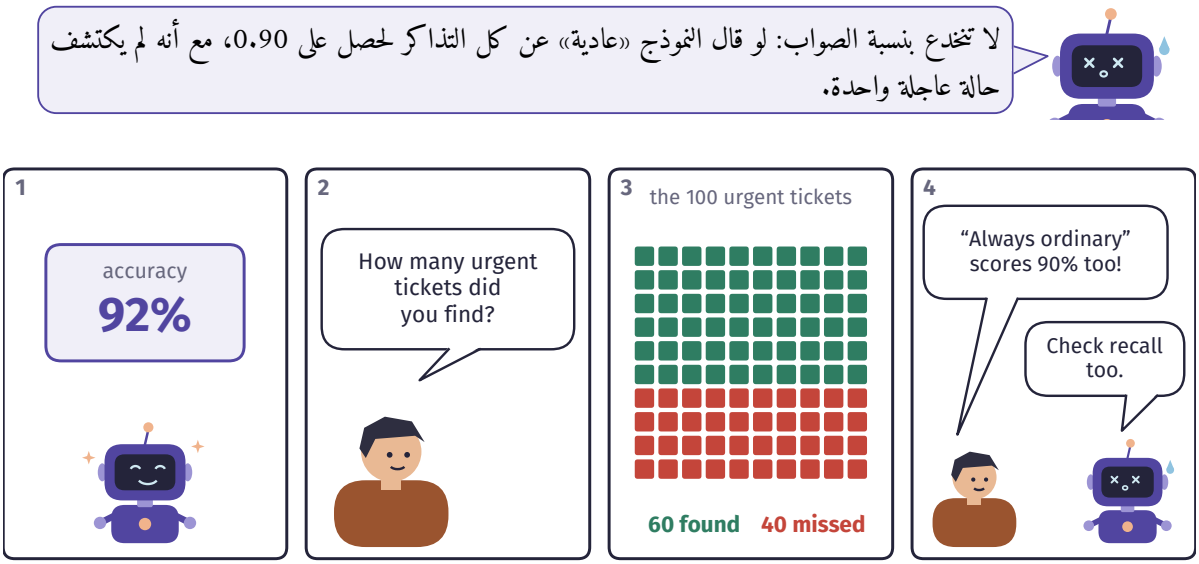



**Figure 1.8** High accuracy, missed students. Sanad's 92% comes mostly from the 900 ordinary tickets. Recall shows that 40 of the 100 urgent tickets were missed, and a model that always answers "ordinary" already scores 90%. The counts are those of Example 1.2, asserted in the figure's script.

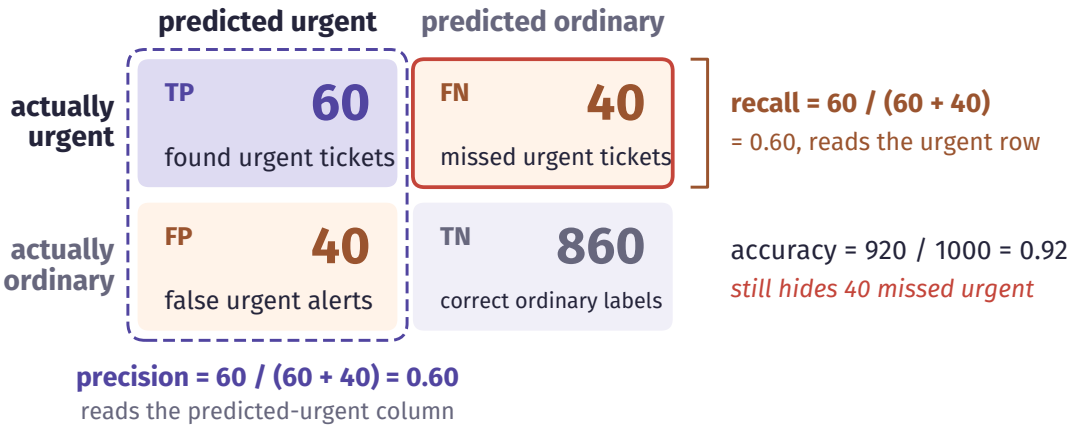



**Figure 1.9** The urgent-ticket counts form four distinct outcomes. Precision reads the predicted-positive column; recall reads the actual-positive row.

**Pause, predict, then explore**

A new constructed set of 100 tickets has $TP = 20$, $FP = 5$, $FN = 10$, $TN = 65$.

1. In grid 1, write each count in its cell. Circle the cell that holds missed urgent tickets.
2. Predict before calculating: if the model now finds five of the missed urgent tickets, which two cells change, and which measure should rise more?
3. Fill grid 2 and both pairs of formula lines. Check your prediction.

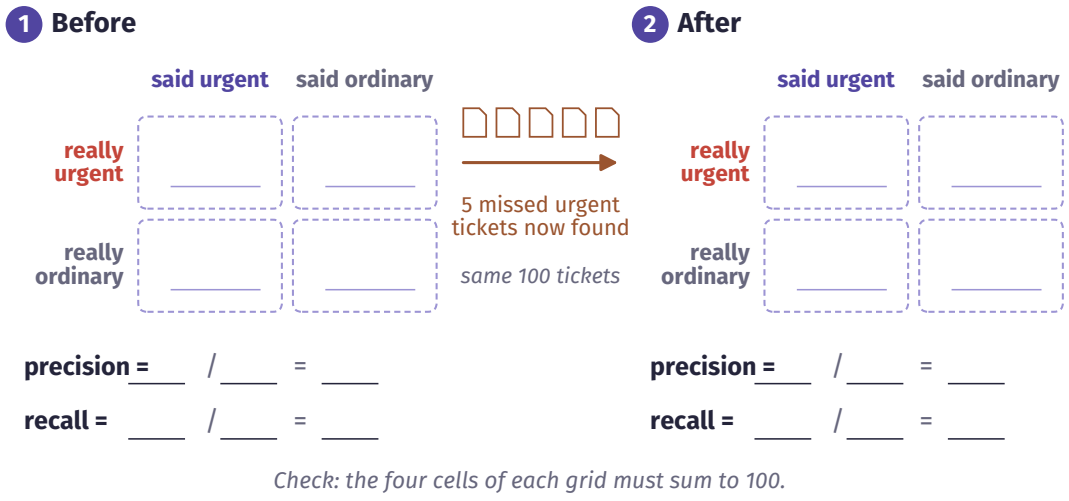



**Figure 1.10** Change an error count and predict which measures respond.

### 1.4.4 Looking inside accuracy: one class at a time

Return to the four-intent router. Accuracy treats every error alike. To see *which* intents are confused, arrange the same ten tickets in a **confusion matrix** (Figure 1.11). Each row is a reference label and each column a predicted label; a cell counts the tickets with that combination. Correct predictions lie on the diagonal, so

$$\text{accuracy} = \frac{\text{sum of the diagonal}}{\text{sum of all cells}} = \frac{2+3+2+0}{10} = 0.70.$$

Off-diagonal cells name the confusions: one access ticket went to assessment, one to registration, and the *other* ticket went to access.

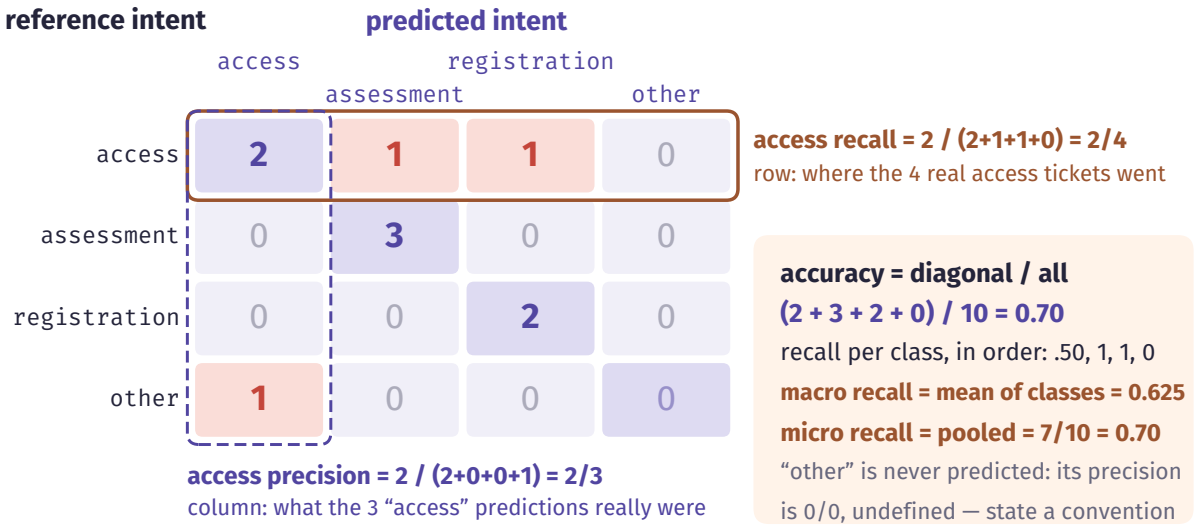



**Figure 1.11** The same ten tickets as a four-intent confusion matrix. The diagonal holds correct predictions; a row gives one class's recall and a column gives its precision. The macro and micro recall lines preview Chapter 6: macro recall averages the four class recalls, while micro recall pools every ticket.

Precision and recall can be computed for one intent at a time, treating it as the positive class. Take *access*. Four tickets really are access requests, and the model labelled two of them *access*: access recall is $2/4 = 0.50$, read along the row. The model labelled three tickets *access*, and two were right: access precision is $2/3 \approx 0.67$, read down the column. Doing this for every class gives recall $1$ for assessment and registration, and $0$ for *other*. *Other* was never predicted, so its precision is $0/0$, which is undefined; say how you report it instead of quietly counting it as 0 or 1.

You now have one precision and one recall per intent. Chapter 6 shows how to combine them into a single summary, and how to weigh errors that have different costs.

## 1.5 Modality and composition

Our student might type the message, or speak it into a phone. The task says what the system must do; **modality** says what kind of input and output it handles; **composition** says how its components connect. Keep these descriptions separate when choosing a help-desk design.

### 1.5.1 Input and output modalities

Modality is named by input and output: text-to-text, speech-to-text, text-to-speech, or image-to-text, for example. Table 1.3 distinguishes input and output modalities. Some cells commonly use one model; others are often implemented by several components. Listing a combination does not establish that a particular product supports it.

**Table 1.3** Input and output modalities in language systems

| Input | Text output | Speech output | Image output |
|---|---|---|---|
| Text | Translation, summary, dialogue, or code. | Synthesis, often through pronunciation and acoustic stages. | A text-conditioned image-generation system. |
| Speech | Transcription or speech translation. | Speech translation, conversion, or a voice response. | Often recognition followed by text-to-image generation. |
| Image | OCR, caption, visual answer, or document reading. | Often OCR/captioning followed by speech synthesis. | Image transformation or instruction-guided editing. |

Modality describes what goes in and out, not how a model is built inside. Chapter 9 introduces the main internal designs; until then, describe a system by its task, its modalities, and how its components connect.

### 1.5.2 Pipelines and end-to-end systems




A voice help desk might chain recognition, a text assistant, and speech synthesis. Intermediate transcripts make individual stages easier to inspect. They also introduce handoffs: misrecognising “fifteen” as “fifty” can cause a correct response to the wrong question. Measure stage performance and end-to-end task success.

A model trained directly on multiple modalities can avoid some explicit handoffs and preserve information that a text transcript loses. It still makes errors, and any latency or quality advantage must be measured under comparable conditions. Meeting minutes, accessibility tools, and document question answering all offer different ways to compose recognition, retrieval, translation, and generation.

> **Example 1.3**
>
> **A voice help desk has several contracts.** The user asks to reopen access for fifteen students. Recognition supplies a transcript, the text component identifies the course and count, a tool checks an authorised operation, and synthesis reads the response.

> If recognition hears “fifty”, even a perfect extractor will use the wrong number. Reading the number back for confirmation catches this error, but it says nothing about the other stages.

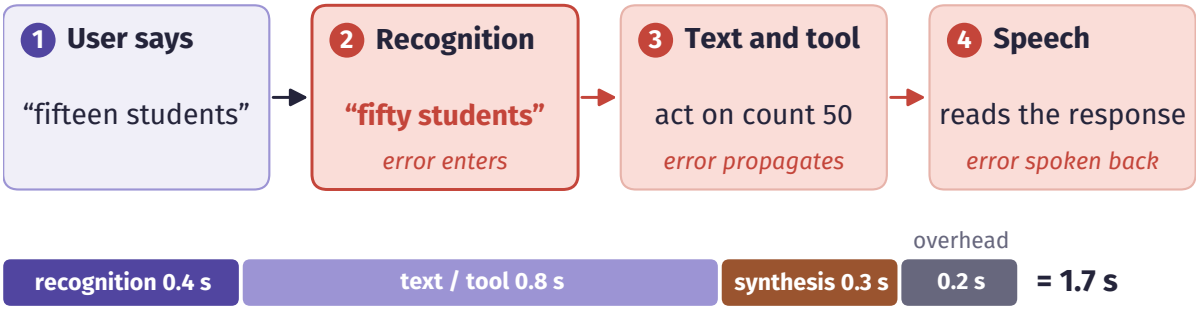



**Figure 1.12** An early recognition error can propagate through otherwise correct downstream stages. Confirming the count tests a different claim from measuring transcript quality.

## 1.6 Arabic requirements and a first project specification

Our student may write in English, in Modern Standard Arabic, in Egyptian Arabic, or in Arabic typed with Latin letters. Arabic shows how the properties of a language shape modelling decisions. One written word can combine a conjunction, a tense marker, agreement, and a pronoun. A normalisation that helps retrieval may remove a distinction needed for pronunciation or grammatical analysis. Modern Standard Arabic and regional dialects may differ in word choice and spelling; code-switching can place English terminology inside Arabic syntax.

*Arabizi* represents Arabic with Latin characters and sometimes digits. The mixed-language request “3ayez a5ls el project bokra” also contains an English word. A Latin-only tokenizer will retain some fragments but need not preserve their intended units. A benchmark made only of edited Modern Standard Arabic cannot establish performance on these help-desk messages.

Arabic toolkits such as CAMeL Tools and Arabic models such as AraBERT and Jais exist, but a model’s name is not an evaluation result: test it on your own messages.

### 1.6.1 A worked help-desk specification

The following constructed contract makes the running case concrete. It is a proposal for one campus, not a claim about a deployed service. Stakeholders must agree the label policy and acceptance targets before using the results for a release decision.

**Example 1.4**

**Start with routing, keep advice separate.**

1. **Input.** One student message, its date, and any earlier turns the service actually receives. Messages may be in English, Modern Standard Arabic, Egyptian Arabic, Arabizi, or a mix. Keep the original text.
2. **Routing output.** One label: *access*, *assessment*, *registration*, or *other*. If a message contains two separate requests, return no label and ask which to handle first.
3. **Urgency output.** A separate yes/no flag: urgent when the student is blocked from an assessment due within 24 hours. If the deadline is unknown, ask for it rather than guess.
4. **Reference labels.** Two annotators label each test message using these written rules, before seeing any model output. A help-desk reviewer settles disagreements.
5. **Evaluation.** On held-out messages, report routing accuracy, per-intent precision and recall, and urgent-flag recall, separately for each language variety. Report the number of reference-urgent messages as well as recall. A proposed target is urgent-flag recall of at least 0.90, with every missed urgent case reviewed.
6. **Fallback.** When the question stays unclear, or no approved procedure applies, hand the message to help-desk staff. Routing never changes a registration or an account by itself.

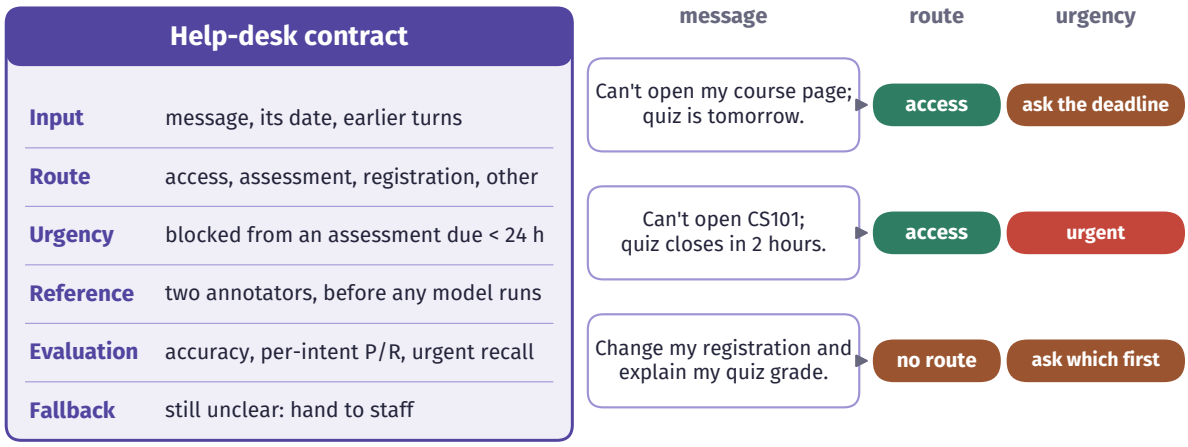



**Figure 1.13** The contract of Example 1.4 on one card, with three messages run through it. A message can have a clear route and still need a question before the help desk acts.

The accuracy calculation above assumes that every ticket has one reference intent and gets one predicted intent. This specification also allows the system to ask a question or pass the message to staff. So, for each test message, the reference records the action it needs: a route (one intent), a question (missing information or two requests), or a

person (no approved procedure). Table 1.4 shows how each combination is counted. Routing accuracy uses only the messages whose reference action is a route, and all of them stay in its denominator: if the system asks or escalates instead, the message counts as wrong. Also report how many of them were routed, and the accuracy on those. A reference-urgent message left unresolved counts as a missed urgent case. Dropping unresolved cases would let a system raise its score by avoiding hard messages.

**Table 1.4** Scoring a system that may route, ask, or pass a message to staff. Only the first row enters routing accuracy; report the counts of the other cells beside it.

| Reference needs | System routes | System asks | System passes to staff |
|---|---|---|---|
| A route | Correct if the label matches; otherwise a routing error. | Routing error: an unneeded question. | Routing error: an unneeded escalation. |
| A question | A guess where a question was needed. | Correct, if the information really was missing. | An acceptable fallback; count it. |
| A person | A route with no approved procedure behind it. | An unneeded question; count it. | Correct. |

Apply the contract to the opening message, as Figure 1.13 does. The main request concerns access, while the quiz supplies context, so the route is *access*. "Tomorrow" does not by itself specify a deadline within 24 hours: ask for the course and quiz deadline before resolving urgency or giving a course-specific procedure. By contrast, "I cannot open CS101; its quiz closes in two hours" has an access route and an urgent flag under this policy. "Change my course registration and explain my quiz grade" has two independent requests, so ask which to handle first. Knowing the route is not always enough to act.

Collect messages under the applicable data policy and remove or secure identifying information. State the campus, semester, language varieties, and context available at evaluation. Performance within those limits is not universal Arabic understanding. Detailed split protocols and experimental comparisons return in Chapter 8.

> **A useful first deliverable is a contract and an error taxonomy.** For each example, record the expected output, allowed alternatives, needed context, and consequence of an incorrect output. Label whether an error arose from missing context, representation loss, model prediction, retrieval, or an external interface. This will guide the representations developed in the next chapters.

## Chapter summary

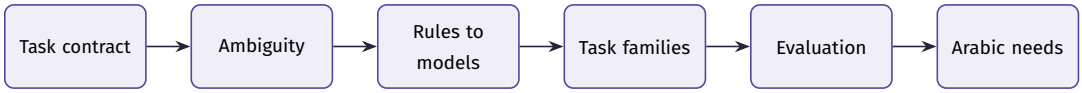
NLP applications turn language into labels, spans, rankings, generated text, speech, or actions. The same words can mean different things; context and clarifying questions help decide which meaning matters. Rules, statistical methods, neural models, and pre-trained models put engineering effort in different places, but all need a clear task contract and suitable evidence. Choose a measure that fits the output, look at the important errors, and keep the task, its modalities, and the way parts connect distinct. Plan for Arabic varieties and writing habits from the start.



**CODE IT IN THE LAB MANUAL**

**Project level 1: the yardstick.** Build Sanad's evaluation harness from this chapter's measures: accuracy, the confusion matrix, per-class precision and recall, and the majority baseline. Then score the first baselines: a router that always answers the majority intent and a search that matches exact words. Sanad's router uses five intents, adding *fees* to this chapter's four: the project has its own task contract, separate from the worked router here.

### Chapter 1 at a glance




#### Keep these ideas

- Every language application has a contract: input, output, users, and what counts as success.
- Ambiguity comes at every level: words, structure, reference, morphology, and intent.
- The field moved from hand rules to statistics, neural networks, and reusable pretrained models; each move traded control for coverage.
- Choose the evidence before the model: accuracy can hide the class that matters, so report precision and recall per class.

#### In one small example

1000 tickets, 100 urgent; the classifier finds 60 and flags 40 others. Accuracy is 0.92, but precision and recall on the urgent class are both 0.60 (Figure 1.7).

**Common mistake.** Reporting accuracy alone on an unbalanced task.
**Instead.** Compare with the majority baseline (0.90 here) and report the measure for the class whose errors cost most.

## 1.7 Review and practice

These questions follow the chapter objectives in order, with two questions on evaluation. Explain the reasoning before checking the optional answers. Workbook sheets 1–2 provide space for a task contract, metric calculations, and a composition diagram.

### Quick quiz

Choose one answer for each question before reading the rest of the review; the key is in the instructor edition.

**1.** In "I saw the man with the telescope", why are there two readings?

- **A** *saw* has two dictionary senses.
- **B** The phrase *with the telescope* can attach to different words.
- **C** The pronoun has two possible referents.
- **D** Diacritics were omitted.

**2.** Which help-desk output is natural language generation (NLG)?

- **A** The intent label *access*.
- **B** The extracted course code.
- **C** Written step-by-step instructions for regaining access.
- **D** A relevance judgement for a passage.

**3.** In the rule-based era, what did engineers mainly specify?

- **A** Patterns, dictionaries, and grammar.
- **B** A training objective and a network architecture.
- **C** Prompts for a pre-trained model.
- **D** Learned word embeddings.

**4.** When measuring accuracy, how should the reference labels be produced?

- **A** Copied from the model's predictions.
- **B** Assigned after reviewing the model's output.
- **C** Taken from the most confident model.
- **D** Assigned by annotators under a written policy before seeing any model output.

**5.** A router labels 7 of 10 tickets correctly. Always predicting the most frequent label scores $4/10$. What is $0.40$?

- **A** The error rate.
- **B** The majority-class baseline.
- **C** The router's recall.
- **D** The 95% margin.

6. Of the tickets a model flagged as urgent, which measure gives the fraction that really were urgent?
   - **A** Recall.
   - **B** Accuracy.
   - **C** Precision.
   - **D** Error rate.

7. In 1000 tickets, 100 are urgent. A model predicts every ticket as non-urgent. What is its accuracy?
   - **A** 0.90.
   - **B** 0.10.
   - **C** 0.
   - **D** 1.00.

8. A student speaks a request; the system transcribes it, routes the text, and reads a reply aloud. What are the input and output modalities of the whole system?
   - **A** Text to text.
   - **B** Speech to speech.
   - **C** Speech to text.
   - **D** Text to speech.

9. Why can an evaluation on edited Modern Standard Arabic fail to support claims about the help desk?
   - **A** Arabic has no written standard.
   - **B** Accuracy is undefined for Arabic.
   - **C** Modern Standard Arabic cannot be tokenized.
   - **D** Students also write in dialect, Arabizi, and code-switched text that it does not contain.

## Check your understanding

**Q1.** How do NLU and NLG differ in the course-help-desk example? Name an output and a possible failure for each.

**Q2.** What changes when "big" becomes "small" in the trophy–suitcase example? Name that kind of ambiguity, then give one pragmatic and one morphological example from the chapter.

**Q3.** Which engineering decisions remain after replacing rules with a pre-trained model?

**Q4.** Why can accuracy hide failure on urgent tickets, and which baseline should be reported beside it?

**Q5.** A voice help desk chains recognition, routing, and speech synthesis. Why measure each stage *and* the end-to-end result?

**Q6.** Why should an Arabic task specify dialect and diacritic policy? Name a help-desk input that an edited Modern Standard Arabic evaluation could miss.

## Work it through

**E1.** Write a one-paragraph task contract for a course help desk adapted to your campus: input and available context, routing output, multi-intent and clarification policy, reference labels, language varieties, important error, metric, proposed target, and fallback. Include one ambiguous example and the question needed to clarify it. Justify a rule-based baseline or reusable model as a first candidate; do not assume that a newer model is better.

**E2.** Measure a router yourself. Ten held-out tickets have these reference (Ref) and predicted (Pred) intents, where A = access, S = assessment, R = registration, and O = other:

| Ticket | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Ref | A | A | A | S | S | R | R | O | A | S |
| Pred | A | A | A | S | A | R | S | O | A | S |

The most frequent label in the training tickets was A. Mark each ticket right or wrong, then compute accuracy and the majority baseline on these tickets. Build the $4 \times 4$ confusion matrix, and calculate recall for each class.

**E3.** Recalculate precision, recall, F1, and accuracy for 80 true positives, 20 false negatives, 60 false positives, and 840 true negatives. Compare with Example 1.2. Explain how equal accuracy can conceal different missed-ticket and false-alarm costs.

**E4.** Design a help-desk scenario that uses text, speech, and an image, such as a spoken request about a screenshot. Name the stages and their input/output modalities, an error that can cross an interface, and one end-to-end test with an expected result. Compare a pipeline with a direct multimodal model and name the evidence needed to choose.

## Read the picture

**V1.** Redraw Figure 1.7 for a classifier that still finds 60 urgent tickets but flags only 20 ordinary ones. Which measures change?

**V2.** In Figure 1.9, where would all 1000 tickets go for the majority baseline that predicts “not urgent” every time?

## Check your progress

Use the objective identifiers from the opening list. Try the linked practice before reading its optional guidance.

**Table 1.5** From Chapter 1 objectives to evidence of learning

| Goal | Preparation | Your evidence |
|---|---|---|
| O1 | Example 1.1: routing and advice. | Q1 and E1: distinguish the two contracts. |
| O2 | Context and the ambiguity examples. | Quiz 1; Q2 and E1: name the kind of ambiguity, missing context, and a clarification. |
| O3 | Table 1.1 and the ten landmarks. | Q3 and E1: justify a baseline or reusable model. |
| O4 | Task families, the ten-ticket measurement, and worked evaluation counts. | Q4 and E2–E3: measure, calculate, and interpret evidence. |
| O5 | Modality table and Example 1.3. | Q5 and E4: name modalities and the stages of a pipeline. |
| O6 | Example 1.4 and Table 1.4: the worked contract and its scoring. | Q6 and E1: specify language varieties and evaluation. |

## Further reading

- **Early landmarks.** Read Turing's 1950 paper, IBM's account of the 1954 demonstration, and Weizenbaum's 1966 ELIZA paper. Distinguish a demonstration of selected behaviour from a claim of general understanding.
- **Learning from data.** Brown and colleagues' 1993 statistical translation paper, Bengio and colleagues' 2003 neural language-model paper, and Mikolov and colleagues' 2013 word-representation paper connect probability estimates to learned representations.
- **Architecture and reuse.** The Transformer paper, BERT paper, and instruction-following study supply primary sources for architecture, pre-training, and adaptation. Their training objectives differ; detailed model internals are later study.
- **Arabic language processing.** The CAMeL Tools paper describes Arabic processing components. Use its task and analyser distinctions when planning an Arabic application.

CHAPTER 2

# Rules and Patterns

IN THIS CHAPTER

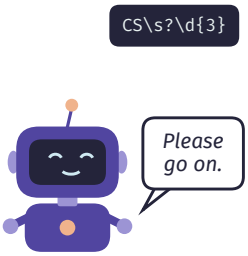

2.1 Regular expressions as written policies 28
2.2 Arabic patterns, step by step 43
2.3 ELIZA: dialogue from patterns 47
2.4 Review and practice 51


**LEARNING OBJECTIVES**

After working through this chapter, you should be able to:

- **O1.** write regular expressions with classes, quantifiers, anchors, and groups, using Unicode ranges for Arabic text;
- **O2.** use backreferences, lookaround, and substitution to clean text and extract values;
- **O3.** state Arabic normalisation and tokenization rules as written policies, and test each one against the distinctions it must keep;
- **O4.** explain how ELIZA's keywords, decomposition patterns, reflection, and reassembly templates produce a reply, and why Arabic morphology breaks naive reflection;
- **O5.** evaluate a rule-based component with precision and recall, and record its known failures instead of hiding them.

Chapter 1 placed regular expressions and ELIZA at the start of the field's history. This chapter shows how they work. Regular expressions are still the right tool when text has a fixed format, such as a course code or a price. ELIZA shows how a smooth reply can look like understanding.




**The goal of this chapter** is to build Sanad's rule layer (normalisation, routing, slot extraction, and a clarifying reply) and then to measure exactly where the rules break. Every technique is applied to Arabic help-desk text. Choices that look small in English (where a word ends, which spellings are the same, how "my" is written) become real design decisions in Arabic.

Follow one message through the chapter: “When is the exma for CS120?”. First we *match* it: a regular expression finds the course code, but no exact rule expects the misspelt word (Section 2.1). Then we *reply* to it: ELIZA answers fluently but routes nothing (Section 2.3). Figure 2.1 shows the route. The typo waits for Chapter 3, which counts what such messages contain and measures how far apart two spellings are. Labs 1 and 2 turn this chapter into tested code.

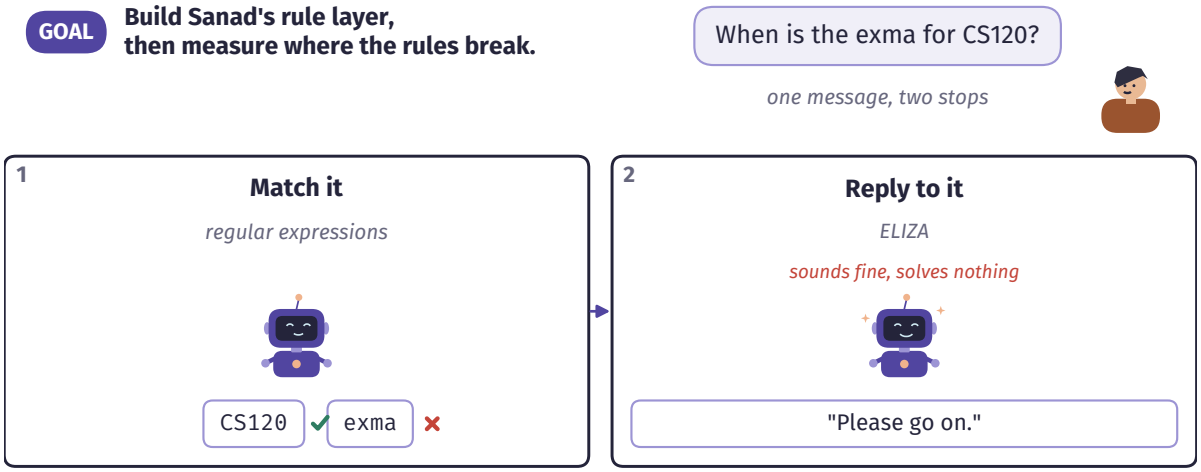



**Figure 2.1** Chapter 2’s goal and route. One help-desk message passes two stops: a pattern finds the course code but misses the misspelt word (Section 2.1), and ELIZA answers fluently without routing anything (Section 2.3). The typo waits for Chapter 3.

## 2.1 Regular expressions as written policies

A **regular expression** is a pattern that describes a set of strings. Kleene introduced the notation in the 1950s. Thompson built it into text editors, and from there it spread to `grep` and to every major programming language.

In Python, the `re` module does the work:

- `re.search` finds the first match anywhere in the text;
- `re.findall` returns every match that does not overlap the one before; if the pattern has *capturing* groups `(…)`, it returns what they captured instead of the whole match (groups written `(?:…)` do not change this);
- `re.fullmatch` succeeds only if the *whole* string matches;
- `re.sub` replaces matches with new text.

Write patterns as raw strings, such as `r"\d+"`, so Python leaves the backslashes alone. The operation matters as much as the pattern. `re.search(r"[0-9]{3}", "12345")` finds `123`, while `re.fullmatch` with the same pattern rejects `12345` because it has five digits.

A pattern is an exact rule that anyone can read and check. That is its strength. It is also its weakness: it fails on any input its author did not think of.

**Table 2.1** Core regular-expression building blocks. Table 2.3 adds the rest in Section 2.1.5.

| Syntax | Meaning | Example |
|---|---|---|
| `abc` | The literal characters, in order. | `exam` matches inside `exams`. |
| `[abc]`, `[a-z]` | One character from a set or a code-point range. | `[A-Z]` is one capital letter. |
| `[^abc]` | One character *not* in the set. | `[^0-9]` is any non-digit. |
| `. \d \w \s` | Any character except newline by default; a digit; a word character; whitespace. | In Python 3 these classes are Unicode-aware (see below). |
| `\D \W \S` | The complements: a non-digit; a non-word character; a non-space. | `\D+` is a run with no digits. |
| `* + ?` | Zero or more; one or more; optional (zero or one). | `EGP ?` allows one optional space. |
| `{n} {m,n} {n,}` | Exactly $n$; between $m$ and $n$; at least $n$ repetitions. | `[0-9]{3}` is exactly three digits. |
| `\. \? \$ \*` | An escaped special character matches itself. | `150\.50` needs a real dot; `150.50` also matches `150x50`. |
| `^ $ \b` | Start of string; end of string; a word boundary. | `^I am` matches only at the start. |
| `(…) (?:…)` | A capturing group; a group that does not capture. | `(\d+)/(\d+)` captures day and month. |
| `a\|b` | Alternation: either side. | `exam\|test`. |

Three entries in the table need a closer look:

- **`\b` is not a real word boundary.** It sits between a `\w` character and a `\W` character, or at the edge of the string. Since `\w` includes digits and underscore, `\b` sees `CS_101` as one "word".
- **`$` can match before a final newline.** With `re.MULTILINE`, `^` and `$` also match at each line. To check a whole field, use `fullmatch`.
- **The caret has two jobs inside brackets.** As the first character it negates the set: `[^0-9]` is a non-digit. Anywhere else it is an ordinary character: `[0-9^]` is a digit or a caret. Outside brackets it means "start of string".

Quantifiers such as `+` and `?` are **greedy**: they take as much text as they can, and give some back only if the rest of the pattern would otherwise fail. Optional groups behave the same way. Later in this section, this causes an Arabic failure.

**Operator precedence.** When a pattern has no parentheses, its parts are grouped in a fixed order, from strongest to weakest:

parentheses > counters `* + ? {}` > sequences and anchors > alternation `|`

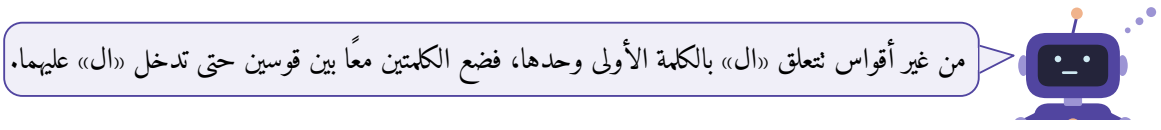
Alternation `|` is the weakest, so it splits the *whole* pattern into two halves, not just the nearest word.

Here is the trap in Arabic. You want الكتاب (*al-kitab*, "the book") or الكتب (*al-kutub*, "the books"). You write the article ال, then كتاب | كتب. The pattern actually means "الكتاب, or a bare كتب". So:

- with `fullmatch`, it rejects الكتب and accepts كتب without the article;
- with `search`, it finds كتب inside الكتب and leaves the article outside the match.

The fix is to group the choice: the article, then `(?:كتاب | كتب)`. In the same way, a counter applies only to the one item just before it: `ha+` repeats only the *a*, while `(?:ha)+` repeats the pair. Figure 2.2 draws both versions of the article pattern.

> **CHECK THE CODE** `examples/ch02/regex_policies.py` – tests both versions of the article pattern.




> **TIPS**
>
> When in doubt, add parentheses. A group `(?:…)` groups without saving anything, and it shows the next reader what you meant.

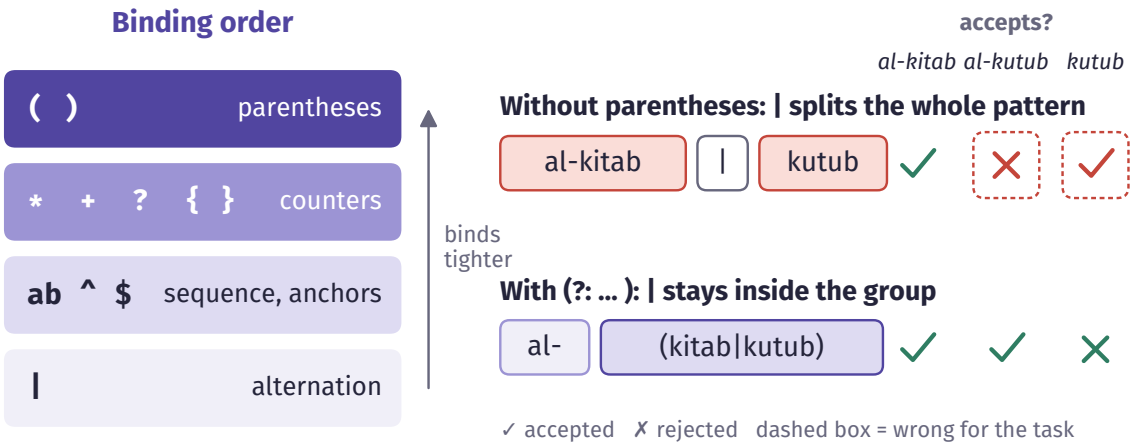



**Figure 2.2** Alternation binds last. Without parentheses the pattern means الكتاب (*al-kitab*) or a bare كتب (*kutub*); with a non-capturing group the alternation stays inside the group.

### 2.1.1 Arabic text is a sequence of code points

A pattern works on Unicode code points, not on the shapes a reader sees. Arabic letters join and change shape by position, but the stored text holds one code point per letter. Short vowels (*harakat*), tanween, shadda, and sukun are separate *combining* code points that follow their letter: a fully vocalised word is stored letter, mark, letter, mark (Figure 2.3). Table 2.2 lists the ranges this chapter uses.

**Table 2.2** Arabic code-point ranges used by the patterns and labs

| Code points | Content |
|---|---|
| U+0621–U+063A, U+0641–U+064A | Letters, including the hamza forms (ء أ إ ؤ ئ), taa marbuta (ة, U+0629), and alef maqsura (ى, U+0649). |
| U+064B–U+0652, U+0670 | Tanween (ً ٌ ٍ), short vowels (َ ُ ِ), shadda (ّ), sukun (ْ); dagger alef (ٰ). |
| U+0640 | Tatweel (ـ, *kashida*), a stretching stroke that changes only how a word looks. |
| U+0622, U+0623, U+0625, U+0671 | Alef with madda (آ), hamza above (أ), hamza below (إ), and wasla (ٱ). |
| U+0660–U+0669; U+06F0–U+06F9 | Arabic-Indic digits; extended (Persian/Urdu) digits. |
| U+060C, U+061B, U+061F | Arabic comma (،), semicolon (؛), and question mark (؟). |

Python's built-in classes are Unicode-aware. This helps in one place and hurts in another:

- **Helps:** `\d` matches Arabic-Indic digits too. `re.findall(r"\d+", …)` finds the room number in "room 204" and in its Arabic-Indic spelling.

- **Hurts:** \w matches Arabic letters but *not* combining marks. On the fully vocalised verb يَدْرُسُونَ (*yadrusuna*, "they study"), \w+ returns five fragments instead of one word, because every short vowel ends a match.

So an Arabic tokenizer should either list its character ranges explicitly or remove the marks first. Listing 2.1 does the second.

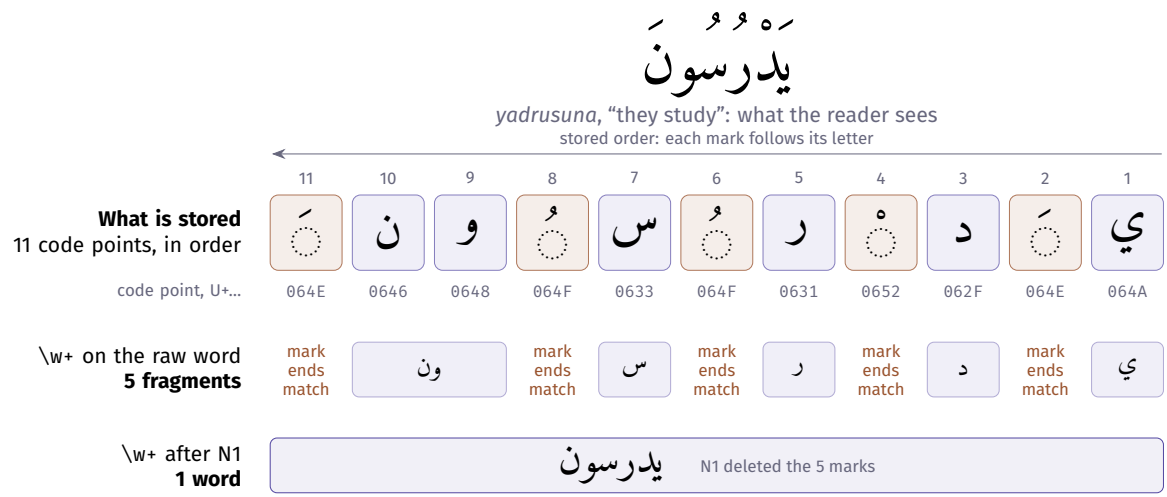
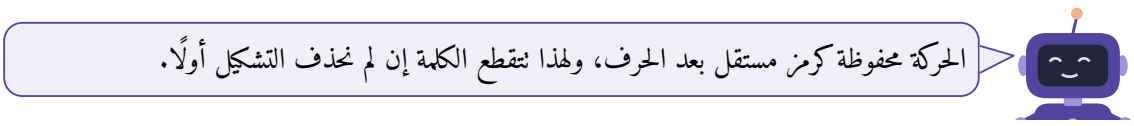



**Figure 2.3** The marks are stored as characters. The reader sees one word, يَدْرُسُونَ; the text holds 11 code points, and each mark (copper, drawn on a dotted circle) comes straight after its letter. \w matches the letters but not the marks, so each mark ends a match and \w+ returns five fragments. After N1 deletes the five marks, it returns one word.




**Listing 2.1** Policies N1–N3 of Lab 1, written with explicit code points (examples/ch02/regex_policies.py)

```
DIACRITICS = re.compile(r"[\u064B-\u0652\u0670]")
TATWEEL = re.compile(r"\u0640")
ALEF_VARIANTS = re.compile(r"[\u0622\u0623\u0625\u0671]")

def remove_diacritics(text):      # N1
    return DIACRITICS.sub("", text)

def remove_tatweel(text):         # N2
    return TATWEEL.sub("", text)

def normalize_alef(text):         # N3
    return ALEF_VARIANTS.sub("\u0627", text)
```

**Reading the listing.** Each policy is one compiled pattern plus one sub call.

- **N1** deletes every code point in U+064B–U+0652, plus the dagger alef U+0670. These are the marks of Table 2.2. After N1, `\w+` returns يدرسون as one word.
- **N2** deletes the tatweel (ـ, U+0640), so a stretched word and its plain form become the same string.
- **N3** replaces any of the four alef forms (آ أ إ ٱ) with bare alef (ا, U+0627). A user who types احمد without hamza now matches أحمد (*Ahmad*) with hamza.

Writing `\u064B` instead of the invisible mark itself makes the rule readable and easy to review.

These policies assume that each hamza letter is stored as one code point. Some keyboards and tools instead store `3260` as bare alef followed by a combining hamza (U+0654); U+0653 and U+0655 are the combining madda and hamza below. N1 and N3 leave such pairs alone, so `F2605460D2603260` stored this way never matches `F2605460D2607260`. Apply Unicode NFC first, which joins each pair into one letter (Section 4.1.4).

### 2.1.2 Normalisation is a choice for a task

Figure 2.4 shows all eight policies of Lab 1. The first five clean the text; the last three cut it into units and pull out what a task needs:

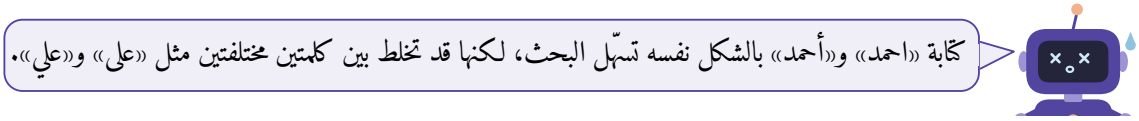
- **N1** deletes the short vowels and other marks: يَدْرُسُونَ → يدرسون;
- **N2** deletes the tatweel (ـ): الطبيعـة → الطبيعة;
- **N3** maps the four alef forms (آ أ إ ٱ) to bare alef (ا): إسلام → اسلام;
- **N4** maps alef maqsura (ى) to ya (ي), and taa marbuta (ة) to haa (ه): مدرسة علي → مدرسه علي;
- **N5** converts Arabic-Indic digits (٠١٢٣٤٥٦٧٨٩) to 0–9: ٢٠٤ القاعة → 204 القاعة;
- **N6** splits the text into tokens: Arabic words, Latin or Arabizi words, and digit runs (Example 2.1);
- **N7** extracts numbers in either script (N7a) and course codes in one canonical form (N7b): `CS ١٢٠` → `CS120`;
- **N8** strips the clitics (و ف ب ك ل ال) from the front of a word, if at least three letters remain (Section 2.1.3).

N1–N5 can make different words look the same. After N1, the unvocalised علم can be عِلْم (*3ilm*, science), عَلَم (*3alam*, flag), or عَلِمَ (*3alima*, he knew); Figure 4.13 in Chapter 4 illustrates this. N4 merges the preposition على (*3ala*, on) with the name علي (*3ali*).

Whether that is acceptable depends on the task. A search engine can accept it: a user who types without hamza still expects to find the document. A system that reads text aloud, or adds the vowel marks, cannot. Order matters too. Run N1 *before* any tokenizer that uses `\w`, or the marks will split words.

| Rule | Pattern or operation | Input | Output | What changes |
|---|---|---|---|---|
| N1 | `[\u064B-\u0652\u0670]` → `""` | يَدْرُسُونَ | يدرسون | Short vowels, tanween, shadda and sukun are deleted. |
| N2 | `\u0640` → `""` | الطبيعـــة | الطبيعية | Only the presentation stretch goes. |
| N3 | `[\u0622\u0623 \u0625\u0671]` → ا | إسلام | اسلام | Hamza and madda distinctions are merged. |
| N4 | maqsura ى to ya ي<br>taa marbuta ة to haa ه | مدرسة على | مدرسه علي | Two common spelling variants are merged. |
| N5 | `str.translate` with a digit table | القاعة ٢٠٤ | القاعة 204 | Arabic-Indic digits become ASCII. |
| N7b | `\b([A-Z]{2,4})\s? ([0-9\u0660-\u0669]{3})\b` | CS ١٢٠ | CS120 | One canonical form for search. |
| N8 | optional و or ف<br>then optional ب or ك or ل<br>then optional ال<br>keep a stem of 3+ letters | والطلاب | طلاب | Conjunction and article removed. |
| N8 | same | وصل | وصل | Kept: و is a root letter here. |
| N8 | same (known failure) | للطلاب | لطلاب | Not modelled: لل is ل plus ال. |
| N8 | same (known failure) | فكتب | فكتب | Greedy match takes ك too, so the stem is too short. |
| **N6 tokenize** | مايو! ١٥ deadline والـ بكرة، 3ayez a5ls el project<br>→ 3ayez · a5ls · el · project · بكرة · والـ · deadline · ١٥ · مايو | | | |

**Figure 2.4** Lab 1 policies applied to Arabic input. Each row is one written rule. Shaded rows are known failures, which the lab records as `xfail` tests rather than hiding.




**Example 2.1**

**A mixed Egyptian request.** A student writes *3ayez a5ls el project bokra* ("I want to finish the project tomorrow"). The message continues in Arabic script, with an English loanword and an Arabic-Indic date. Try three simple tokenizers:

- **Split on spaces.** The Arabic comma stays attached to a word.
- **A Latin-only pattern** such as `[a-z']+`. Every Arabic word is lost.
- **An Arabic-only pattern.** *3ayez* is lost, because it uses the digit 3 as a letter (Arabizi).

Each tokenizer fails because it assumes one script. Lab 1's tokenizer N6 instead tries three alternatives, in order:

1. an Arabic word (letters, marks, and tatweel);
2. a Latin or Arabizi word (letters and digits, with at least one letter);
3. a digit run in any script.

The "at least one letter" condition is what keeps *3ayez* as a word but the date as a number. The bottom row of Figure 2.4 shows the nine tokens N6 returns.

### 2.1.3 The clitic trap: greedy rules and root letters

Arabic attaches conjunctions (و *wa-*, ف *fa-*), prepositions (ب *bi-*, ك *ka-*, ل *li-*), and the article ال *al-* to the next word. Removing them with one pattern is tempting:

```
CLITIC_PREFIX = re.compile(r"^(?:[\u0648\u0641])?(?:[\u0628\u0643\u0644])
    ?(?:\u0627\u0644)?")
```

Read it left to right: an optional و or ف, then an optional ب, ك, or ل, then an optional ال. Policy N8 applies this pattern only if at least three letters remain.

Two cases work:

- والطلاب (*wa-l-tullab*, "and the students") becomes طلاب.
- وصل (*wasala*, "he arrived") is left alone, because removing the و would leave only two letters.

Three cases fail:

- **للطلاب** (*lil-tullab*, "for the students"). ل followed by ال contracts to لل, which the pattern does not describe.

- فكتب (*fa-kataba*, “so he wrote”). The optional preposition group is greedy, so it also takes the root letter ك. Only two letters remain, and N8 refuses the whole split.
- بحثنا (*bahthuna*, “our research”). The ب is a root letter, but the rule strips it as if it were the preposition ب.

A pattern sees only letters. It cannot know whether a letter is a clitic or part of the root. For that you need a word list or a morphological analyser such as CAMeL Tools or Farasa. Lab 1 keeps all three cases as *expected failures*, a written record of the rule’s limits. Figure 2.5 shows the rule at work on three of Nour’s words.

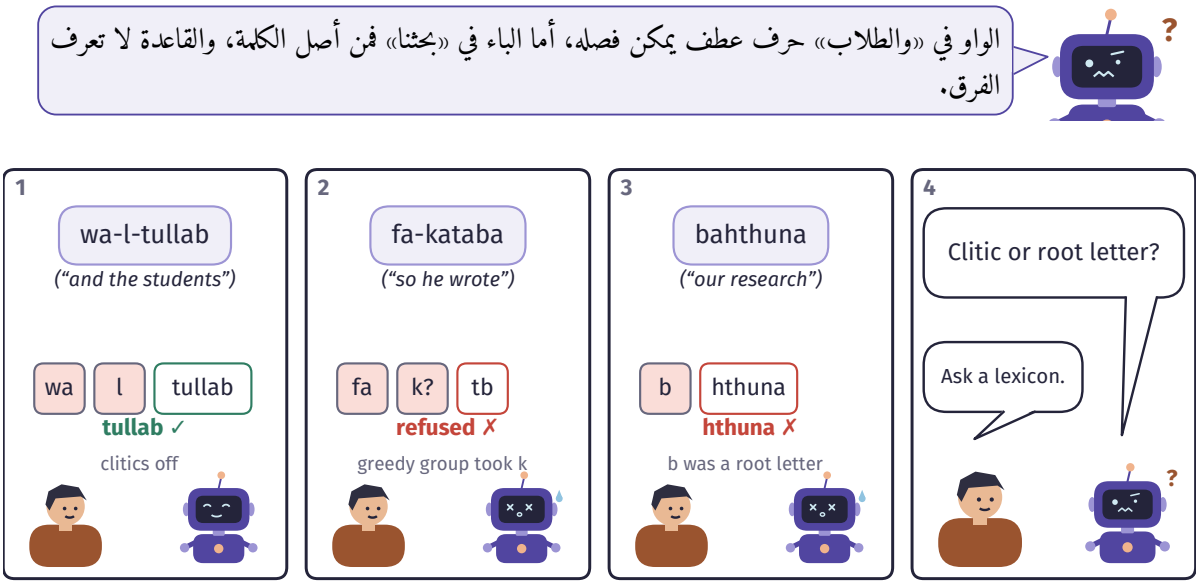



**Figure 2.5** Policy N8 on three of Nour’s words. It strips و and ال from والطلاب (*wa-l-tullab*) correctly, refuses فكتب (*fa-kataba*) because the greedy preposition group also takes the root ك, and strips the root ب of بحثنا (*bahthuna*). A string rule cannot tell a clitic from a root letter.

### 2.1.4 Evaluate a pattern like any classifier

A course-code pattern works like a classifier: each piece of text it returns (a *span*) is a claim “this is a course code”, so precision and recall from Chapter 1 apply. Here a span is correct only if it starts and ends exactly where a true code does, and each true code can be matched only once.

- A **false positive** is a span the pattern returns but should not.
- A **false negative** is a wanted span the pattern misses.

A partly correct span counts as two errors: the short span is a false positive, and the full span it missed is a false negative. Figure 2.6 shows all three outcomes in one message.

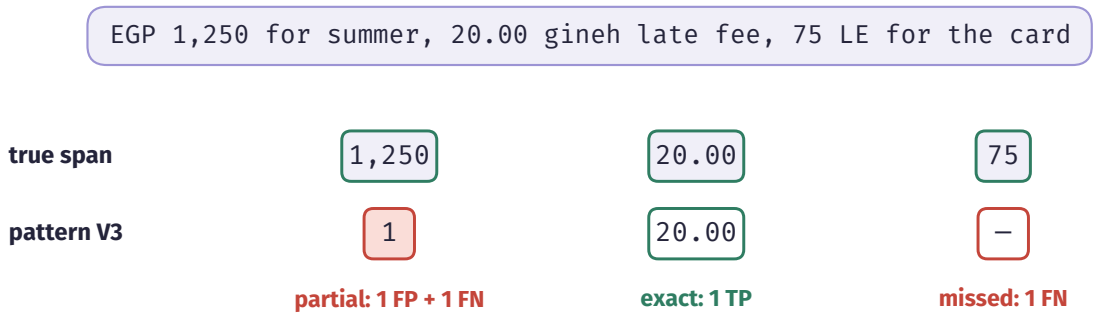



**Figure 2.6** Exact-span scoring of price pattern V3 on one fee message. A span is correct only if it starts and ends where the true span does. The partial match `1` is a false positive *and* leaves `1,250` as a false negative, so this message scores precision $1/2$ and recall $1/3$.

**Example 2.2**

**Course codes in constructed messages.** Thirteen constructed messages contain 20 true course codes. The pattern

```
\b[A-Z]{2,4}\s?[0-9]{3}\b
```

reads "two to four capitals, an optional space, three ASCII digits, all between word boundaries". It returns 18 spans, and 16 are correct.

**Why the 2 false positives?** `ROOM 204` and `HALL 110` have exactly the shape the pattern describes. Shape alone cannot tell a room from a course.

**Why the 4 false negatives?** Each one breaks a part of the pattern:

- `cs101` is lower case, but the pattern asks for `[A-Z]`;
- `CS-101` has a hyphen, but the pattern allows only a space;
- two codes use Arabic-Indic digits, but `[0-9]` is ASCII only.

So precision is $16/18 \approx .889$ and recall is $16/20 = .80$.

**What would fix it?** Using `\d` fixes the Arabic-Indic cases. Allowing a hyphen and lower case raises recall, but more look-alikes get through, so precision may fall. Rejecting `ROOM` needs a list of words to exclude, or a look at the nearby words. Always report both measures, and say which messages you used.

**CHECK THE CODE** `examples/ch02/regex_policies.py` – holds the thirteen messages and checks every count.

A pattern is rarely right the first time. The usual loop is:

1. write a version;
2. list its false positives and false negatives;
3. change the pattern to fix one named error;

4. measure again.

Figure 2.7 shows the loop as Nour, a student, and Sanad, the help-desk bot, meet it.

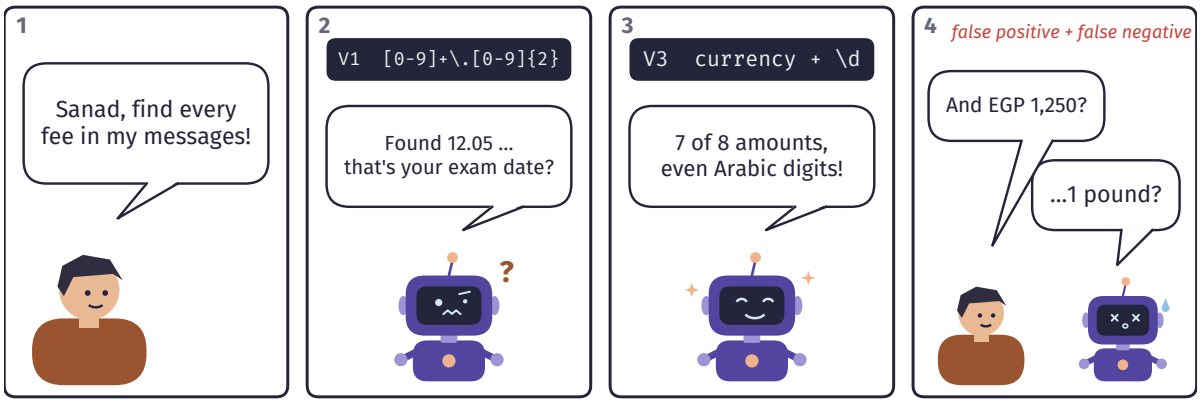



**Figure 2.7** Sanad's fee patterns, version by version: V1 takes an exam date for a fee, V3 finds most amounts, and one misread thousands separator counts twice, as a false positive and a false negative.

**Example 2.3**

**Refining a price pattern.** Eleven constructed fee messages quote Egyptian pounds in three ways: as جنيه (*gineh*), as the abbreviation ج.م (جنيه مصري, *gineh masri*), or as `EGP`. Some write the amount in Arabic-Indic digits. Together they contain eight true amounts. Listing 2.2 shows three versions; each one fixes the errors of the one before.

- **V1: "digits, a dot, two digits".** It returns 4 spans and 2 are correct: precision .50, recall .25. The false positives are a date (*12.05*) and an app version (*2.10*): they have the shape of a price but no currency. The false negatives are all whole-pound amounts, such as جنيه ٧٥, which have no dot.
- **V2: require a currency word.** The currency may come before (`EGP`) or after (جنيه, ج.م) the number. The piastres `(?:\.[0-9]{2})?` become optional, and `\b` stops a match from starting or ending inside a longer number. It returns 6 spans and 5 are correct: precision $5/6 \approx .833$, recall $5/8 = .625$. It still misses the two Arabic-Indic amounts, because `[0-9]` is ASCII only.
- **V3: replace `[0-9]` with `\d`.** Nothing else changes. It returns 8 spans and 7 are correct: precision and recall are both .875.

**The one error left** is `EGP 1,250`. V3 returns only `1`, because `\b` finds a boundary between the `1` and the comma. One mistake therefore counts twice: the span `1` is a false positive, and the true amount `1,250` is a false negative. Exercise E4 asks for the next version.

**CHECK THE CODE** `examples/ch02/regex_policies.py` – holds the eleven fee messages and checks all three versions.

**Listing 2.2** Three versions of the price pattern. The \u escapes spell جنيه and ج.م (examples/ch02/regex_policies.py)

```
CURRENCY = r"(?:\u062c\u0646\u064a\u0647|\u062c\.\u0645)"        # gineh | g.m.
PRICE_V1 = re.compile(r"[0-9]+\.[0-9]{2}")
PRICE_V2 = re.compile(r"EGP ?([0-9]+(?:\.[0-9]{2})?)\b|\b([0-9]+(?:\.[0-9]{2})?) ?" + CURRENCY)
PRICE_V3 = re.compile(r"EGP ?(\d+(?:\.\d{2})?)\b|\b(\d+(?:\.\d{2})?) ?" + CURRENCY)
```

**Reading the listing.** CURRENCY lists the two written forms, جنيه or ج.م; the dot is escaped so it means a real dot. V2 and V3 each have two halves joined by |: "EGP, then a number" or "a number, then CURRENCY". Each half saves the number in its own group, so the code returns the amount without the currency word. The only difference between V2 and V3 is [0-9] → \d.

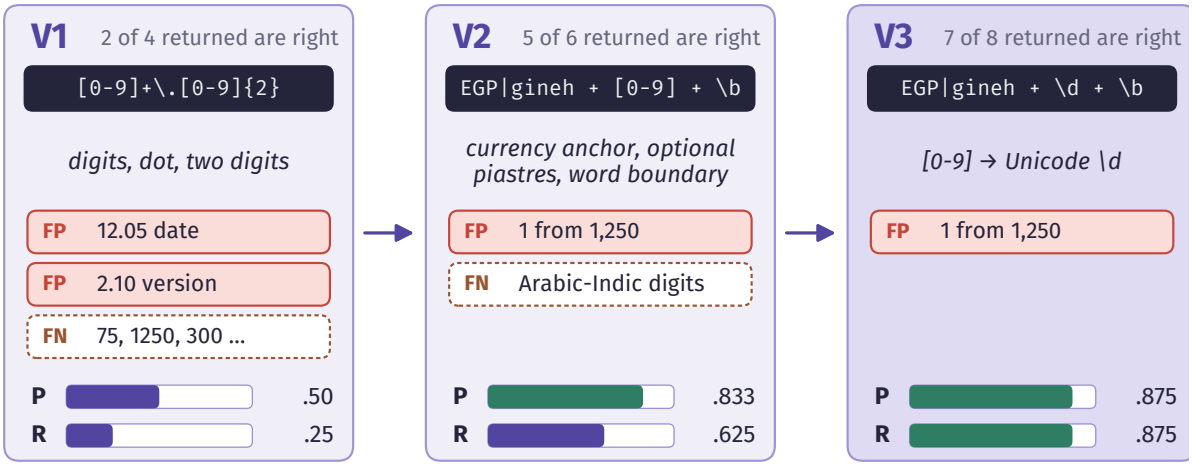



**Figure 2.8** Each version of the price pattern fixes a named error. Red chips are false positives, dashed chips are false negatives, and the bars show precision (P) and recall (R) on the eight true amounts of Example 2.3.

**KEY POINT**

A regular expression is an exact rule you can test. Keep a table of inputs it must accept, inputs it must reject, and known failures. Measure precision and recall on messages you did not use to write it.

نجاح القاعدة في الأمثلة التي كتبتها لا يعني الكثير؛ جرّبها على رسائل جديدة لم ترها من قبل.



### 2.1.5 Substitution, capture, and lookaround

Table 2.1 was enough to write and test patterns. This section adds the rest of the syntax, in Table 2.3, one idea at a time.




**Table 2.3** Further building blocks, introduced in this section

| Syntax | Meaning | Example |
|---|---|---|
| `\1 \2` | A backreference: the text that group 1 (or 2) matched, again. | `(\w)\1{2,}` is one word character typed three or more times. |
| `(?=…) (?!…)` | Lookahead: what follows does (or does not) match; it is checked, not taken. | ال only when three letters follow. |
| `(?<=…)` `(?<!…)` | Lookbehind: what comes before does (or does not) match. | `(?<=EGP )\d+` returns the digits without `EGP`. |

A *substitution* finds a pattern and replaces each match. Python writes it as

```
re.sub(pattern, replacement, text)
```

Unix tools such as `sed` write the same idea as `s/pattern/replacement/`.

Besides grouping, parentheses **capture**: the text a group matched is saved in a numbered slot, a *register*, counted by opening parentheses from the left. The replacement writes register 1 as `\1`. For example, `re.sub(r"(\d+)", r"<\1>", text)` turns `35` طلب (*talab*, "order 35") into `<35>` طلب: the group captures `35`, and the replacement writes it back inside angle brackets.

Each tool has its own regex dialect. For example, `sed` does not usually understand `\d`, so check a Python pattern before you paste it into a shell command.

**Backreferences.** A **backreference** such as `\1` can also be used inside the pattern. There it means "the same text again". Egyptian chat stretches words for emphasis by repeating a letter, as in *gamiiiil* ("beautiful"). The pattern `(\w)\1{2,}` matches one word character followed by at least two more copies of it. Step by step on *gamiiiil*:

1. `(\w)` captures the first *i*;
2. `\1{2,}` matches the next three *i*s, so the whole run *iiii* matches;
3. replacing the run with `\1` leaves one *i*: *gamil*.

The same rule works in Arabic script on a run of ي: جمييييل becomes جميل. Only runs of three or more change, so Arabizi *gameel*, with two *e*s, survives.

The rule can also do harm, and the script records each case:

- Laughter written as a run of *h* collapses to one *h*. A task that counts laughter must exempt it.

- `\w` includes digits, so `1000` would become `10`. Apply the rule only to suitable text, or use an explicit letter class.

This rule and N2 work together. N2 removes stretching drawn with tatweel; this rule removes stretching made by typing a letter again. Figure 2.9 shows both outcomes.

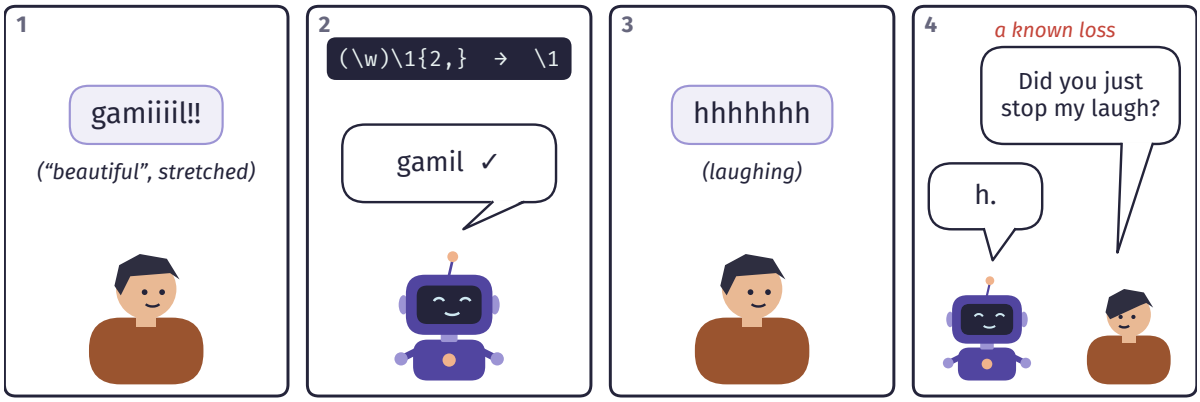



**Figure 2.9** Collapsing stretched letters with a backreference repairs Nour's *gamiiiil*, but the same rule flattens a laugh to a single *h*: a fix with a known loss.

If you only need to group, use `(?:…)`: it groups without saving. Only saving groups get numbers, so adding a `(?:…)` group never changes what `\1` means. The clitic pattern in Section 2.1.3 uses this kind of group.

**Lookaround.** A **lookaround** checks the text next to the match, but does not include that text in the match.

- **Lookahead** `(?=…)` succeeds if the text that follows matches; `(?!…)` succeeds if it does not. Policy N8 checks "at least three letters remain" as a separate step. A lookahead puts that check inside the pattern: the article followed by `(?=[\u0621-\u063A\u0641-\u064A]{3})`, the letter ranges of Table 2.2. It strips ال from الطلاب (*al-tullab*, "the students"), but leaves الله (*Allah*) alone, because only two letters follow.
- **Lookbehind** `(?<=…)` or `(?<!…)` tests the text *before* the match. In رقم 4589 والمبلغ 300 (*raqam 4589 wa-l-mablagh 300*, "number 4589 and the amount 300"), `(?<=رقم )\d+` returns only the order number, `4589`. The word رقم is checked but not returned.

Figure 2.10 sets the three mechanisms side by side.



**TIPS**

Python requires a lookbehind to have a fixed width. `(?<=رقم )\d+` works, but putting `\s*` inside the lookbehind raises an error. When the prefix varies, match it normally and capture the part you want, as in `رقم\s*(\d+)`.

**Listing 2.3** Substitution, a backreference, and lookaround (`examples/ch02/regex_policies.py`)

```
NUMBER = re.compile(r"(\d+)")
def mark_numbers(text):                    # s/(\d+)/<\1>/
```

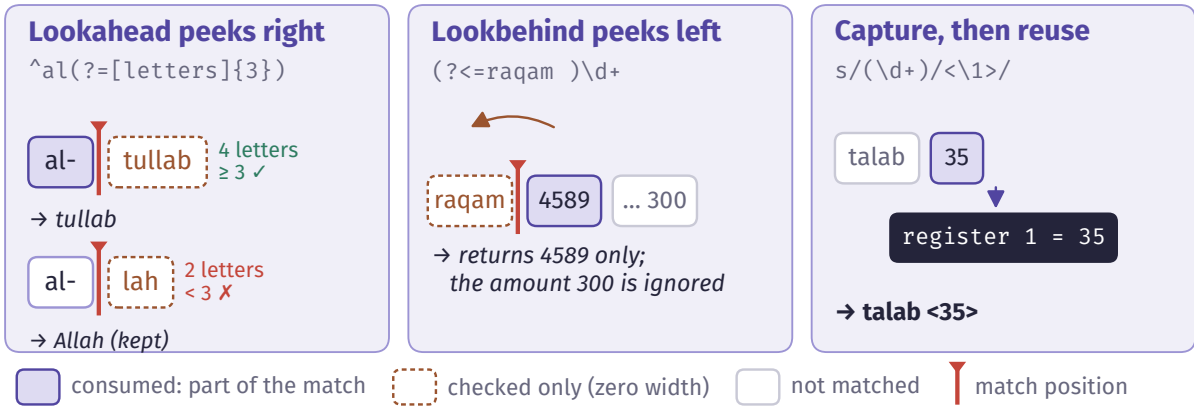



**Figure 2.10** Lookaround checks text without consuming it. Solid boxes are part of the match; dashed boxes are only checked. Left: *al-* is stripped only when at least three letters follow. Middle: the lookbehind returns the order number after *raqam*. Right: a capture group stores 35 in register 1, and the replacement reuses it.

```
    return NUMBER.sub(r"<\1>", text)

ELONGATION = re.compile(r"(\w)\1{2,}")     # one word character typed 3+ times
def collapse_elongation(text):
    return ELONGATION.sub(r"\1", text)

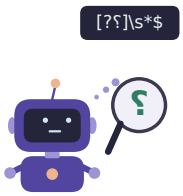
ARTICLE = re.compile(r"^\u0627\u0644(?=[\u0621-\u063a\u0641-\u064a]{3})")
    # al- + 3 letters
def strip_article(word):
    return ARTICLE.sub("", word)

AFTER_RAQAM = re.compile(r"(?<=\u0631\u0642\u0645 )\d+")       # digits after "raqam "
```

**Reading the listing.** The four patterns are the four ideas of this subsection, one each:

- `NUMBER` and `mark_numbers`: capture a digit run, then write it back as `<\1>`.
- `ELONGATION` and `collapse_elongation`: a backreference inside the pattern, then `\1` in the replacement to keep one copy.
- `ARTICLE`: `^` means "start of the word"; the lookahead checks that three letters follow; `sub` deletes only ال, because the lookahead looked at the letters without taking them.
- `AFTER_RAQAM`: a fixed-width lookbehind, so `findall` returns digits only.

## 2.2 Arabic patterns, step by step




In this section, we work through six complete examples, W1–W6, taken from Sanad's inbox. Each one is laid out the same way: the *task*, a first *try*, the *problem* a test message reveals, the *fix*, and what *still fails*. Cover each fix and predict it before you read it. W1–W3 are on the core route. W4–W6 chain several policies in a fixed order and belong to the challenge route; their headings carry the CHALLENGE badge. Figure 2.11 summarises the six patterns with their test messages.

> **CHECK THE CODE** `examples/ch02/arabic_patterns.py` – checks every result of W1–W6.

### W1. A word, not part of a word.

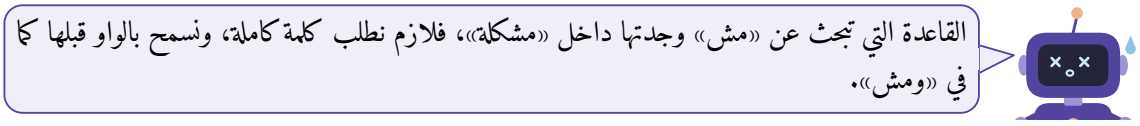
**Task.** Notice negation. The Egyptian مش (*mish*, "not") turns "I know the password" into "I don't know the password".

**Try.** Search for the two letters مش. They match انا مش عارف الباسورد ("I don't know the password"), as wanted.

**Problem.** They also match عندي مشكلة في التسجيل ("I have a problem with registration"), which has no negation. Written without vowels, مشكلة (*mushkila*, "problem") begins with the same two letters.

**Fix.** Ask for a whole word: `\bمش\b`. In Python 3, `\w` matches Arabic letters, so `\b` works as in English, and there is no boundary in the middle of مشكلة. Run N1 first: a vowel mark written inside مِش would split the two letters, and the pattern would miss it.

**Still fails.** ومش لاقي الجدول ("and I can't find the timetable"): the conjunction و is joined to مش, so there is no boundary before م. Allow the two conjunctions in front, `\bمش?[وف]\b`, and all three forms match while مشكلة is still skipped. This is the clitic problem of Section 2.1.3, solved by listing only the clitics this task needs (Figure 2.12).




### W2. Search that forgives spelling.

**Task.** Find الاختبار (*al-ikhtibar*, "the test") however the user writes the hamza, *without* changing the stored text, as N3 would.

**Try.** Search for the word as it is. It misses موعد الإختبار امتى ("when is the test?"), written with hamza below, one of the most common spellings.

**Fix.** Change the pattern instead of the text: replace each letter that users spell in more than one way by a class of its spellings, `[اأإآ]` for alef, `[ةه]` for taa marbuta and haa, and `[ىي]` for ya and alef maqsura. A short function, `tolerant`, does this for any word; for

| | Task | Pattern | Message | Result |
|---|---|---|---|---|
| W1 | negation مش as a word | `\b[وف]?مش\b` | انا **مش** عارف الباسورد | ✓ negation |
| | | | عندي مشكلة في التسجيل | ✓ no match |
| W2 | find a word in any hamza spelling | `ر[اأإآ]ختب[اأإآ]ل[اأإآ]` | موعد **الإختبار** امتى | ✓ text unchanged |
| | | | موعد الاختبر امتى | × a letter dropped |
| W3 | one word, many forms | `\b[وف]?(?:ال)?امتحان(?:ات)?\b` | **والامتحانات** امتى؟ | ✓ whole word |
| | | | امتحاني اتأجل | × suffix ي not listed |
| W4 | Egyptian mobile number | N5, drop separators, `\b01[0125]\d{8}\b` | رقمي **٠١٠١٢٣٤٥٦٧٨** | ✓ `01012345678` |
| | | | كلمني على **6789 345 0122** | ✓ `01223456789` |
| W5 | time with ص or م | `(\d{1,2}):(\d{2})(?:\s*([صم])\b)?` | المحاضرة الساعة **١٠:٣٠ ص** | ✓ `10:30` |
| | | | الساعة **٩:٠٠** مدرج ٥ | ✓ م of مدرج left out |
| W6 | is it a question? | `[؟?]\s*$` or `^(?:امتى\|امتى\|فين\|…)\b` | **امتى** الامتحان | ✓ question word |
| | | | عايز اعرف ميعاد الامتحان | × indirect question |

Boxed: the text the pattern returns. Phone and time patterns run after N5, which turns Arabic-Indic digits into 0–9. A × row is a known failure the chapter keeps on record.

**Figure 2.11** Summary of the six Arabic patterns W1–W6. Each pattern is tried on one message it should handle and one that tests its limit; a box marks the text the pattern returns. Rows marked with a cross are known failures, discussed in the text.

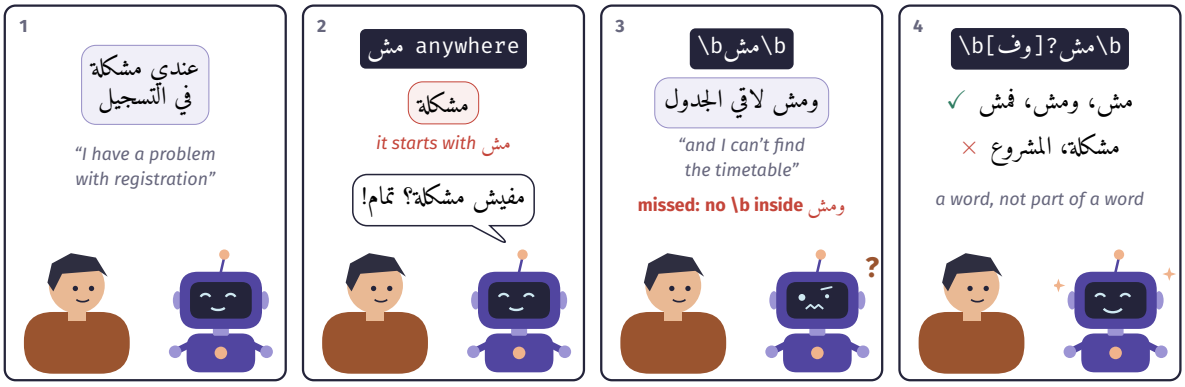



**Figure 2.12** Sanad's negation rule, two fixes later. The bare مش (*mish*, "not") fires inside مشكلة (*mushkila*, "problem") and answers "No problem? Great!"; \b asks for a whole word but then misses ومش, where the conjunction is joined on; allowing و or ف in front reads all three and still skips مشكلة.

الاختبار it gives [اأإآ]ل[اأإآ]ختب[اأإآ]ر. It matches الاختبار, الإختبار, and ألاختبار, and the match is the user's own spelling, so Sanad can highlight it in the original message.

**Still fails.** A class swaps one letter for another; it cannot put back a missing letter. الاختبر, with one letter dropped, needs edit distance (Chapter 3).

### W3. One word, many forms.

**Task.** Find "exam" with or without the article ال, in the singular امتحان (*imtihan*) or the plural امتحانات (*imtihanat*).

**Try.** Make the article optional with a question mark: امتحان?ال.

**Problem.** A counter applies only to the one item just before it (Section 2.1), here the ل, not the whole article. So the pattern rejects the bare امتحان and accepts the typo اامتحان.

**Fix.** Group each optional part: امتحان?(?:ال)(?:ات)? fully matches all four forms. To search inside a message, add W1's \b and [وف]?; the pattern then returns the whole word والامتحانات ("and the exams").

**Still fails.** امتحاني (*imtihani*, "my exam") and the dual امتحانين ("two exams"). Every new ending is one more group, and the list grows fast; Chapter 4 turns to tokenizers that handle word structure.

### CHALLENGE W4. Egyptian mobile numbers.

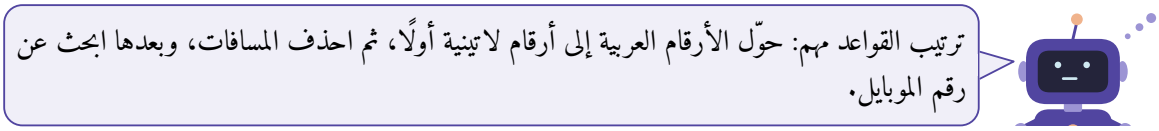
**Task.** Pull a mobile number out of a message: 11 digits, `01`, then a network digit (`0`, `1`, `2`, or `5`), then eight more.

**Try.** `\b01[0125]\d{8}\b`. It finds `01012345678`.

**Problem.** It finds nothing in رقمي ٠١٠١٢٣٤٥٦٧٨ ("my number is …"). `\d` accepts Arabic-Indic digits, but the literal `0` and `1` and the class `[0125]` are ASCII only. It also misses `0122 345 6789`, written with spaces.

**Fix.** Prepare the text first, in this order: N5 turns the digits into 0–9; the lookaround `(?<=\d)[\s-](?=\d)` deletes a space or hyphen between two digits; then the pattern runs. The order of the steps is part of the rule.

**Still fails.** The international form `+20 10…`, which drops the leading `0`. The separator rule also joins a number to its neighbour: in `01012345678 3 الغرفة` (“room 3 …”) the room number is glued to the front, and the mobile number is lost.




### CHALLENGE W5. Times with ص and م.

**Task.** Read times such as `١٠:٣٠ ص`, where ص stands for صباحًا (“in the morning”) and م for مساءً (“in the evening”).

**Try.** Three groups capture the hour, the minute, and the optional letter:

```
(\d{1,2}):(\d{2})(?:\s*([صم]))?
```

On `١٠:٣٠ ص` المحاضرة الساعة (“the lecture is at 10:30 a.m.”) it returns ١٠, ٣٠, and ص, which a short function turns into `10:30`.

**Problem.** In `٩:٠٠ مدرج ٥` الساعة (“at 9:00, lecture hall 5”) it takes the first letter of مدرج (*madrag*, “lecture hall”) and reads the time as p.m.

**Fix.** Put `\b` after the letter, inside the optional group: “this letter must be a whole word”. The م of مدرج is followed by another letter, so it is left out.

**Still fails.** The full word مساءً is not read as p.m.; Exercise E4 fixes it.

### CHALLENGE W6. Is it a question?

**Task.** Sanad’s router answers questions and files reports, so it needs a rough test: is this message a question?

**Try.** A question mark at the end, ASCII or Arabic (؟, U+061F): `[?؟]\s*$`.

**Problem.** Chat messages often drop the mark, so امتى الامتحان (“when is the exam”) is missed.

**Fix.** Add a second signal, a question word at the start, after N3 so that إمتى matches امتى and أين matches اين:

```
^(?:متى | امتى | فين | اين | ازاي | كيف | هل)\b
```

Write the words as the normaliser leaves them. This pattern expects N3 alone; if N4 also runs, ى becomes ي, so متى must be written متي, or the pattern finds nothing. Now امتى الامتحان and هل في محاضرة بكرة (“is there a lecture tomorrow”) are questions, and المحاضرة اتلغت. (“the lecture was cancelled.”) is not.

**Still fails.** القاعة فين (“the hall, where?”) puts the word at the end (Exercise E5), and عايز اعرف ميعاد الامتحان (“I want to know the exam date”) has neither signal. A pattern sees the form of a question, not its intent.

**Pause, predict, then explore**

Write what each pattern returns (the match, the groups, or "nothing"), or whether W6 calls the message a question. W4–W6 run after the steps their paragraphs name; items f–h use these challenge-route patterns.

| | Pattern | Message | Your prediction |
|---|---|---|---|
| a. | W1, with clitics | فش هاجي بكرة | |
| b. | W1, with clitics | المشكلة اتحلت | |
| c. | W3, with `\b` | امتحانين في يوم واحد | |
| d. | W3, with `\b` | فالامتحانات | |
| e. | W2, `tolerant(مدرسة)` | مدرسه | |
| f. | W5, then to 24 hours | الساعة ١١:٠٠ م | |
| g. | W4 | ٠١٥٥١٢٣٤٥٦٧ | |
| h. | W6 | القاعة فين and فين القاعة | |

Which two items are known failures of the rule, not mistakes in your prediction?

**Figure 2.13** Predict what each pattern of W1–W6 returns, then check your answers in Python.



**TIPS**

A recipe for any Arabic pattern: normalise what you can list (N1–N5), write the pattern in the spelling the normaliser produces, allow the clitics the task needs, put `\b` around whole words, test one message that must match and one that must not, and keep the failures you cannot fix in a written list.

Every ELIZA rule in the next section is a substitution of the kind in Section 2.1.5. The decomposition pattern captures part of the input, and the reassembly template writes the captured text back.

## 2.3 ELIZA: dialogue from patterns

Joseph Weizenbaum's ELIZA (1966) held typed conversations by pattern matching. Its best-known script, DOCTOR, played a therapist who turns the patient's words back into questions. ELIZA knows nothing about the world or about the user. Each reply comes from four mechanisms:




1. **Keywords with ranks.** The script lists keywords such as *I am*, *I feel*, or *mother*. When several occur, the highest-ranked keyword chooses the rule.
2. **Decomposition patterns.** Each rule has a pattern that splits the input around the keyword, for example `.* I am (.*)`.

3. **Reflection.** Captured text has its pronouns swapped: *I*→*you*, *my*→*your*, *me*→*you*.
4. **Reassembly templates.** The reflected text fills a template such as *How long have you been {0}?*. Successive turns cycle through a rule's templates. If no keyword matches, a fallback such as *Please go on* keeps the conversation going.

Figure 2.14 follows one turn of the English ELIZA through the four steps. ELIZA does not know what worry or exams are. It copies the user's own words into a fixed sentence.

> **CHECK THE CODE** `examples/ch02/eliza_english.py` – runs the English ELIZA turn by turn.

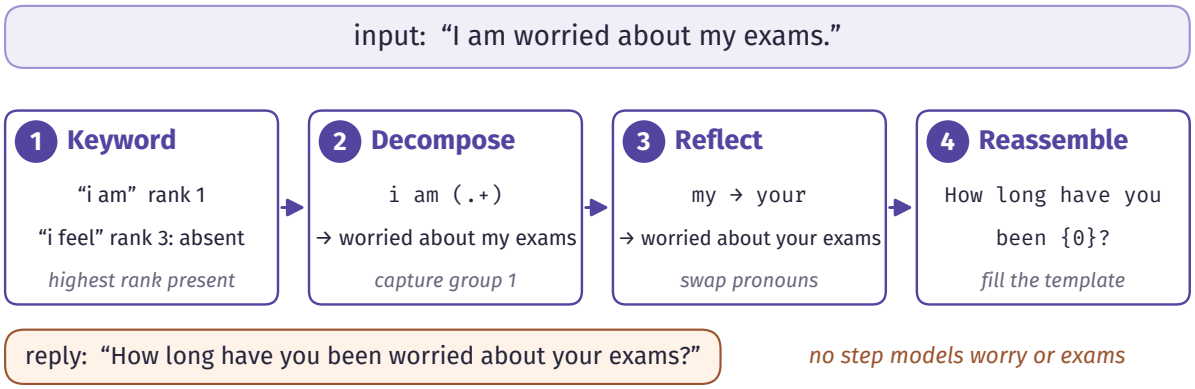



**Figure 2.14** ELIZA's four mechanisms traced on one input. Each stage is a string operation, and the reply is assembled from the user's own words.

### 2.3.1 An Arabic ELIZA and where it breaks

Lab 2 builds the same machine for Arabic help-desk messages. Its six rules cover *I am*, *I feel*, *I want*, *I cannot*, family words, and exam words. The exam rule has the highest rank. Figure 2.15 shows one session. Turn 3 contains both *I want* (rank 2) and *the exam* (rank 5), so the exam rule wins. Users often type *I feel* and *I want* with a bare alef instead of hamza, so the patterns use a class such as `[\u0623\u0627]` to accept both.

Reflection is where Arabic morphology matters. English *my* is a separate word; Arabic writes it as the suffix ي (*-i*). Lab 2 therefore uses two rules:

- a dictionary for separate pronouns: أنا↔أنت, لي↔لك;
- a suffix rule that rewrites a final ي as ك (*-ak*) on words of four or more letters.

So مستقبلي (*mustaqbali*, "my future") becomes مستقبلك (*mustaqbalak*, "your future"), as in turn 0. The suffix rule has three known failures:

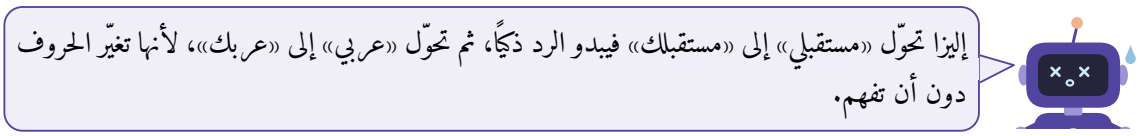
- **Some adjectives also end in ي** (*nisba* adjectives). عربي (*3arabi*, "Arab") becomes عربك ("your Arab"). A list of exceptions can protect words such as الذي (*alladhi*, "who, which"), but no list can hold every adjective.
- **"You" has gender.** أنتَ and كَ are masculine; a woman is addressed as أنتِ and كِ. A student who writes "I am a new student" with the feminine طالبة (*taliba*) still gets the

| Turn | User | Rule (rank) | ELIZA reply |
|---|---|---|---|
| 0 | أنا قلق على مستقبلي | *I am* (1) | منذ متى وأنت قلق على مستقبلك؟ |
| 1 | أشعر بالتعب | *I feel* (3) | منذ متى تشعر بالتعب؟ |
| 2 | لا أستطيع دخول المنصة | *I cannot* (4) | ما الذي يمنعك من دخول المنصة؟ |
| 3 | أريد تأجيل الاختبار | *exam* (5) | ما الذي يقلقك في الاختبار؟ |
| 4 | أمي تسأل عن نتيجتي | *family* (3) | حدثني أكثر عن عائلتك. |
| 5 | شكرا | fallback | كيف يجعلك ذلك تشعر؟ |
| – | أنا طالب عربي | *I am* (1) | منذ متى وأنت طالب عربك؟ |
| – | أنا طالبة جديدة | *I am* (1) | منذ متى وأنت طالبة جديدة؟ |

**Figure 2.15** One Arabic ELIZA session from Lab 2. Each turn shows the rule that fired and its rank. The two shaded rows are known failures of reflection: an adjective is rewritten as "your Arab", and a female student is addressed with the unvocalised masculine-default أنت.

masculine reply. Without vowel marks the two forms look the same on screen, but they sound different when read aloud. To choose correctly, the system must know who is speaking, and ELIZA does not.

- **Dialect.** The Egyptian أنا مش عارف (*ana mish 3arif*, "I don't know") fires the *I am* rule. The reply then mixes a Modern Standard Arabic template with an Egyptian capture.




### 2.3.2 The ELIZA effect and what it teaches about evaluation

Weizenbaum was disturbed to find that users, even people who knew how ELIZA worked, believed it understood them. This is now called the **ELIZA effect**; Figure 2.16 shows it in a help-desk chat.

The lesson holds for every chapter: a reply that sounds good does not prove the problem was solved. "What is stopping you from accessing the platform?" sounds caring but solves nothing; judge whether the ticket was routed, answered correctly, or passed to a person. Rules still serve modern systems for routing, input checks, and safe fallbacks, and they are measured like any other part.

**KEY POINT**

ELIZA works in four steps: rank keywords, split the input, swap pronouns, and fill a template. Arabic attached pronouns, gender, and dialect each break one of these steps in a predictable way. A reply that sounds good is not proof of understanding, or that the task was done.

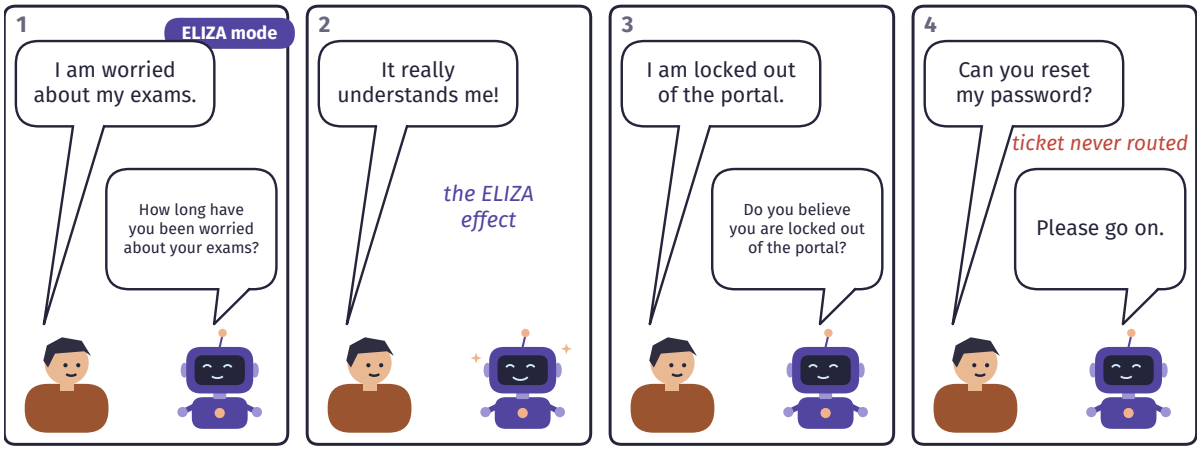



**Figure 2.16** Sanad in ELIZA mode reflects Nour's words back so convincingly that Nour feels understood, yet the ticket is never routed: fluency is not task success.

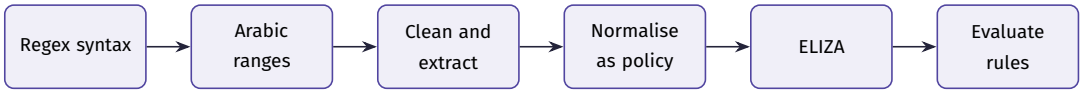
## Chapter summary

Regular expressions are exact text rules whose results depend on the function, the character classes, and the boundaries. Arabic cleaning and clitic rules can merge words that should stay apart, so keep the original text and record known failures. ELIZA ranks keywords, splits the input, swaps pronouns, and fills a template; a good-sounding reply does not mean the problem was solved. Score a rule like any classifier, with precision and recall, and test every rule on messages it was not written from.

**CODE IT IN THE LAB MANUAL**

**Lab 1** (an Arabic normaliser), **Lab 2** (an Arabic ELIZA), and **project level 2**: add a shared normaliser, replace Sanad's baseline router with your own regular-expression rules, and add slot extraction and an ELIZA-style clarifier. Then measure what the rules learned by heart.

**Chapter 2 at a glance**




**Keep these ideas**

- A regular expression is a written policy: classes, quantifiers, anchors, and groups say exactly what matches.
- Arabic text needs Unicode ranges and explicit decisions about diacritics, elongation, and clitics.
- Lookaround and backreferences extract values without consuming the context around them.
- ELIZA replies with keywords, decomposition, reflection, and reassembly: string operations, not understanding.
- Score a rule with precision and recall, and keep a list of its known failures.

**In one small example**

Each refinement of the price pattern fixes one named error; false positives lower precision and missed amounts lower recall (Figure 2.8).

**Common mistake.** Fixing a pattern until it matches the examples in front of you.
**Instead.** Keep a test set of true and false cases, and rerun it after every change.

## 2.4 Review and practice

These questions follow the chapter objectives; items added for extra practice come at the end of each list, and the progress table maps every item to its objective. Try each one on paper before you look at the answers. Items marked CHALLENGE belong to the challenge route; skip them on the core route.

### Quick quiz

Choose one answer for each question; the key is in the instructor edition.

1. In Python 3, why does `\w+` split a fully vocalised Arabic word into fragments?
   - **A** `\w` does not match combining marks, so each mark ends a match.
   - **B** `\w` does not match Arabic letters.
   - **C** The `re` module is ASCII-only.
   - **D** Arabic is written from right to left.

**2.** Which pattern matches exactly three digits?

- **A** `[0-9]*`
- **B** `[0-9]{3}`
- **C** `[^0-9]{3}`
- **D** `\d?`

**3.** A pattern is written as the article ال followed by كتب | كتاب, with no parentheses. Which of these strings does it fully match?

- **A** الكتب.
- **B** Bare كتب.
- **C** Both الكتاب and الكتب.
- **D** Nothing, because alternation needs parentheses.

**4.** What does `(\w)\1{2,}` match?

- **A** Any word of at least two letters.
- **B** One word character followed by at least two more copies of that character.
- **C** Two different letters in a row.
- **D** A word that occurs twice.

**5.** What is special about a lookahead such as `(?=[a-z]{3})`?

- **A** It matches three letters and removes them.
- **B** It checks the text before the current position.
- **C** It checks the text that follows without consuming it.
- **D** It makes the previous quantifier lazy.

**6.** For which task is alef normalisation (N3) most likely to help?

- **A** Diacritisation.
- **B** Pronunciation for speech synthesis.
- **C** A search index where users often omit hamza.
- **D** Choosing gender agreement.

**7.** Why does policy N8 refuse to split فكتب (*fa-kataba*)?

- **A** ف is not in the pattern.
- **B** The word is too long.
- **C** Arabic clitics are always suffixes.
- **D** The greedy optional group takes the root letter ك, leaving a two-letter stem.

**8.** Which ELIZA mechanism turns “worried about my exams” into “worried about your exams”?

- **A** Keyword ranking.
- **B** Reflection.
- **C** Decomposition.
- **D** The fallback reply.

**9.** A turn contains *I want* (rank 2) and *the exam* (rank 5). Which rule fires?

- **A** *I want*, because it appears first.
- **B** Both rules, one after the other.
- **C** The exam rule, because it has the higher rank.
- **D** The fallback, because two keywords conflict.

**10.** A course-code pattern returns 18 spans, 16 of them correct, from messages containing 20 true codes. What are precision and recall?

- **A** .889 and .80.
- **B** .80 and .889.
- **C** .80 and .80.
- **D** .889 and .889.

**11.** The pattern مش (*mish*, "not") with no boundaries matches عندي مشكلة ("I have a problem"). Why?

- **A** `\b` does not work on Arabic letters.
- **B** The message is written without vowel marks.
- **C** The pattern needs `re.fullmatch`.
- **D** The two letters also begin مشكلة, and nothing asks for a whole word.

**12.** CHALLENGE The pattern `\b01[0125]\d{8}\b` finds no number in رقمي ٠١٠١٢٣٤٥٦٧٨. What is the best fix?

- **A** Convert Arabic-Indic digits to ASCII (N5) before matching.
- **B** Remove both `\b`.
- **C** Replace `\d` with `[0-9]`.
- **D** Use `re.search` instead of `re.findall`.

## Check your understanding

**Q1.** In Python, `\w+` breaks a fully vocalised Arabic word into pieces. Why? Give two ways to fix it.

**Q2.** Name one task where alef normalisation (N3) helps, and one task where it does harm.

**Q3.** Policy N8 will not split فكتب (*fa-kataba*). Explain why, using the word "greedy".

**Q4.** Look at the two shaded rows in Figure 2.15. Which ELIZA mechanism causes each failure?

**Q5.** A set of routing rules is right on almost every message it was written from, but on far fewer new messages. Why? Which messages should you write rules from, and which should you score them on?

**Q6.** Policy N3 and the `tolerant` search pattern of W2 both treat الإختبار and الاختبار as the same word. What does each one change, and when would you choose the pattern instead of normalising?

## Work it through

**E1.** Write a pattern that finds dates written as day/month, such as 15/10. Each part has one or two digits, in ASCII or Arabic-Indic. Give two inputs it must accept, two it must reject, and one wrong input it still accepts.

**E2.** A pattern returns 30 spans; 24 are correct. The messages contain 32 true spans. Compute precision and recall. Then say what could happen to each one if you change the pattern to accept lower case too.

**E3.** V3 of the price pattern in Example 2.3 returns only `1` from `EGP 1,250`. Write V4 so it returns `1,250`. Name one new false positive V4 might let in, and one more input it should accept (the Arabic thousands separator is U+066C).

**E4.** CHALLENGE Extend the time pattern of W5 so the full words صباحًا and مساءً, with or without the tanween mark, are read as a.m. and p.m. Test it on ٧:٠٠ مساءً, on ١٠:٣٠ ص, and on ٩:٠٠ مدرج ٥. Which earlier policy would let you write the words without the mark?

**E5.** CHALLENGE The question detector of W6 misses فين القاعة ("the hall, where?"). Add a third signal: a question word at the *end* of the message. Give one statement that your new rule wrongly calls a question.

**E6.** Use a lookbehind to return only the amount from the message رقم 4589 والمبلغ 300 ("number 4589 and the amount 300"). Then write one `re.sub` call that turns `CS 120` and `CS120` into `<CS120>`, using two capturing groups. Why can the lookbehind not allow any number of spaces?

## Read the picture

**V1.** In Figure 2.8, a new version removes a false positive and changes nothing else. Which of P and R must rise, and which cannot change?

**V2.** In Figure 2.14, what would the reply look like without the reflection step, and why?

## Check your progress

Use the objective identifiers from the opening list. Quiz items check recall; Q items ask for explanation; E items, labs, and assignments ask you to calculate, implement, or report.

**Table 2.4** From Chapter 2 objectives to evidence of learning

| Goal | Preparation | Your evidence |
|---|---|---|
| O1 | Section 2.1, the code-point primer, Figure 2.3; Section 2.2, W1–W3 (CHALLENGE W4–W6). | Quiz 1–3, 11 (CHALLENGE 12); Q1; E1, E3: write and test a pattern (CHALLENGE E4, E5); the W1–W6 activity; Worksheet 3; Lab 1, N1–N7. |
| O2 | Section 2.1.5. | Quiz 4–5; E6: lookbehind and substitution; Worksheet 4, tasks 1–2. |
| O3 | Figure 2.4, Section 2.1.3, Figure 2.5. | Quiz 6–7; Q2–Q3, Q6; Worksheet 3, task 3; Lab 1, N8 and its expected failures; project level 2, the normaliser. |
| O4 | Section 2.3, Figures 2.14 and 2.15. | Quiz 8–9; Q4; Worksheet 4, task 3; Lab 2 and its transcript discussion; project level 2, the clarifier. |
| O5 | Examples 2.2 and 2.3. | Quiz 10; Q5; E2: precision and recall; Lab 1 and Lab 2 failure reports; project level 2, the rule router. |

## Further reading

Chapter 2 of Jurafsky and Martin's *Speech and Language Processing* covers regular expressions. Weizenbaum's 1966 ELIZA paper describes the keyword and reassembly mechanism in his own words. The Python `re` documentation defines the Unicode behaviour of `\w` and `\d`. For Arabic segmentation that does not rely on string rules, see the CAMeL Tools paper cited in Chapter 1.

CHAPTER 3

# Counting and Comparing Words

IN THIS CHAPTER


3.1 Words and corpora: counting what is there 58

3.2 Minimum edit distance 64

3.3 Review and practice 77


**LEARNING OBJECTIVES**

After working through this chapter, you should be able to:

- **O1.** describe a corpus in a data statement, and count its tokens and types under a stated tokenization policy;
- **O2.** distinguish word forms, lemmas, and roots, and explain why Arabic text has many types;
- **O3.** fill a unit-cost edit-distance table by hand, and decide when to use edit distance instead of normalisation;
- **O4.** CHALLENGE choose weighted edit costs, recover an alignment by backtrace, and compute word error rate.




Chapter 2 used rules to recognise predictable patterns in text: a course code, a price, or a pronoun to replace. We now move from recognising patterns in individual messages to asking two broader questions: what words occur in a collection of messages, and how can we match a word when its spelling is imperfect? Corpus counts describe the vocabulary we observe and lay the groundwork for the probability models of Chapter 5. Edit distance measures how much one string must change to become another, giving us a systematic way to compare spellings.

**The goal of this chapter** is to describe Sanad's tickets with counts you can explain and reproduce, then use edit distance to make its search tolerant of typos and evaluate whether that tolerance improves retrieval. Arabic help-desk text remains our focus: attached clitics and spelling variants make decisions about what counts as a word especially important.

Return to the message “When is the exma for CS120?” The pattern from Chapter 2 found the course code, but recognising *CS120* does not help an exact-match search connect *exma* with *exam*. We first *count* the words in a corpus of messages, making our tokenization choices explicit (Section 3.1). We then *compare* spellings with edit distance to find plausible matches without writing a separate rule for each typo (Section 3.2). Figure 3.1 shows this route. Lab 3 brings these ideas together after Chapters 4 and 5, while project level 3 puts them to work in Sanad’s search.

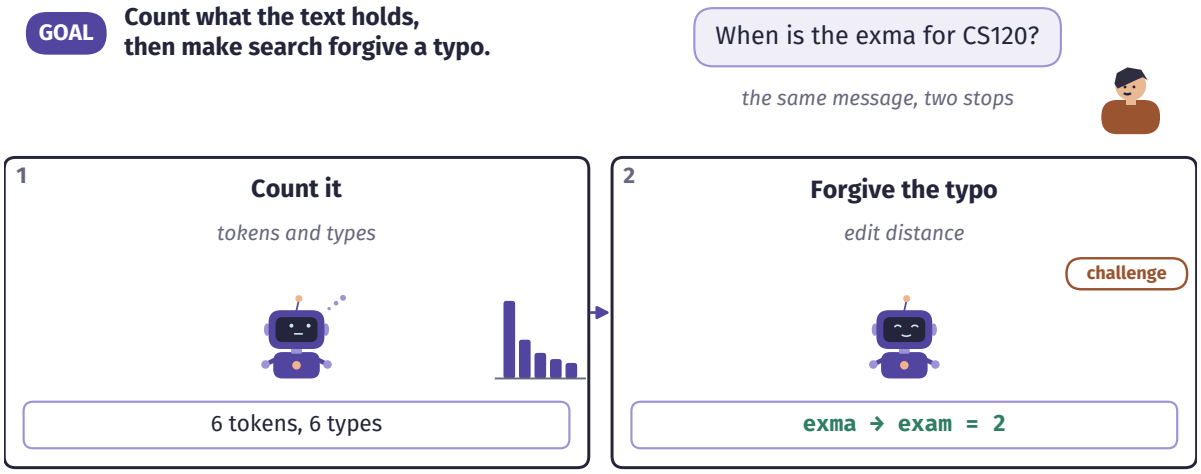


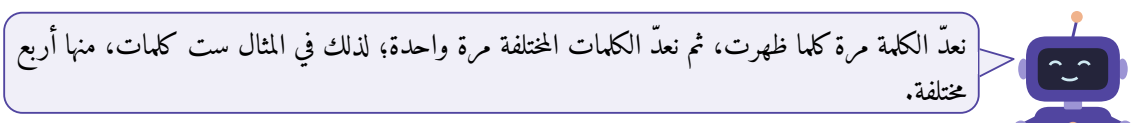

**Figure 3.1** Chapter 3’s goal and route. The same message passes two more stops: counting shows what the text contains (Section 3.1), and edit distance accepts the typo (Section 3.2).

## 3.1 Words and corpora: counting what is there

A **corpus** is a collection of text chosen for a purpose. It is described by its source, its language varieties, and how it was collected. Counting starts with three units:

- a **token** is one occurrence of a word (or other unit) in the text;
- a **type** is a distinct unit, so the vocabulary size $V$ is the number of types;
- a **lemma** is the dictionary form that several word forms share, such as *study* for *studies* and *studied*.

The number of tokens is written $N$. For example, “the exam and the exam date” has $N = 6$ tokens but only $V = 4$ types: *the*, *exam*, *and*, *date*. Figure 3.2 draws the two counts. All counts depend on the tokenization policy, so a reported vocabulary size must name its policy.




Lab 3’s corpus has 40 help-desk requests, written for teaching. Most are Modern Standard Arabic; a few are Egyptian (مش, عايز, امتى). With N1, N2, and an Arabic-letters tokenizer:

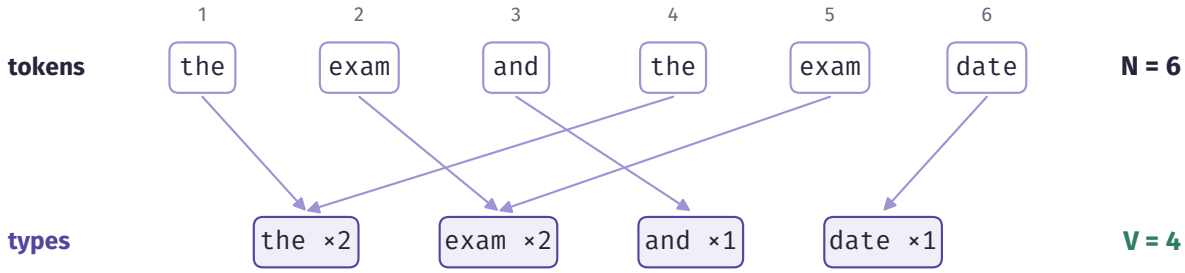



**Figure 3.2** Six tokens, four types. Every occurrence is a token; each distinct word is a type, counted once with its frequency. Both numbers depend on the tokenization policy that produced the tokens.

- $N = 185$ tokens and $V = 105$ types;
- type–token ratio $105/185 \approx .57$;
- 61 types, more than half the vocabulary, occur exactly once.

Words that occur only once are called **hapax legomena**. When most words are this rare, the counts are *sparse*: we see too little of each word to learn much about it. Chapter 5 must deal with this when it counts pairs of words.

### 3.1.1 Describe the corpus before using its counts

A corpus comes from certain writers, times, genres, and dialects; write these down before claiming that it represents a language. Say whether language labels apply to whole messages or to single words, since one message can switch languages (*code-switching*), and never guess private facts about writers from their names or dialect. A short **corpus data statement** records:

- why the texts were collected, where from, when, and how they were chosen;
- languages and dialects, genre, and what is really known about the writers;
- who may use it, whether the writers agreed, and how personal data is protected;
- how labels were added, how sure they are, and what was left out;
- the cleaning and tokenization rules, how duplicates were handled, and how the data was split;
- what it is meant for, what it does not cover, and what its counts cannot tell you.

For Lab 3, the statement begins “40 invented teaching requests, mainly Modern Standard Arabic with some Egyptian forms”. These are not real student messages, so their counts say nothing about which dialects students use. A new corpus needs its own statement and its own training and test split (Chapter 8); the further reading gives a fuller checklist.

**A counting command is a tokenization policy.** A common ASCII-only pipeline for counting words in a file is

```
LC_ALL=C tr -cs 'A-Za-z' '\n' < corpus.txt |
```

```
    sed '/^$/d' | sort | uniq -c | sort -nr
```

Read it stage by stage:

- `tr -cs 'A-Za-z' '\n'` turns every run of non-ASCII-letters into one line break, so each line holds one word;
- `sed '/^$/d'` deletes empty lines;
- `sort | uniq -c` groups equal lines and counts them;
- `sort -nr` puts the most frequent word first.

The first step *is* the tokenizer, and it changes the text in ways you may not want. It keeps case, so `The` and `the` are two types. It splits `can't` into `can` and `t`. It drops every Arabic letter, and it drops the digit in Arabizi `3ayez`. These come from the rule, not from the language. Figure 3.3 shows the losses on one of Nour's messages.

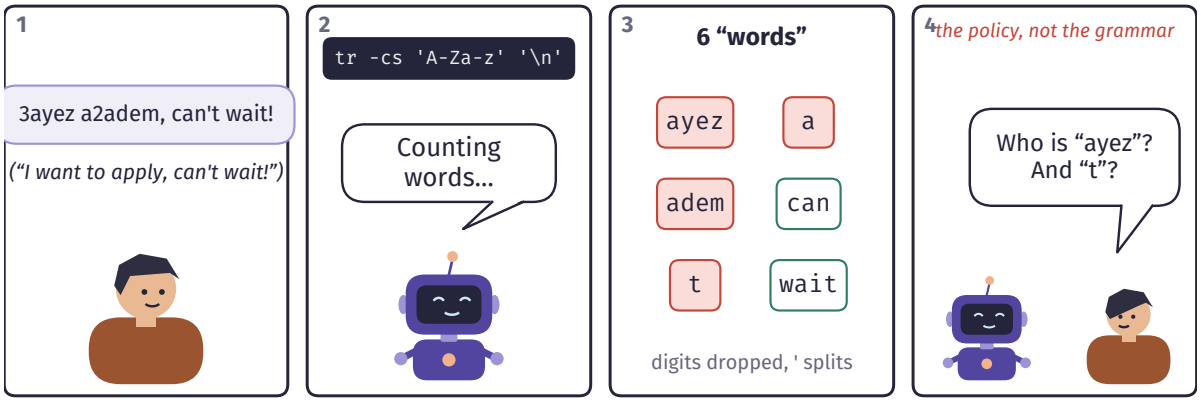



**Figure 3.3** Sanad counts Nour's message with an ASCII-letters command. The digits in the Arabizi *3ayez* and *a2adem* and the apostrophe in *can't* become boundaries, so four of the six "words" are fragments.

**Example 3.1**

**Rewrite the command as a stated policy.** Listing 3.1 counts in Python instead. It lower-cases the text, then keeps runs of Unicode word characters, so Arabic letters and digits stay inside words.

**Listing 3.1** A Unicode-aware word count (`examples/ch03/count_words.py`)

```
def count_words(text, keep_apostrophe=False):
    """Lower-case, then keep runs of Unicode word characters (N1 first
        for Arabic marks)."""
    pattern = r"\w+(?:'\w+)?" if keep_apostrophe else r"\w+"
    return Counter(re.findall(pattern, text.lower()))
```

Run three policies on Nour's message "3ayez el-exam, can't find the Exam! امتحان؟":

- **The ASCII command:** $N = 8$, $V = 8$ (`ayez`, `el`, `exam`, `can`, `t`, `find`, `the`, `Exam`). The Arabic word is gone, the digit splits *3ayez*, and `exam` and `Exam` are two types.

- **`count_words`:** $N = 9$, $V = 8$. *3ayez* and امتحان survive, and `exam` is now counted twice. The apostrophe still splits `can't`.
- **`keep_apostrophe=True`:** $N = 8$, $V = 7$, with `can't` as one token.

None of these is *the* word count; each is a decision you report. On vocalised Arabic, `\w` also stops at every mark (Figure 2.3), so apply N1 first.

**CHECK THE CODE** `examples/ch03/count_words.py` – runs the three policies on Nour's message.

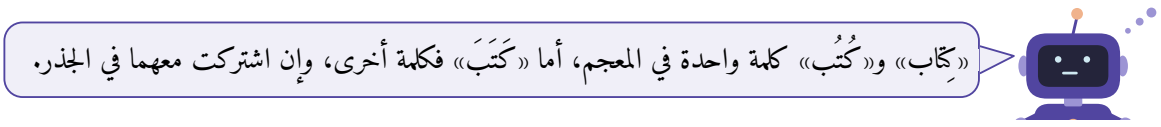
### 3.1.2 Why Arabic text has many types

Arabic packs into one written word what English spreads over several. The stem طلاب (*tullab*, "students") appears as الطلاب, والطلاب, للطلاب, بالطلاب, and طلابنا ("our students"). That is five types for one stem, before any spelling variation.

Two levels group these forms, and they are easy to confuse:

- A **lemma** groups the forms of *one dictionary word*. كَتَبَ (*kataba*, "he wrote") and يَكْتُبُ (*yaktubu*, "he writes") are forms of the verb lemma كَتَبَ; كِتَاب (*kitab*) and its plural كُتُب (*kutub*) are forms of the noun lemma كِتَاب. Without vowel marks, كَتَبَ and كُتُب are both written كتب.
- A **root** is deeper. The root ك-ت-ب (*k-t-b*) is shared by the verb كَتَبَ and by the nouns كِتَاب ("book") and مَكْتَبَة (*maktaba*, "library"). These are three separate dictionary words, each with its own lemma.

Figure 3.4 draws one small word family. A lemmatiser maps forms to lemmas, not to roots; say which tool and lemma policy you used. Word structure is one reason Arabic has many types, but genre, sample size, and tokenization matter too.




Spelling variation adds more types: hamza is often left out (أ written as ا), and taa marbuta (ة) is often written as haa (ه). Figure 3.6(b) shows that N3 and N4 merge three pairs in the Lab 3 corpus. $V$ falls from 105 to 102, and $N$ does not change, because merging spellings removes types but not tokens.

Splitting clitics (N8) is different: it changes both $N$ and $V$. Splitting والطلاب turns one token into three. Some types disappear, because stems are now shared, and some appear, because the clitics become words of their own. Count the result instead of guessing, and check the N8 errors from Section 2.1.3.

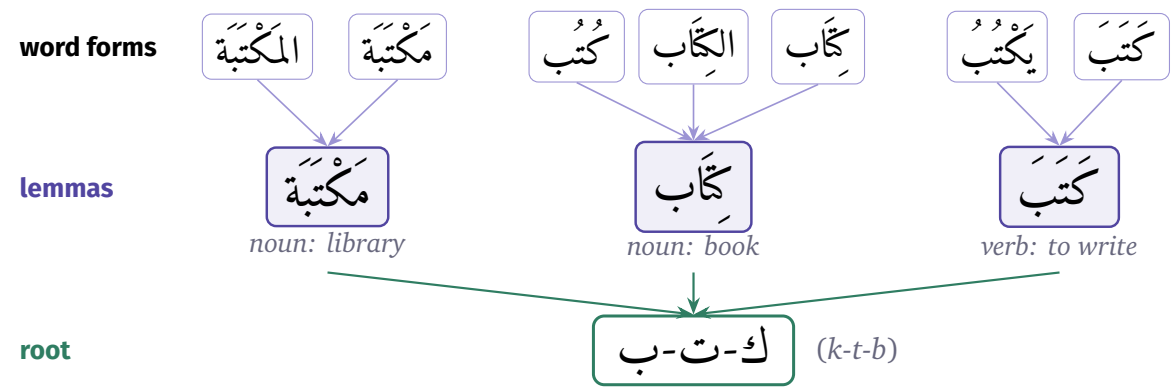



**Figure 3.4** Word forms, lemmas, and a root, in reading order from the right. A lemma groups the forms of one dictionary word; the root ك-ت-ب links three different dictionary words. A lemmatiser maps the top row to the middle row, not to the bottom one.

**Example 3.2**

**One question, three counts.** Sanad's team asks: "How often do students mention the exam?" Take three short messages, each asking when the exam is: امتحان الرياضة امتى؟ ("the maths exam, when?"), الإمتحان إمتى؟ (the same question, with hamza), and والامتحان في القاعه ٢٠٤ ("and the exam is in hall 204", taa marbuta written as haa). Count them under three policies; Figure 3.5 shows every token.

- **P1, split on spaces.** $N = 9$ and $V = 9$: every token is a different type. امتى؟ carries its question mark, and امتى؟ and إمتى؟ differ in one letter.
- **P2, N1–N5, then Arabic words and digit runs.** $N = 9$ and $V = 8$. The punctuation is gone and the two spellings of امتى (*imta*, "when") merge into one type, written امتي after N4, but امتحان, الامتحان, and والامتحان are still three types.
- **P3, P2 and then N8 splits the clitics off as tokens.** $N = 14$ and $V = 8$. Four ال and one و become tokens of their own, and the stem امتحان is now counted three times.

**The answer depends on the policy.** P1 and P2 say "exam" appears once; P3 says three times. P3 answers the team's question, and the type–token ratio falls from 1.0 to .89 to .57 as the policy groups more forms. P3 still counts a plural such as امتحانات as its own type (Exercise E5); grouping it with امتحان needs a lemma. Choose the policy from the question you are asking, and report it with the numbers.

**CHECK THE CODE** `examples/ch03/policy_counts.py` – counts the three messages under P1–P3.

**The corpus** three messages, each asking when the exam is:
(١) امتحان الرياضة امتى؟ (٢) الإمتحان إمتى؟ (٣) والامتحان في القاعه ٢٠٤

| Policy, then its tokens (message (1) on the right) | $N$ | $V$ | exam |
|---|---|---|---|
| **P1** split on spaces | 9 | 9 | 1 |
| امتحان \| الرياضة \| امتى؟ \| الإمتحان \| إمتى؟ \| والامتحان \| في \| القاعه \| ٢٠٤ | | | |
| **P2** N1–N5, then Arabic words and digit runs | 9 | 8 | 1 |
| امتحان \| الرياضه \| امتي \| الامتحان \| امتي \| والامتحان \| في \| القاعه \| 204 | | | |
| **P3** P2, then N8 splits the clitics off as tokens | 14 | 8 | 3 |
| امتحان \| ال \| رياضه \| امتي \| ال \| امتحان \| امتي \| و \| ال \| امتحان \| في \| ال \| قاعه \| 204 | | | |

امتي two spellings now one type ال a clitic counted as a token امتحان the exam stem. "exam": how many tokens are exactly the stem.

**Figure 3.5** The same three messages under three tokenization policies. P2 merges the two spellings of امتى (purple), and P3 splits the clitics off as tokens (copper), so only P3 counts the exam stem (green) three times.

**(a) Most frequent words, raw spelling**

| Rank $r$ | Word | $f$ |
|---|---|---|
| 1 | إلى (to) | 5 |
| 2 | المنصة (the platform) | 5 |
| 3 | في (in) | 5 |
| 4 | لا (not) | 5 |
| 5 | أريد (I want) | 4 |
| 6 | الاختبار (the exam) | 4 |

**(b) Types merged by normalisation N3–N4**

| Form A | $f$ | Form B | $f$ | Merged | $f$ |
|---|---|---|---|---|---|
| أريد | 4 | اريد | 1 | اريد | 5 |
| المنصة | 5 | المنصه | 1 | المنصه | 6 |
| الاختبار | 4 | الإختبار | 1 | الاختبار | 5 |

Tokens $N = 185$ in both. Types $V$: $105 \to 102$.
Hapax legomena: $61 \to 58$.

**Figure 3.6** Counting the Lab 3 corpus. (a) The six most frequent words. (b) Spelling variants that normalisation policies N3–N4 merge.

**KEY POINT**

**A count someone else can reproduce** names the corpus, every policy, and each count after each change, for example: "Lab 3 corpus, 40 constructed requests; N1, N2, Arabic-letter tokens: $N = 185$, $V = 105$, 61 hapax legomena; after N3–N4, $V = 102$, $N$ unchanged."

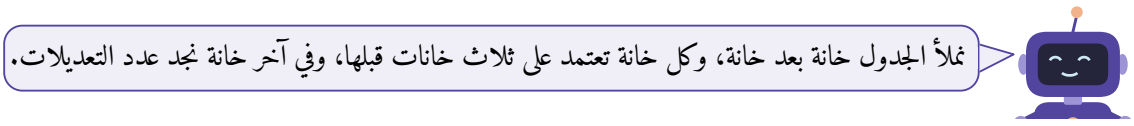
## 3.2 Minimum edit distance

Normalisation decides in advance which spellings count as the same. Edit distance instead measures *how far apart* two strings are. It is used to suggest spelling corrections, to search despite typos (fuzzy search), and, over words, to score speech recognition (word error rate, Section 3.2.5). The **minimum edit distance** between a source and a target is the smallest total cost of insertions, deletions, and substitutions that turns one into the other. When every operation costs one, it is called **Levenshtein distance** (Levenshtein's paper appeared in Russian in 1965 and in English in 1966), and the cost is simply the number of operations.

Let $D[i][j]$ be the distance between the first $i$ characters of the source $s$ and the first $j$ characters of the target $t$. The first column and row are easy: turning $i$ characters into nothing takes $i$ deletions, so $D[i][0] = i$; likewise $D[0][j] = j$. Every other cell takes the cheapest of three moves:

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{delete } s_i, \\ D[i][j-1] + 1 & \text{insert } t_j, \\ D[i-1][j-1] + \begin{cases} 0 & s_i = t_j \\ c_{\text{sub}} & s_i \neq t_j \end{cases} & \text{match or substitute.} \end{cases}$$

Filling the table row by row is **dynamic programming**: each cell reuses three cells that are already solved. The table has $(|s| + 1)(|t| + 1)$ cells, each filled in constant time, so the work grows in proportion to the product of the two lengths, written $O(|s|\,|t|)$: doubling both words makes about four times as many cells. Jurafsky and Martin's textbook sets $c_{\text{sub}} = 2$, so a substitution costs the same as a deletion plus an insertion. This book uses $c_{\text{sub}} = 1$ unless it says otherwise.



**Example 3.3**

**Filling a table by hand: `exma` → `exam`.** A student types `exma` instead of `exam`, swapping the last two letters. Figure 3.7 shows the three stages at a glance: compute one cell, fill the table, and read the answer. The source runs down the rows and the target along the columns. Shaded cells form the path found by backtrace.

| | # | e | x | a | m |
|---|---|---|---|---|---|
| # | 0 | 1 | 2 | 3 | 4 |
| e | 1 | 0 | 1 | 2 | 3 |
| x | 2 | 1 | 0 | 1 | 2 |
| m | 3 | 2 | 1 | 1 | 1 |
| a | 4 | 3 | 2 | 1 | 2 |

**Step 1: the edges.** Row # and column # are the empty string. Turning `exm` into nothing takes 3 deletions, so $D[3][0] = 3$. Making `exa` from nothing takes 3 insertions, so $D[0][3] = 3$.

**Step 2: matches are free.** At $D[1][1]$, `e` meets `e`. The diagonal gives $D[0][0]+0 = 0$, which beats $1 + 1$ from above or the left. In the same way $D[2][2] = 0$: `ex` already equals `ex`.

**Step 3: the first mismatch.** At $D[3][3]$, source `m` meets target `a`. The three moves cost

$$\underbrace{D[2][3] + 1}_{\text{delete } m} = 2, \qquad \underbrace{D[3][2] + 1}_{\text{insert } a} = 2, \qquad \underbrace{D[2][2] + 1}_{\text{substitute } m \to a} = 1.$$

The minimum is 1: `exm` becomes `exa` with one substitution.

**Step 4: the last cell.** At $D[4][4]$, source `a` meets target `m`. Now all three moves tie:

$$D[3][4] + 1 = 2, \qquad D[4][3] + 1 = 2, \qquad D[3][3] + 1 = 2.$$

So the distance is 2, and there are three optimal alignments, one through each neighbour:

| Last move | Operations | Meaning |
|---|---|---|
| diagonal | `MMSS` | substitute `m`→`a`, then `a`→`m` |
| from above | `MMIMD` | insert `a`, match `m`, delete the old `a` |
| from the left | `MMDMI` | delete `m`, match `a`, insert a new `m` |

The diagonal-first backtrace of Section 3.2.4 returns `MMSS`, the shaded path.

**Change the cost.** With $c_{\text{sub}} = 2$, the two substitutions cost 4, but `MMIMD` and `MMDMI` still cost 2. The distance stays 2; only the preferred alignment changes.

**What the table cannot see.** To a person this is *one* mistake: two letters swapped. Levenshtein distance has no swap operation, so it charges 2. A spelling corrector for typed text often adds a fourth move, transposition of two neighbouring letters, at cost 1; with it, `exma`→`exam` costs 1. This variant is called Damerau–Levenshtein distance. Choose the operations to match the errors your users actually make. Figure 3.8 shows the surprise from Nour's side.

**CHECK THE CODE** `examples/ch03/edit_distance.py` – fills every table, distance, and backtrace in this section.

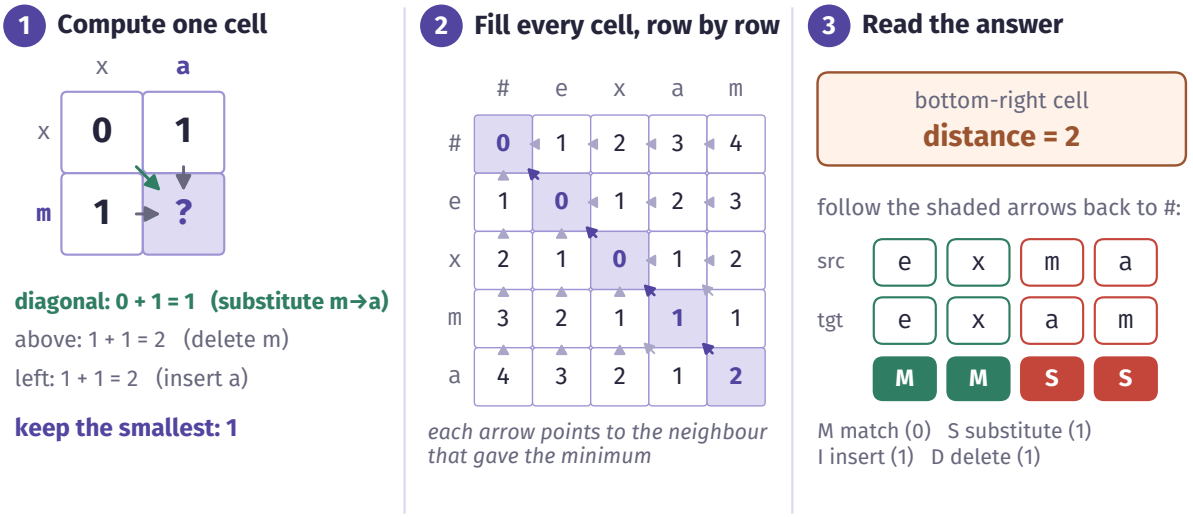



**Figure 3.7** How minimum edit distance is computed and solved, for `exma` → `exam`. (1) A cell takes the cheapest of three moves: the diagonal neighbour plus a substitution (or 0 for a match), the cell above plus a deletion, or the cell to the left plus an insertion. (2) Filling the table row by row gives every cell a value and an arrow back to the neighbour that produced it. (3) The bottom-right cell is the distance; following the arrows back to the top-left cell gives the alignment.

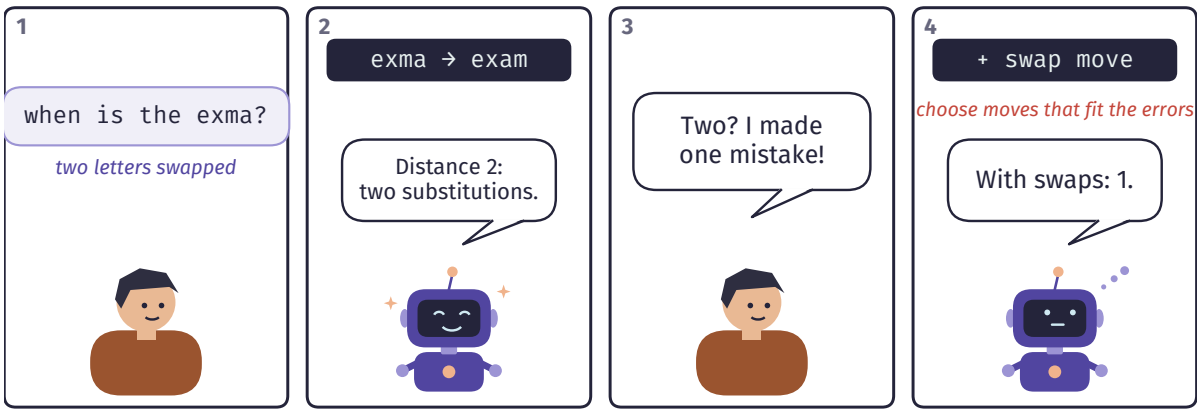



**Figure 3.8** A swapped-letter typo is one mistake to Nour but two edits to Levenshtein distance, which has no swap move. Adding a swap at cost 1 (Damerau–Levenshtein) gives distance 1.

**Example 3.4**

**From كتب to مكتبة.** The root form كتب (*ktb*) has 3 letters. مكتبة (*maktaba*, "library") is written with 5 letters, ending in taa marbuta; its short vowels are not written. Figure 3.9 fills the table for these written forms.

**Read the shaded path.** Insert م, match ك, ت, and ب, then insert ة. Two insertions and three free matches give distance 2. No cheaper path exists: the target is two letters longer, so at least two insertions are needed.

**Spelling matters.** Compare the Latin transliterations instead: *ktb* → *maktaba* gives 4, because *maktaba* spells out two vowels as letters (Figure 3.10). Always say which spelling the table compares.

**What the distance does not know.** Edit distance sees only that the strings share letters. It does not know that both words come from the same root.

| | # | م | ك | ت | ب | ة |
|---|---|---|---|---|---|---|
| # | **0** | **1** | 2 | 3 | 4 | 5 |
| ك | 1 | 1 | **1** | 2 | 3 | 4 |
| ت | 2 | 2 | 2 | **1** | 2 | 3 |
| ب | 3 | 3 | 3 | 2 | **1** | **2** |

Rows: source كتب; columns: target مكتبة, both in reading order.
Shaded path: insert م, match ك, ت, ب, insert ة. Distance 2.

**Figure 3.9** The Levenshtein table for كتب → مكتبة, compared as written, without short vowels. Letters are listed in reading order, from the first letter onward.

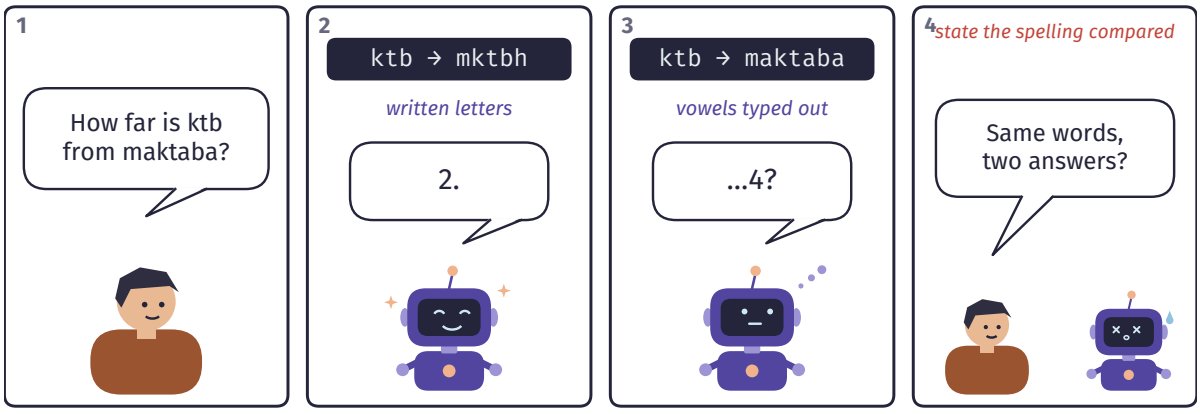


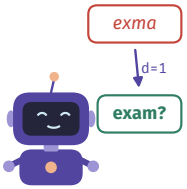

**Figure 3.10** The same word pair has edit distance 2 as written consonants and 4 with the vowels typed out. A distance is a property of the strings you compared, so report the spelling with the number.

Common spelling variants are at distance one: مدرسة and مدرسه (*madrasa*, with taa marbuta or with haa), and إسلام and اسلام (*islam*, with or without hamza). The article adds two letters, ا and ل, so الامتحان and امتحان (*imtihan*, "exam") are at distance two. These small distances suggest two designs:

- **Normalise** when the variation is noise for the task. After N3 and N4 the variant pairs above become identical, so nothing needs to be computed when a user searches.
- **Measure distance** when you must accept errors you cannot list in advance, as in spelling suggestion or fuzzy search. Lab 3's `closest` function maps the misspelling الإختبار, with hamza below, to the lexicon entry الاختبار (*al-ikhtibar*, "the test").

**Distance suggests candidates; it does not decide.** For the input `graffe`, unit-cost distances to `graf`, `graft`, `grail`, and `giraffe` are 2, 2, 3, and 1. `giraffe` is nearest, but a good correction also depends on the lexicon and the sentence. In the same way, similar spelling can suggest that "IBM" and "IBM Inc." are the same company, but it cannot prove it. Use fuzzy matching to suggest candidates, then check the final choice in a separate step.

A spelling suggester puts these ideas to work. Sanad's search box is one.




**Example 3.5**

**Sanad's "did you mean?"** Sanad keeps a lexicon of eight help-desk keywords, stored after N3 and N4: محاضره (lecture), امتحان (exam), تسجيل (registration), منحه (grant), مكتبه (library), نتيجه (result), جدول (timetable), and غياب (absence). For each query word, Sanad normalises it the same way, finds the nearest entry by unit-cost edit distance, and suggests it only if the distance is at most 1.

- محضرة, with a dropped long vowel: after N4 it is one deletion from محاضره. Suggested, correctly.
- امتحن: one insertion from امتحان. Suggested, correctly.
- التسجيل ("the registration"): the nearest entry تسجيل is at distance 2, because the article adds two letters. No suggestion. Removing the article first (N8) gives distance 0.
- مكتب (*maktab*, "office"): one letter from مكتبه ("library"). Sanad suggests "library", and it is wrong: the user wrote a correct word that the lexicon does not contain.

**What the example shows.** Combine the two designs above: normalise what you can list (hamza, taa marbuta, the article), then measure distance for what you cannot (Figure 3.12). Raising the threshold to 2 would accept التسجيل, but it would also let in more distant, unrelated words, and it would not fix مكتب. A suggestion is a guess; show it as a question. Figure 3.11 acts out the four queries.

**CHECK THE CODE** `examples/ch03/fuzzy_search.py` – runs the search and checks every distance.

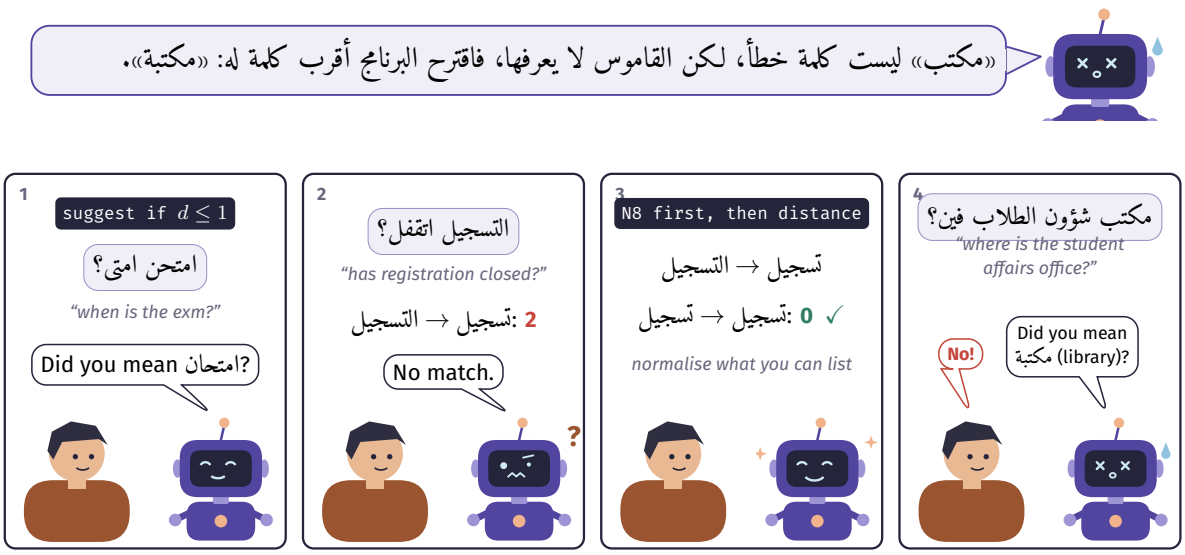



**Figure 3.11** Sanad's "did you mean?" search suggests the nearest keyword when it is at most one edit away. It repairs امتحن, misses التسجيل until N8 removes the article, and confidently offers مكتبة ("library") for مكتب ("office"), a correct word that its lexicon lacks.

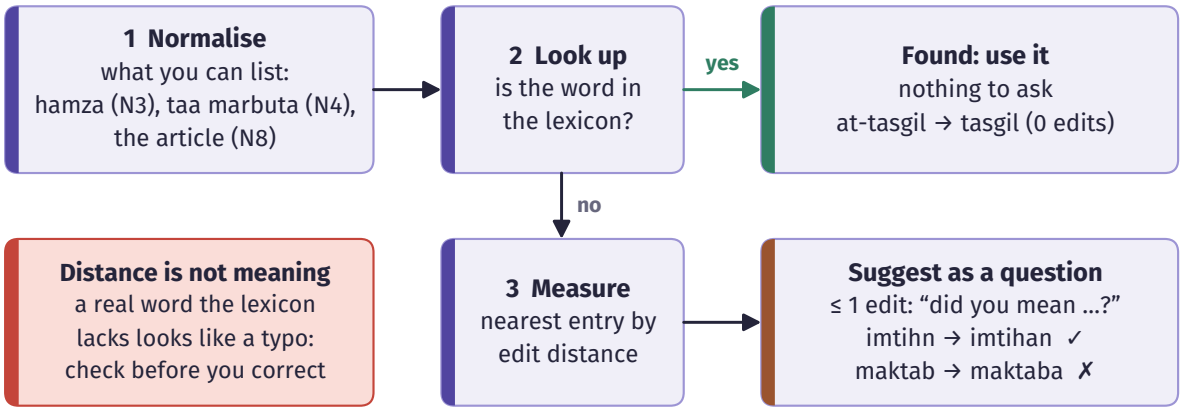



**Figure 3.12** Sanad's search combines the two designs. It first normalises the variation it can list, then looks the word up. Only a word it cannot find is matched by edit distance, and the result is offered as a question.

### 3.2.1 CHALLENGE Choosing the costs

The rest of this section is the challenge route. On the core route, you need the unit-cost table of Example 3.3 and the two designs above.

Equal costs are a simplification. Writing taa marbuta (ة) as haa (ه) is a common habit; writing it as ب is a real error. Yet both cost one. Weighted costs, such as lower costs for confusable letters or neighbouring keys, make the measure fit the task.

**Example 3.6**

**Why the substitution cost matters.** تعليم (*ta3lim*, "education") and معلومات (*ma3lumat*, "information") share the root ع-ل-م. One optimal alignment, with letters in reading order and ∗ marking a gap, is

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| source | ت | ع | ل | ي | م | * | * |
| target | م | ع | ل | و | م | ا | ت |
| operation | S | M | M | S | M | I | I |

Here S is a substitution, M a match, and I an insertion.

**Count the cost.** There are two substitutions (ت→م and ي→و) and two insertions (the plural ending ات).

- With unit costs: $2 \times 1 + 2 \times 1 = 4$.
- With $c_{\text{sub}} = 2$: $2 \times 2 + 2 \times 1 = 6$. Alignments that use a gap pair instead of a substitution also cost 6.

**Read the morphology.** The root letters ع, ل, م all match. The substitutions fall on the prefix and on the long vowel of the word pattern, and the insertions fall on the suffix. The alignment shows where the two words differ, not just how much.

### 3.2.2 CHALLENGE The table as a shortest-path problem

Think of each cell $(i, j)$ as a place: "$i$ source letters and $j$ target letters used so far". Each operation is a step:

- deletion moves to $(i + 1, j)$;
- insertion moves to $(i, j + 1)$;
- a match or substitution moves to $(i + 1, j + 1)$.

An edit sequence is then a path from $(0, 0)$ to $(|s|, |t|)$, and the edit distance is the cost of the cheapest path. Every step moves down, right, or both, so a path can never come back to a cell. That is why the cells can be filled in order.

Why keep only the cheapest cost in each cell? The rest of the path from $(i, j)$ is the same however you got there, so the cheapest way in is always the best start. This **optimal substructure** makes the formula correct. It holds because no cost depends on the previous move.

**A quick lower bound.** An insertion or a deletion changes the length by one; a match or a substitution leaves it unchanged. So the edit distance is never smaller than the difference of the two lengths: كتب (3 letters) and مكتبة (5 letters) are at least 2 apart, and the table confirms exactly 2. A search can use this to skip a lexicon entry whose length differs from the query by more than its threshold, without filling a table (Exercise E8).

**Example 3.7**

**Check the cost convention.** For `intention` → `execution`, unit costs give distance 5. Raising the substitution cost to 2 gives distance 8. One optimal cost-2 alignment is

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| source | i | n | t | e | * | n | t | i | o | n |
| target | * | e | x | e | c | u | t | i | o | n |
| operation | D | S | S | M | I | S | M | M | M | M |

**Count the cost.** Three substitutions, one deletion, and one insertion: $3\times2+1+1=8$.

**Compare with unit costs.** With unit costs, the diagonal-first trace substitutes the first five letters (`inten`→`execu`) and matches the last four (`tion`): five operations, cost 5. The cost-2 alignment above also has five operations, yet costs 8. The number of operations and their total cost are different quantities.

### 3.2.3 CHALLENGE Weighted edit distance

**Weighted edit distance** uses the same table with different prices. Each insertion, deletion, and substitution can have its own cost, set letter by letter, so a common mistake can be cheap and a rare one dear. A match still costs 0, and no price may be negative. Two things change in the table:

1. **Every move adds its own price.** A cell is still the cheapest of the three moves, but each move adds its price instead of 1:

$$D[i][j] = \min\big(D[i-1][j] + c_{\text{del}}(s_i),\ \ D[i][j-1] + c_{\text{ins}}(t_j),\ \ D[i-1][j-1] + c_{\text{sub}}(s_i, t_j)\big).$$

2. **The first row and column add up prices.** $D[i][0]$ is the cost of deleting the first $i$ source letters, so it adds their deletion prices; $D[0][j]$ adds insertion prices. Write $0, 1, 2, 3, \ldots$ there only when every price is 1. If deleting ا costs 3 and every other deletion 1, the first column for the source الا is $0, 3, 4$.

**Example 3.8**

**A one-way spelling price.** Writing taa marbuta (ة) as haa (ه) is a habit, not a real error, so give it the price $0.25$. Every other edit costs 1.

**Fill only what changes.** For مدرسة→مدرسه, the first four letters match, so the diagonal reaches $D[4][4] = 0$, and the two neighbours of the last cell are $D[4][5] = D[5][4] = 1$. The last cell compares ة with ه:

$$D[5][5] = \min(\underbrace{1+1}_{\text{delete}},\ \underbrace{1+1}_{\text{insert}},\ \underbrace{0+0.25}_{\text{ة→ه}}) = 0.25.$$

> **Now reverse it.** For مدرسه→مدرسة the last step is ه→ة, which has no discount, so the same cell gives $\min(2, 2, 1) = 1$. The score depends on the direction, so it is not a distance in the mathematical sense. To make both directions equal, discount both.

> **CHECK THE CODE** `examples/ch03/weighted_edit_distance.py` – the weighted table with added-up borders; Lab 3, task D3, asks you to write it.

**Where the prices come from.** In pairs of intended and typed words, count which letter was typed for which (a *confusion table*). Turn each share $p$ into a price with $-\ln p$, so frequent mistakes are cheap: a mistake made one time in five costs $-\ln 0.2 \approx 1.61$. Learn the insertion and deletion prices from the same pairs, on the same scale, so the three moves can be compared fairly, then tune them on development data. A backtrace through a weighted table must use the same prices.

### 3.2.4 CHALLENGE Backtrace: from a distance to an alignment

The table gives one number, but many tasks need the alignment itself. A spelling suggester must show which letter changed. Word error rate (Section 3.2.5) needs separate counts of substitutions, deletions, and insertions. To recover the alignment, note for each cell which of the three moves gave its minimum:

- from $D[i-1][j-1]$: a match or a substitution (DIAG);
- from $D[i-1][j]$: delete $s_i$ (DOWN in Jurafsky and Martin's drawing, where the source runs up the left side);
- from $D[i][j-1]$: insert $t_j$ (LEFT).

Start at $D[|s|][|t|]$, follow the moves back to $D[0][0]$, and read the operations in reverse. When two moves tie, both lead to an optimal alignment.

For مكتبة→كتب the trace returns `IMMMI`, the shaded path in Figure 3.9.

**Listing 3.2** Backtrace that returns the two aligned rows and the operation string (`examples/ch03/edit_distance.py`)

```
def align(source, target, sub_cost=1):
    """Backtrace: return (source row, target row, operations) for one best path."""
    d = distance_table(source, target, sub_cost)
    i, j, top, bottom, ops = len(source), len(target), [], [], []
    while i > 0 or j > 0:
        same = i > 0 and j > 0 and source[i - 1] == target[j - 1]
        if i > 0 and j > 0 and d[i][j] == d[i - 1][j - 1] + (0 if same else sub_cost):
            top.append(source[i - 1]); bottom.append(target[j - 1])
            ops.append("M" if same else "S"); i, j = i - 1, j - 1      # DIAG
        elif i > 0 and d[i][j] == d[i - 1][j] + 1:
            top.append(source[i - 1]); bottom.append("*")
            ops.append("D"); i -= 1                                    # DOWN
        else:
            top.append("*"); bottom.append(target[j - 1])
            ops.append("I"); j -= 1                                    # LEFT
    return top[::-1], bottom[::-1], "".join(reversed(ops))
```

**Reading the listing.** `align` first builds the full table, then walks backwards from the last cell.

- It stores no pointers. At each cell it simply asks which neighbour, plus that move's cost, equals the current value.
- It tests the diagonal first, so ties go to a match or substitution. This is the tie-breaking rule.
- Each move appends one column to the aligned rows, with `*` for a gap, and one letter to `ops`.
- The rows were built backwards, so the last line reverses them.

The full table also needs $O(|s|\,|t|)$ space. The walk back visits at most $|s| + |t|$ cells. If you need only the distance, you can keep just two rows of the table, but then the backtrace above is no longer possible. Figure 3.13 draws the moves for the longer pair of Example 3.6, معلومات→تعليم.

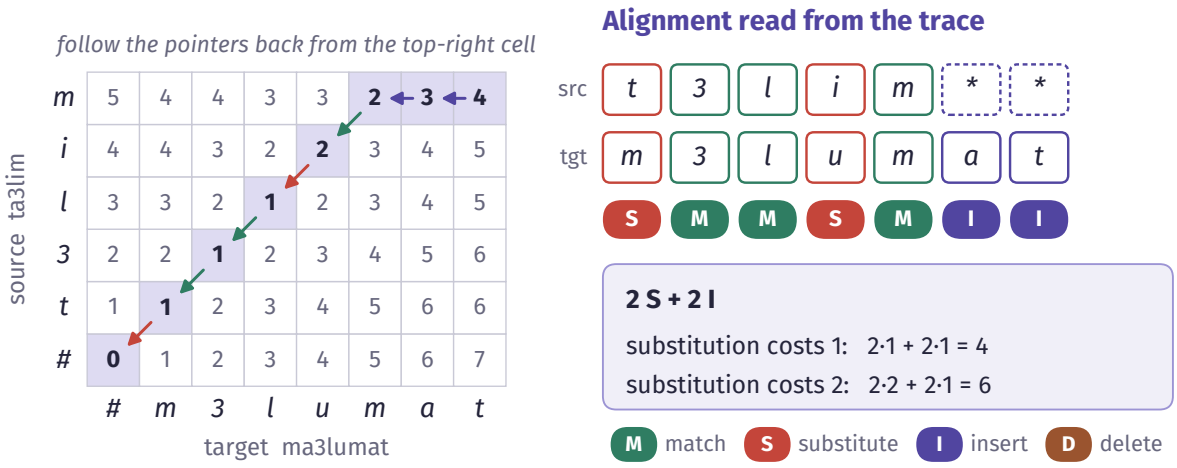



**Figure 3.13** Backtrace for تعليم→معلومات (*ta3lim→ma3lumat*), drawn with the transliterated letters in reading order. The source runs up the left side, so a deletion points down and an insertion points left. Following the pointers from the top-right cell gives S M M S M I I: distance 4 when a substitution costs 1, and 6 when it costs 2.

**TIPS**

When two neighbours explain a cell equally well, every choice leads to an optimal alignment. State your tie-breaking rule next to any alignment you report. Exercise E3 shows that the costs, and the tie-breaking rule, can change which letters line up.

### Pause, predict, then explore

The table compares درس (*dars*, "lesson", source, up the left side) with مدارس (*madaris*, "schools", target, along the bottom), letters in reading order, with every operation costing one. Start at the top-right 2. For each cell on your path, draw an arrow to the neighbour that explains its value: diagonal, down, or left. Prefer the diagonal on ties.

| | | | | | | |
|---|---|---|---|---|---|---|
| س | 3 | 3 | 3 | 3 | 3 | 2 |
| ر | 2 | 2 | 2 | 2 | 2 | 3 |
| د | 1 | 1 | 1 | 2 | 3 | 4 |
| # | 0 | 1 | 2 | 3 | 4 | 5 |
| | # | م | د | ا | ر | س |

Write the operation string and the alignment. Which letters do the insertions add?

---

**Figure 3.14** Draw the backtrace yourself, then check it against the table.

### 3.2.5 CHALLENGE Word error rate

To score speech recognition, align the system's text (the *hypothesis*) with a correct text written by a person (the *reference*). The method is edit distance, with words in place of letters. For a reference of $N$ words, **word error rate (WER)** is

$$\mathrm{WER} = \frac{S + D + I}{N},$$

where $S$, $D$, and $I$ count the substitutions, deletions, and insertions in the cheapest alignment.

**How to compute WER.**

1. Split the reference and the hypothesis into words.
2. Fill the edit-distance table over words, with every edit costing 1.
3. Backtrace one cheapest alignment (Section 3.2.4) and count $S$, $D$, and $I$.
4. Divide by $N$, the number of *reference* words, never the hypothesis words.

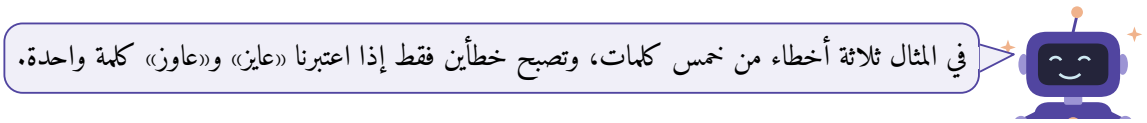
WER is not a percentage of words: it can exceed 1. For the reference "reset password" ($N = 2$) and the hypothesis "reset my password please now", the three inserted words give $\mathrm{WER} = 3/2 = 1.5$. With an empty reference, WER is undefined; state how you report that case.

**A worked case.** Take the reference انا عايز اجدد الكارنيه بكرة (*ana 3ayiz agaddid il-karneh bukra*, "I want to renew the ID card tomorrow") and the recogniser's hypothesis عاوز اجدد الكارنيه بكرة الصبح. The table lists the words in reading order, first word on the left.

| | | | | | | |
|---|---|---|---|---|---|---|
| reference | انا | عايز | اجدد | الكارنيه | بكرة | * |
| hypothesis | * | عاوز | اجدد | الكارنيه | بكرة | الصبح |
| operation | D | S | M | M | M | I |

انا is deleted, عايز becomes عاوز, and الصبح (*is-subh*, "in the morning") is inserted. The reference has 5 words, so WER is $3/5 = .60$.

But عايز and عاوز (*3ayiz*, *3awiz*) are two Egyptian spellings of the same word. If your stated policy treats them as one word in both texts, the substitution becomes a match and WER falls to $2/5 = .40$. As with normalisation, the score depends on the policy. Figure 3.15 shows the alignment and both scores.




**Pool, do not average.** Sanad's recogniser hears Nour's "yes" as "no" (1 edit, 1 reference word), and hears "I want to renew my card before the exam" with *car* for *card* (1 edit, 9 words). The mean of the two sentence WERs is $(1.00 + 0.11)/2 \approx .56$; the pooled WER is $(1 + 1)/(1 + 9) = .20$. The mean gives the one-word reply as much weight as the nine-word request, and Figure 3.16 shows the result. Pooling has its own blind spot: every word weighs the same, so *no* for *yes*, which reverses the reply, costs no more than *car* for *card*. When some errors matter more to the task, count them separately

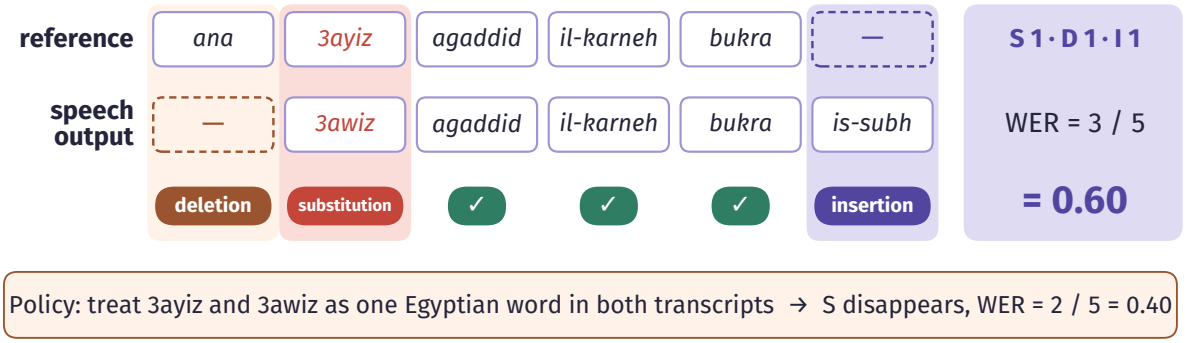



**Figure 3.15** WER is edit distance over words. The backtrace gives one deletion, one substitution, and one insertion against five reference words. A declared policy that treats عايز and عاوز (*3ayiz*, *3awiz*) as one Egyptian word lowers WER from 0.60 to 0.40.

next to WER. In short: **for a whole test set, add up all the edits and all the reference words first, then divide once.**

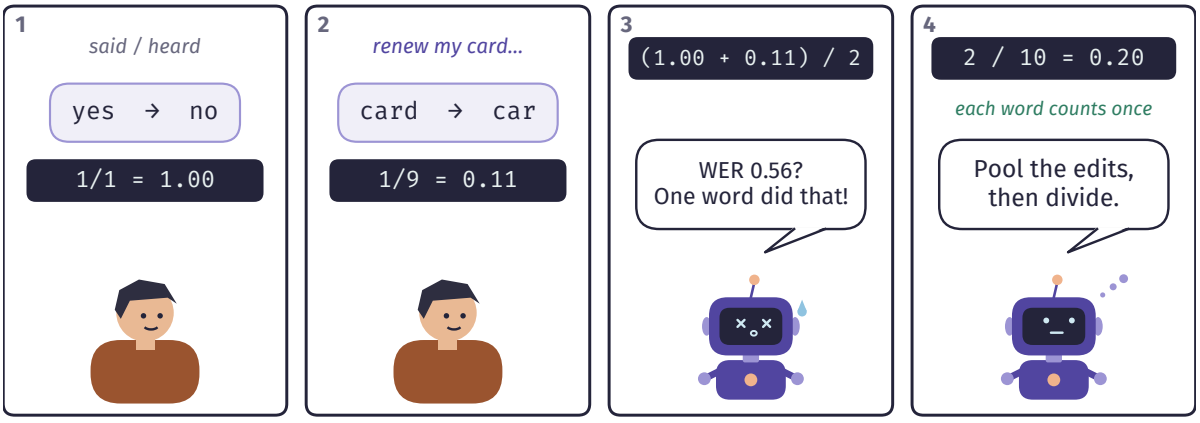



**Figure 3.16** Averaging per-sentence WERs gives a one-word reply the same weight as a nine-word request, so a single error reads as 0.56. Pooling counts every reference word once: two edits over ten words is 0.20.

## Chapter summary

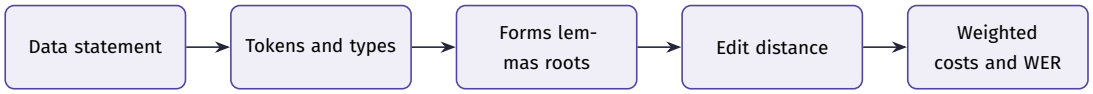
Word counts depend on the tokenization rule, so every vocabulary size must name its policy, and a corpus data statement must say where the text came from before its counts are trusted. Arabic clitics and spelling variants multiply types; lemmas and roots group them in different ways. Edit distance is the cheapest way to turn one string into another under stated costs, and WER applies it to words. Normalise variation you can list; measure distance for errors you cannot.



**CODE IT IN THE LAB MANUAL**

**Project level 3**: describe Sanad's training tickets with token and type counts per variety, then replace exact search with edit-distance matching and measure what fuzzy search gains on dev. Lab 3, part C, computes this chapter's corpus counts after Chapter 4; its part D (edit distance, weighted costs, backtrace, and WER) is on the challenge route.

**Chapter 3 at a glance**




**Keep these ideas**

- Every count depends on a stated tokenization policy; write it in the data statement.
- Types are distinct forms; rich Arabic morphology makes many types from few roots.
- Edit distance fills a table: each cell is the cheapest of delete, insert, or substitute (free if the letters match).
- The backtrace recovers an alignment; word error rate is edit distance over words divided by the reference length.

**In one small example**

`exma→exam`: the table ends in 2, with three equally good alignments (Figure 3.7).

**Common mistake.** Comparing counts made under different tokenization policies.
**Instead.** Fix one policy, state it, and recount both corpora with it.

## 3.3 Review and practice

These questions follow the chapter objectives; items added for extra practice come at the end of each list, and the progress table maps every item to its objective. Try each one on paper before you look at the answers. Items marked CHALLENGE belong to the challenge route; skip them on the core route.

### Quick quiz

Choose one answer for each question; the key is in the instructor edition.

**1.** Split on spaces, how many tokens ($N$) and types ($V$) does "pay the fee and the late fee today" have?

- **A** $N = 8, V = 8$.
- **B** $N = 6, V = 6$.
- **C** $N = 8, V = 6$.
- **D** $N = 7, V = 6$.

**2.** In the Lab 3 corpus, 61 of the 105 types occur exactly once. What are such words called?

- **A** Hapax legomena.
- **B** Stop words.
- **C** Lemmas.
- **D** Roots.

**3.** Before you trust word counts from a corpus, what should its data statement record?

- **A** Its most frequent word.
- **B** Where and why the texts were collected, their varieties and genre, and the cleaning and tokenization rules.
- **C** Its type–token ratio.
- **D** Only the number of documents.

**4.** The Lab 3 corpus has $N = 185$ tokens and $V = 105$ types. N3–N4 merge three pairs of spellings. What changes?

- **A** $N$ falls to 182.
- **B** Both $N$ and $V$ fall by three.
- **C** Neither changes.
- **D** $V$ falls to 102 while $N$ stays 185.

**5.** Which statement about كِتَاب (*kitab*, "book"), كُتُب (*kutub*, "books"), and كَتَبَ (*kataba*, "he wrote") is true?

- **A** All three share one lemma.
- **B** كِتَاب and كَتَبَ share a lemma.
- **C** They share neither a lemma nor a root.
- **D** كِتَاب and كُتُب share a lemma; all three share the root ك-ت-ب.

**6.** With every operation costing one, what is the edit distance between امتحان and الامتحان?

- **A** 1.
- **B** 2.
- **C** 3.
- **D** 0.

**7.** With unit costs and no swap move, what is the Levenshtein distance from `exma` to `exam`?

- **A** 1.
- **B** 0.
- **C** 2.
- **D** 4.

**8.** A search box must accept misspellings that users invent on the spot. Which tool fits?

- **A** A fixed table of spelling variants.
- **B** Alef normalisation alone.
- **C** Lower-casing the query.
- **D** Edit distance, because the errors cannot be listed in advance.

**9.** CHALLENGE In a backtrace, a cell's value came from the diagonal cell $D[i-1][j-1]$, and the two letters are different. Which operation is it?

- **A** A substitution.
- **B** A match.
- **C** An insertion.
- **D** A deletion.

**10.** CHALLENGE The reference is "reset my password today" and the hypothesis is "reset password today please". What is the WER?

- **A** 0.50.
- **B** 0.25.
- **C** 0.75.
- **D** 1.00.

**11.** CHALLENGE A test set has three sentences with 2, 4, and 4 reference words. The recogniser makes 1, 1, and 0 edits on them. What WER do you report for the test set?

- **A** 0.25.
- **B** 0.67.
- **C** 0.20.
- **D** 0.75.

## Check your understanding

**Q1.** Two reports give different vocabulary sizes for the same messages. Neither is wrong. How can that be, and what must each report state?

**Q2.** Arabic attaches clitics and endings to words. Why does this give Arabic text more types? What must be the same before you compare vocabulary sizes of two languages?

**Q3.** More than half of the Lab 3 corpus's types occur once. Why does that make word counts *sparse*, and why is it a problem for a model that learns from counts?

**Q4.** When should you normalise spelling variants, and when should you use edit distance instead?

**Q5.** CHALLENGE In weighted edit distance, why must the first row and column add up each letter's real cost? When can swapping source and target give a different score?

**Q6.** CHALLENGE The table keeps only the cheapest cost in each cell. Why is that safe with the costs of this chapter? Suppose a run of insertions were cheaper, so that each insertion straight after another costs 0.5. Why would one number per cell no longer be enough, and what would each cell have to remember?

## Work it through

**E1.** Count $N$ and $V$ for والطلاب يسألون عن الطلاب ("and the students ask about the students") twice: splitting on spaces only, and also splitting و off as its own token. List the types each time and give the type–token ratio. Which policy shows that the message mentions the same students twice?

**E2.** Fill the Levenshtein table for قلم → علم ("pen" and "knowledge", written without vowels). Give the distance and the operations on one best path. Why does a small distance tell you nothing about meaning?

**E3.** CHALLENGE Use the written forms كاتب (*katib*, "writer") and مكتب (*maktab*, "office"), letters in reading order. Fill the table twice: once with $c_{\text{sub}} = 1$ and once with $c_{\text{sub}} = 2$. Backtrace each one, choosing the diagonal on ties. Compare the two distances and alignments. Which one lines up the root letters ك-ت-ب?

**E4.** CHALLENGE Use these costs: delete `a` 0.5, delete `b` 2, insert `a` 1.5, insert `b` 0.25, and any substitution 3. Fill the table for `ab` → `ba`. Give the minimum cost and one edit sequence. Why does counting edits give a different answer?

**E5.** Add a fourth message, امتحانات الترم امتى ("the term exams, when?"), to the three messages of Example 3.2. Give $N$ and $V$ under P1, P2, and P3. How many times does P3 now count the stem امتحان, and why does the new message not raise that count?

**E6.** With the lexicon and threshold of Example 3.5, what does Sanad suggest for جدل (*gadal*, "argument") and for غيب (*ghayb*, "the unseen")? Are these good suggestions? Propose a rule, using the length of the query, that would make Sanad more careful with short words.

**E7.** CHALLENGE The reference is "I cannot open the quiz page" and the hypothesis is "I can open the quiz page now". Align the two and compute the WER.

**E8.** CHALLENGE **A filter for a large lexicon.** (a) Use the lower bound of Section 3.2.2 to give the smallest possible distance between غيب and محاضره without filling a table.

(b) With the lexicon and threshold of Example 3.5, which entries can Sanad skip for the query غيب without filling a table? (c) A table has $(|s| + 1)(|t| + 1)$ cells. What share of the cells does the filter save for this query? Can it ever skip the right answer?

**E9.** CHALLENGE **Costs learned from errors.** In 1,000 typed words whose intended letter was taa marbuta (ة), writers typed ة 880 times, ه 100 times, and ت 20 times. (a) Set the cost of typing each letter for ة to $-\ln p$. (b) Which substitution is cheaper, and by how much? (c) Insertions and deletions still cost 1. Why does that make the policy inconsistent, and how would you fix it?

## Read the picture

**V1.** In Figure 3.7, the typo is a swap of two adjacent letters. If a swap were one operation with cost 1, what would the distance from `exma` to `exam` be?

**V2.** Read Figure 3.13. What does a diagonal step mean when the two letters are the same, and what does it cost?

## Check your progress

Use the objective identifiers from the opening list. Quiz items check recall; Q items ask for explanation; E items, labs, and assignments ask you to calculate, implement, or report.

**Table 3.1** From Chapter 3 objectives to evidence of learning

| Goal | Preparation | Your evidence |
|---|---|---|
| O1 | Sections 3.1 and 3.1.1, Figure 3.2, Example 3.1. | Quiz 1–3; Q1, Q3; E1, E5; Worksheet 5, tasks 1, 3, and 6; project level 3, the corpus description; Lab 3, C1–C4, after Chapter 4. |
| O2 | Figures 3.4 and 3.6; Example 3.2. | Quiz 4–5; Q2; Worksheet 5, task 4. |
| O3 | Section 3.2, Examples 3.3, 3.4, and 3.5, Figure 3.12. | Quiz 6–8; Q4; E2: fill the table; E6: a fuzzy-search threshold; CHALLENGE E8: a length filter; Worksheet 5, tasks 2 and 5; project level 3, `edit.py` and fuzzy search. |
| O4 | CHALLENGE Sections 3.2.1–3.2.5, Figure 3.16. | Quiz 9–11; Q5–Q6; E3 backtrace, E4 and E9 weighted costs, E7 word error rate; Worksheet 5, tasks 7–9; Lab 3, D1–D6. |

## Further reading

Chapter 2 of Jurafsky and Martin's *Speech and Language Processing* covers corpus counts and minimum edit distance. For corpus documentation, see Bender and Friedman, Data Statements for Natural Language Processing (2018). Describe the actual sample rather than assuming that a language label establishes representativeness.

CHAPTER 4

# Tokenization, Encoding, and Preprocessing

IN THIS CHAPTER

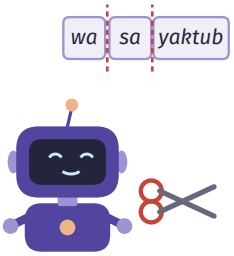

4.1 Tokenization and the vocabulary contract 84
4.2 Preprocessing without losing the task 95
4.3 An explicit English–Arabic preprocessing policy 98
4.4 From IDs back to text 100
4.5 Review and practice 103


**LEARNING OBJECTIVES**

After working through this chapter, you should be able to:

- **O1.** trace characters, bytes, tokens, IDs, vectors, and detokenization without confusing them;
- **O2.** compare word, character, byte, and subword tokenization on the same input;
- **O3.** reproduce weighted BPE merges and explain vocabulary/model compatibility;
- **O4.** justify boundary, unknown, padding, and masking conventions;
- **O5.** evaluate stemming, lemmatization, sentence segmentation, and English/Arabic normalisation against a task.

Chapter 3 counted words, and every count depended on a tokenization policy that we stated in one line. This chapter opens that line up. Before a model learns anything, a pipeline decides which characters to keep, where each token starts, and which vocabulary row each token selects. Each decision can keep a distinction the task needs or erase it.




**The goal of this chapter** is to turn Sanad's messages into model IDs by a rule you can *state*, *test*, and *reverse*. Stating the rule makes counts reproducible; testing it shows what it keeps (the course code, a negation); reversing it reveals what survives and what is lost in the tested messages.

Return to "When is the exma for CS120?". We first *split* it: into words, characters, bytes, or learned subword pieces (Section 4.1). We then *clean* it only as far as the task allows (Sections 4.2 and 4.3). Finally we *bring it back* from IDs to text and check the round trip (Section 4.4). Figure 4.1 shows the route.

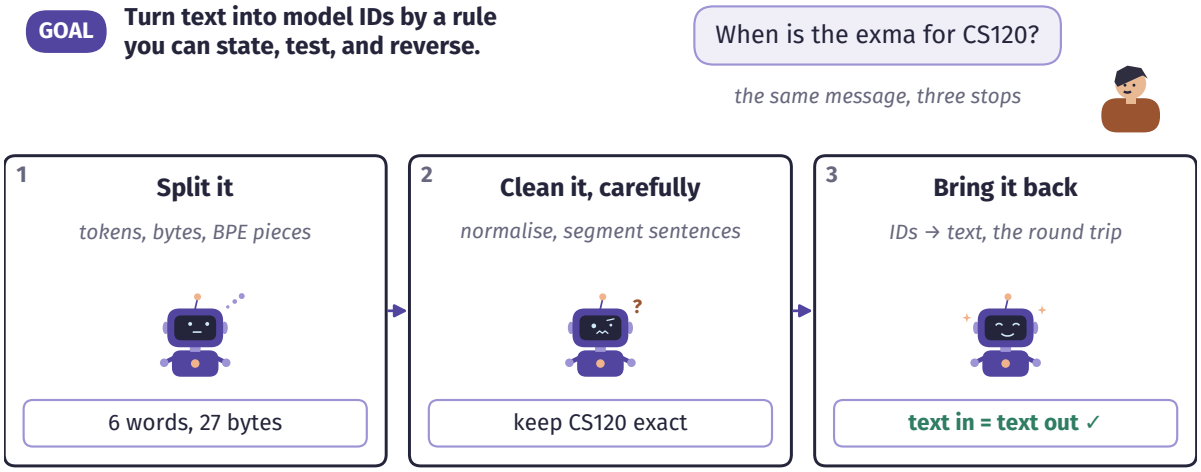



**Figure 4.1** Chapter 4's goal and route. The same message is split into units (Section 4.1), cleaned only as far as the task allows (Sections 4.2 and 4.3), and brought back from IDs to text to check that nothing was lost (Section 4.4).

## 4.1 Tokenization and the vocabulary contract

**Tokenization** divides text into units; a vocabulary then maps each unit to an integer ID. In the teaching corpus, lowercasing and splitting on spaces gives word tokens. That works because the sentences were chosen for it, not because it handles punctuation, Arabic clitics, or mixed scripts.

> **Example 4.1**
>
> **A policy changes the units.** Splitting "Students love NLP!" on spaces gives `Students`, `love`, `NLP!`. A word-and-punctuation tokenizer gives `Students`, `love`, `NLP`, `!`. Lowercasing is a third decision. Name the policy before you report a vocabulary size or a token count.

### 4.1.1 Compare tokenization policies on paper

Keep one input, "Students love NLP!", and change only the policy:

- characters, counting spaces: 18 units;
- lowercase and split on spaces: 3 tokens, with `nlp!` keeping its punctuation;
- split punctuation off as well: 4 tokens;

**Table 4.1** Token granularity and its consequences

| Unit | Benefit | Tradeoff |
|---|---|---|
| Word | Short, readable sequences for familiar words. | Large vocabularies; unseen words need a policy. |
| Character | Word forms share spelling pieces. | Longer sequences; unseen characters may still need handling. |
| Subword | Frequent pieces share statistics across forms. | The split depends on the learned vocabulary. |
| Byte | 256 byte values cover any encoded input. | More steps to represent the same text. |

- a toy subword vocabulary: yet another sequence. WordPiece notation marks continuation pieces with ##, but a greedy toy split is not a trained WordPiece model.

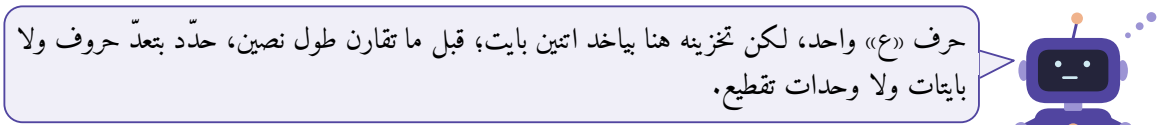
A byte tokenizer works on encoded bytes. UTF-8 stores each ASCII character in one byte, many Arabic letters in two, and some other characters in more. So bytes, Unicode code points, and the characters you see are three different counting units. "Count the characters" is incomplete until you say which one.

**Example 4.2**

The letter `A` is code point U+0041 and the single UTF-8 byte `41`. The Arabic letter *ayn*, U+0639, is stored as two bytes, `D8 B9`. A byte tokenizer sees two units; a code-point policy sees one. This check does not depend on how your terminal draws the letter:

```
letter = chr(0x0639)
print([f"{b:02X}" for b in letter.encode("utf-8")])
# ['D8', 'B9']
```

Bytes describe storage only. They give the letter no meaning and no learned vector (Figure 4.2).



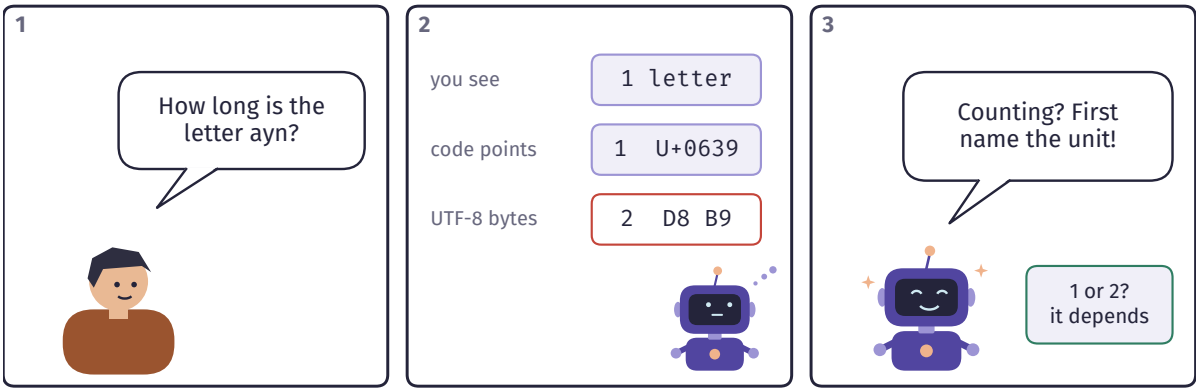



**Figure 4.2** How long is one Arabic letter? One letter on screen, one code point, two UTF-8 bytes. Every answer is right for its unit, so a length or a token budget must name the unit it counts.

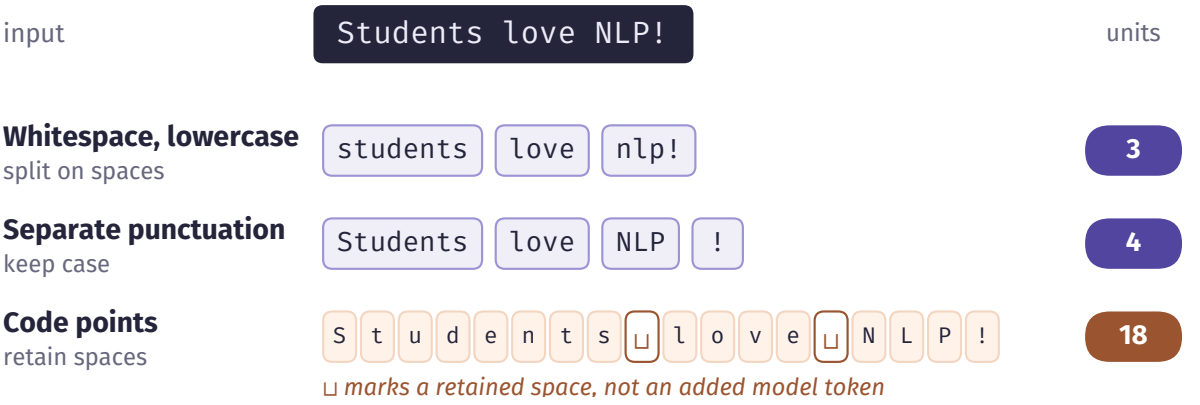



**Figure 4.3** One text, three declared token policies. The space marker denotes a retained space, not an added model token.

### 4.1.2 Why not just split on spaces?

Real help-desk messages contain units that a space split cuts apart or glues to a neighbour. Figure 4.4 marks them on one message:

- **Amounts and times.** In جنيه 150.50 (*gineh*, Egyptian pounds), *EGP 1,250*, and *10.30*, the dot and comma belong to the number. Removing punctuation breaks the amount; keeping it leaves a sentence-final full stop stuck to the last number (Example 2.3).
- **Dates.** *15/10*, in ASCII or Arabic-Indic digits, is one unit, not two numbers and a slash.
- **URLs, e-mail addresses, hashtags.** In a portal link, a staff address, or #الامتحانات (*al-imtihanat*, "#the-exams"), the punctuation is part of the token.
- **Arabizi.** *3ayez* and *a5ls* use digits as letters, so splitting digits from letters breaks the word.
- **Clitics.** و *wa-*, ب *bi-*, ل *li-*, and ال *al-* attach to the next word, so one space-separated token can hold two or three grammatical words (Section 2.1.3).
- **Multiword expressions.** عبد الله (*3abd allah*), القاهرة الجديدة (*al-qahira al-gadida*, New Cairo), and ما شاء الله (*ma sha2 allah*) act as one unit across spaces. Whether to join

them depends on the task: a name tagger can label a multi-token span, while a frequency count may keep the pieces separate.

Lab 1's tokenizer N6 (Example 2.1) handles Arabizi and mixed scripts with regular expressions. Amounts, URLs, and multiword expressions would each need another alternative and another test. Subword tokenizers, introduced below, learn their units from data instead of hand-written lists.

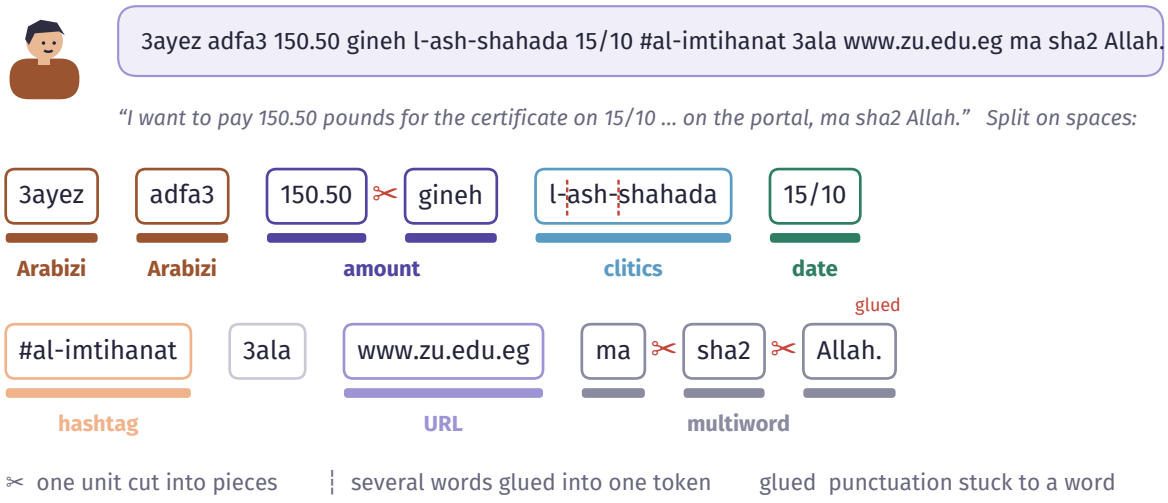



**Figure 4.4** One help-desk message split on spaces. Scissors mark a unit cut into pieces, dashed red lines mark several grammatical words glued into one token, and "glued" marks a sentence-final dot stuck to a word. The coloured bars name the unit each token belongs to.

### 4.1.3 Linguistic words and model tokens

A written word, a grammatical word, a morpheme, and a model token need not share boundaries. English `I'm` is one written form but two grammatical words. A **morpheme** is a minimal unit of meaning: `cats` is `cat` plus a plural marker. Inflection adds grammar (`worked`); derivation can change the word class (`careful`). A clitic, such as Arabic *wa-* (and), behaves like a word but attaches to its host. Chapter 2's clitic failures showed that a plausible spelling boundary need not be a morphological one.

Some writing systems, such as Chinese, Japanese, and Thai, do not mark words with spaces, so word segmentation there needs a declared convention. A learned BPE piece can also cut through a morpheme: frequency-based pieces are not a linguistic analysis.

> **Example 4.3**
>
> **Count under an explicit policy.** "They picnicked by the pool, then lay back on the grass and looked at the stars." has 16 word tokens and 14 types when punctuation is excluded and spelling is kept: the three occurrences of `the` share one type. Counting the comma and full stop as tokens raises $N$ to 18. Speech adds more choices: "I do uh main- mainly business data processing" has a filled pause and

> a cut-off word. A verbatim transcript keeps both; a cleaned one may drop them. Keep the source and state which version you counted.

### 4.1.4 Unicode units, normalization, and offsets

A *code point* identifies a character in Unicode; a font draws its glyph. A *grapheme cluster* groups code points into what a reader sees as one character. Python's `len` counts code points; `len(s.encode("utf-8"))` counts bytes. Finding grapheme boundaries needs a proper segmenter, not a rule that glues every mark to the letter before it.

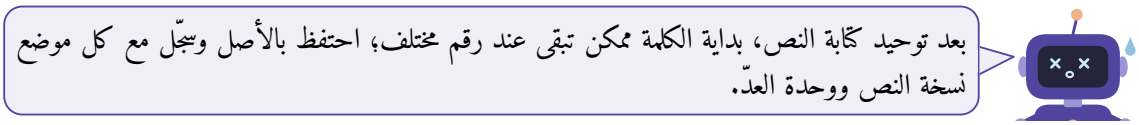
**Table 4.2** Equivalent-looking text can have different stored units

| Form | Code points | UTF-8 bytes | Length in code points |
|---|---|---|---|
| Precomposed ñ | U+00F1 | `C3 B1` | 1 |
| Decomposed ñ | U+006E, U+0303 | `6E CC 83` | 2 |
| Arabic *ayn* | U+0639 | `D8 B9` | 1 |

**NFC** composes canonically equivalent text, turning the decomposed ñ into U+00F1. **NFKC** also removes compatibility distinctions, for example mapping a circled one (U+2460) to `1`; that may help an index or erase meaningful formatting. Neither form performs Chapter 2's alef mapping or removes diacritics: those are task decisions. Choose the form, keep the raw text, and inspect what changed.

> **Example 4.4**
>
> **Offsets belong to a representation.** In `"cafe\u0301 NLP"`, `NLP` starts at code-point index 6. After NFC the `e` and its accent compose into one code point, so `NLP` starts at index 5. In `ayn + space + A`, `A` starts at code-point index 2 but byte offset 3. A span is therefore meaningful only with its unit (code points or bytes) and its text version (raw or normalised). Figure 4.5 draws both cases.

> **CHECK THE CODE** `examples/ch04/unicode_policy.py` – checks these counts without relying on glyph rendering.




**Byte pieces need not decode alone.** A byte-level token can hold part of a multibyte character. The byte `D8` alone is not a valid character; joined to `B9` it is *ayn*. So join a token sequence's bytes *before* decoding, rather than decoding each token on its own. Covering every byte guarantees that any input can be represented; it does not guarantee that every generated byte sequence is valid text.

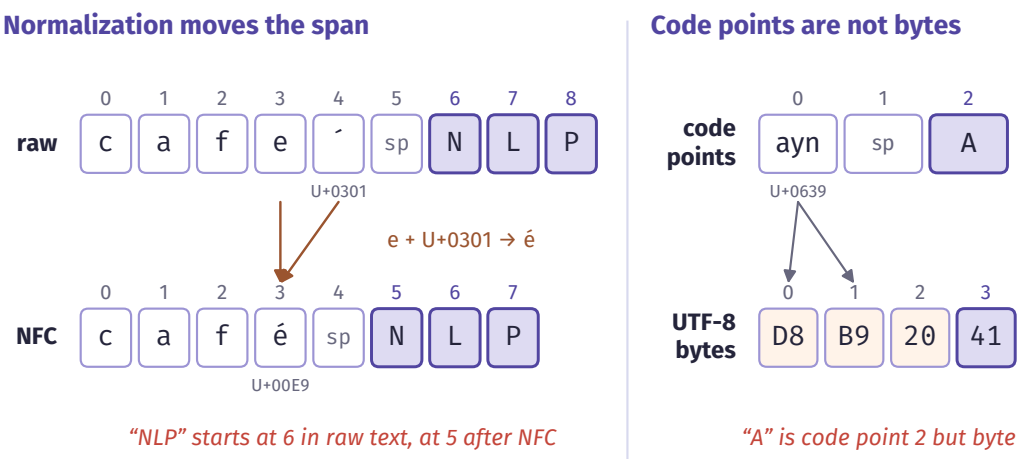



**Figure 4.5** Offsets belong to a representation. Left: NFC composes e and the combining accent U+0301 into one code point, so the span NLP moves from index 6 to index 5. Right: *ayn* is one code point but two UTF-8 bytes, so A is at code point 2 and byte 3.

### 4.1.5 Vocabulary size, sequence length, and unknowns

Each policy trades vocabulary size against sequence length:

- a word vocabulary needs one entry per surface form it keeps apart (lowercasing merges `Students` and `students`);
- a character vocabulary is small but needs more prediction steps;
- a subword vocabulary keeps frequent words whole and splits rare ones.

Longer sequences leave less room in a fixed token budget, and the same content can need more tokens in one language than another. Measure tokens per word on real documents rather than assuming a fixed factor for Arabic.

Unknown words are handled differently too. A word model may map unseen `university-wide` to one unknown marker and lose its spelling; a subword model can build it from smaller pieces. A byte fallback makes every input representable, but not every prediction useful.

### 4.1.6 Learning subword pieces

**Byte-pair encoding** (BPE) starts from small units and repeatedly merges the most frequent adjacent pair. Take `low` twice and `lower` once, written as characters with a word-end marker. The pairs `l o` and `o w` each occur three times. If the tie rule picks `l o`, it becomes `lo`; then `lo w` occurs three times and becomes `low`. Both words now share the piece `low`.

WordPiece and unigram tokenizers learn their vocabularies differently, and SentencePiece is a framework that supports several algorithms (Section 4.1.9). "Subword" alone does not guarantee coverage: only a full byte alphabet rules out unknown input.

### 4.1.7 Weighted BPE, step by step

Take the weighted corpus `low`:5, `lower`:2, `newest`:6, `widest`:3. Start from characters plus an end marker `_`. Each adjacent pair adds its word's frequency, so a pair inside `newest` counts six, not one.

At the start, `e s`, `s t`, and `t _` each count $6 + 3 = 9$ (from `newest` and `widest`). `w e` counts $2 + 6 = 8$, and `l o` and `o w` count $5 + 2 = 7$. Ties need a rule. Ours scans words and pairs in corpus order and keeps the first pair that *beats* the best count so far, so `e s` wins first.

**Table 4.3** The first five merges under the stated tie policy

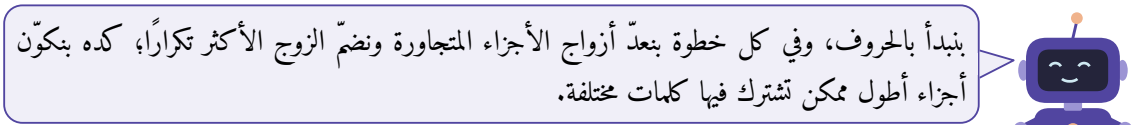
| Step | Selected pair | Weighted count | Effect |
|---|---|---|---|
| 1 | `e + s` | 9 | `es` is shared by `newest` and `widest`. |
| 2 | `es + t` | 9 | The shared piece becomes `est`. |
| 3 | `est + _` | 9 | An ending piece `est_` is formed. |
| 4 | `l + o` | 7 | `lo` is shared by `low` and `lower`. |
| 5 | `lo + w` | 7 | The shared piece becomes `low`. |

After step five, the segmentations are

| Word | Segmentation |
|---|---|
| low | `low _` |
| lower | `low e r _` |
| newest | `n e w est_` |
| widest | `w i d est_` |




**Figure 4.6** Weighted BPE shares an ending across two words before learning the beginning of two others. Counts are weighted occurrences, not numbers of distinct words.



**Pause, predict, then explore**

Use the five merges already learned: `e+s`, `es+t`, `est+_`, `l+o`, `lo+w`. Draw joins over this new word:

```
l o w e s t _
```

Write the final pieces. Explain why the new word must not change the learned counts.

---

**Figure 4.7** Apply the learned BPE rules to new input without relearning frequencies.

To segment the unseen word `lowest`, apply the *learned merges in order*; do not recount anything. Its characters first merge into `est_`, then `lo`, then `low`, giving `low est_`. Pieces learned from other words can represent a word never seen in training (Figure 4.8).

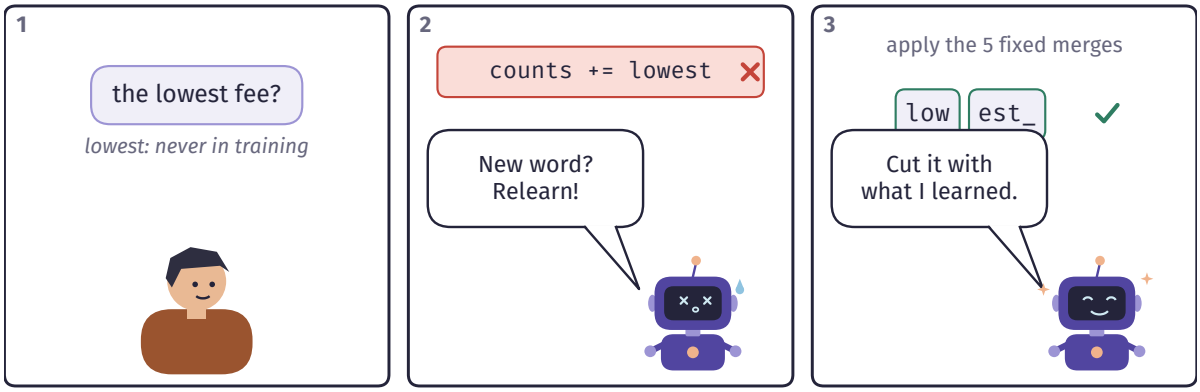
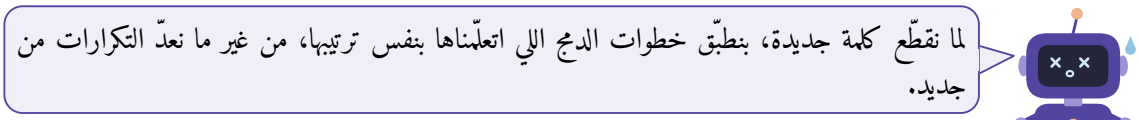



**Figure 4.8** A word never seen in training is cut with the merges already learned, giving `low est_`. Adding it to the counts would change the tokenizer itself.

> **Training and application are separate algorithms.** Training learns a list of merges from a corpus. Application uses that fixed list on new text. A deterministic tie rule makes this toy example reproducible; another valid tie rule could learn a different list.




### 4.1.8 Pre-tokenization and leading boundaries

*Pre-tokenization* limits which units BPE may merge. Many pipelines first split text into chunks with a regular expression, then merge only inside each chunk; others learn from raw sentences and allow pieces that cross spaces. Record which one you used.

Take the six-word corpus `set new new renew reset renew`. For a toy version, use the lowercase Latin alphabet plus a marker _, put _ in front of every word, and never

merge across words. (This _ marks a word start, unlike the end marker above.) The word frequencies are `set`:1, `new`:2, `renew`:2, `reset`:1. Count each pair with its word's frequency; break ties by preferring `n + e`, then by alphabetical pair order, with the marker _ sorted before every letter.

**Table 4.4** Four leading-boundary BPE merges

| Step | Pair | Count | Reason |
|---|---|---|---|
| 1 | `n + e` | 4 | Two `new` and two `renew` occurrences. |
| 2 | `ne + w` | 4 | The same four occurrences after step 1. |
| 3 | `_ + r` | 3 | Two `renew` and one `reset`; wins the stated tie. |
| 4 | `_r + e` | 3 | Shared initial sequence in the same three words. |

The result is `[_, new]` for `new`, `[_re, new]` for `renew`, and `[_re, s, e, t]` for `reset`. `_re` looks like a prefix, but the algorithm only counted symbols; it did not analyse morphology. Applying the fixed merges to unseen `rearrange` gives `[_re, a, r, r, a, n, g, e]`: the alphabet covers `a` and `g` although they never occurred in training. After four merges the vocabulary has $27 + 4 = 31$ entries, but only six distinct pieces appear in the segmented training words.

> **CHECK THE CODE** `examples/ch04/leading_space_bpe.py` – reproduces the four merges and checks application separately.

This toy marker cannot restore tabs, repeated spaces, or exact spacing, so a real tokenizer must say what detokenization preserves. The pre-tokenization pattern also decides which contractions, numbers, and punctuation can merge. Note that Python's built-in `re` has no Unicode property escapes such as `\p{L}`; the separate `regex` package does. Name the engine before copying a pattern.

### 4.1.9 Compare subword algorithms without conflating them

- **BPE** merges the most frequent pair, step by step.
- **WordPiece** chooses merges by a different score (below) and marks continuation pieces with `##`.
- **Unigram** starts from many candidate pieces, each with a probability. It segments a word by the split whose pieces have the highest product of probabilities, and training removes the pieces whose loss lowers the corpus likelihood least.
- **SentencePiece** is a framework, not an algorithm: it trains BPE or unigram models directly from raw sentences.

**Example 4.5**

**Same corpus, different score.** BPE merges the pair with the largest count $C(ab)$. WordPiece merges the pair with the largest

$$\text{score}(a, b) = \frac{C(ab)}{C(a)\,C(b)},$$

with every count weighted by word frequency. Redo the weighted corpus of Section 4.1.7 (`low`:5, `lower`:2, `newest`:6, `widest`:3) with the same symbols and the same tie rule. BPE's first choice, `e s`, now scores only $9/(17 \cdot 9) = 1/17$, because `e` occurs 17 times and mostly in other pairs.

| Step | Pair | $C(ab)$ | $C(a)$ | $C(b)$ | Score |
|---|---|---|---|---|---|
| 1 | `i + d` | 3 | 3 | 3 | $1/3$ |
| 2 | `l + o` | 7 | 7 | 7 | $1/7$ |
| 3 | `s + t` | 9 | 9 | 9 | $1/9$ |

WordPiece first joins `i d`, a pair seen only three times, because `i` and `d` never occur apart. After three steps the words read `lo w _`, `lo w e r _`, `n e w e st _`, and `w id e st _`; BPE had already built `est_`. The score rewards pieces whose parts belong together, not pieces that are merely frequent.

**CHECK THE CODE** `examples/ch04/wordpiece_score.py` – reproduces both merge orders and these scores.

The same input therefore has no single "subword segmentation". Read each marker (`##`, an end marker, a leading-space marker) by its own tokenizer's rules; they are not interchangeable.

### 4.1.10 Special tokens and compatibility

Special tokens each have one job:

- start and end markers mark sentence boundaries;
- padding fills short sequences so a batch has equal lengths;
- a mask hides positions a training objective must not see;
- an unknown marker collects words missing from a closed vocabulary.

An encoder's classification or separator marker is not automatically a generator's start or end token: the meaning comes from how the model was trained.

The model and its vocabulary mapping form one *contract*. In the sorted vocabulary of the running corpus (Chapter 7 builds a model on it), ID 2 means `i`. Load a different mapping and row 2 names a different word, even though every matrix has the same shape (Figure 4.9). Always save the mapping with the model.

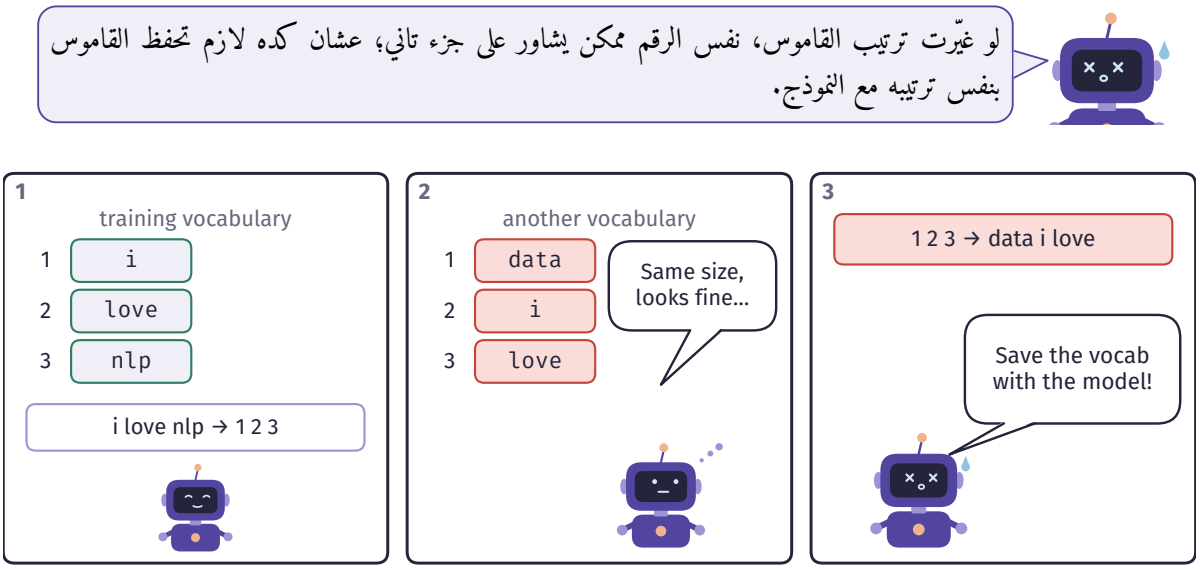



**Figure 4.9** The vocabulary mapping is part of the model. The IDs 1 2 3 mean "i love nlp" under the training vocabulary and "data i love" under another one of the same size.

Tokenization also decides how much text fits in a token-limited context. A tokenizer may cut comparable Arabic text into more pieces than English; measure the ratio on representative documents, since it depends on the tokenizer, the writing style, and the data.

Table 4.5 does that measurement for two real byte-level BPE vocabularies: GPT-2's, learned mostly from English web text, and the multilingual vocabulary of the Whisper speech model. Both use the same pre-tokenization pattern and about 50,000 tokens. The input is eight help-desk requests, each written once in English and once in Arabic (*I want to postpone the exam*, *When is the exam?*, and six more).

**Table 4.5** Tokens per word and per message for the same eight requests

| | English (46 words) | | Arabic (27 words) | | |
|---|---|---|---|---|---|
| **Tokenizer** | **Tokens** | **Per word** | **Tokens** | **Per word** | **Arabic / English** |
| GPT-2 | 48 | 1.04 | 165 | 6.11 | 3.4 |
| Multilingual | 50 | 1.09 | 93 | 3.44 | 1.9 |

Read the last column, not the per-word columns. An Arabic word often holds what English writes as two or three words, such as an article or a pronoun, so the same eight requests have 27 Arabic words and 46 English ones. Per message, GPT-2 needs 3.4 times as many tokens for the Arabic, and the multilingual vocabulary 1.9 times. For one request, *I want to postpone the exam*, GPT-2 uses 6 tokens for the English and 20 for the Arabic; the multilingual vocabulary uses 7 and 12. A context window of a fixed size therefore holds far less Arabic, and an API that charges per token charges more for it. The ratio belongs to a tokenizer and a data set, so measure it again on your own documents.

**CHECK THE CODE** `examples/ch04/real_tokenizers.py` – stores the measured tokens and asserts every number in this table.

**KEY POINT**

Tokens are model units, not necessarily words. A token count is meaningful only with a stated tokenizer. Vocabulary coverage and language coverage are properties to measure, not assumptions to make from an algorithm's name.

## 4.2 Preprocessing without losing the task

Preprocessing may include Unicode normalisation, lowercasing, punctuation handling, stopword removal, stemming, or lemmatization. Every operation discards or changes information, so its value depends on what the task must keep.

A **stemmer** cuts affixes by rules and may return a non-word. A **lemmatizer** looks up a dictionary form using lexical and sometimes grammatical knowledge; it too can make mistakes. The noun *meeting* and the verb *meeting* need not share a lemma.

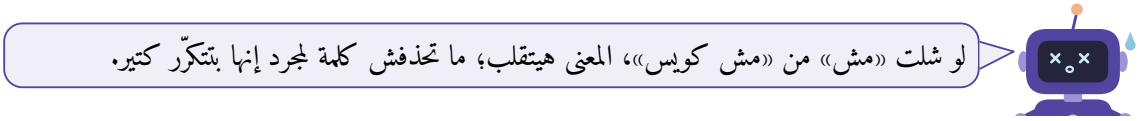
**Table 4.6** Illustrative English stemming and lemmatization decisions

| Input | Porter-style stem | Lemma | Condition |
|---|---|---|---|
| running | run | run | Verb reading. |
| studies | studi | study | Dictionary form differs from the stem. |
| better | better | good | Adjective analysis; an irregular comparison. |
| was | wa | be | Verb analysis and lexical knowledge. |
| meeting | meet | meeting / meet | Noun or verb reading changes the lemma. |

*Over-stemming* merges forms whose difference matters; *under-stemming* leaves related forms apart. Arabic root extraction groups library, writer, and writing under k–t–b, and school forms under d–r–s. A shared root is not a shared meaning: a search for a library should not return every text about writing. *Light stemming* removes only selected affixes and keeps more of the word. Neither is always better; decide whether the task wants those words to match.

**Example 4.6**

A sentiment pipeline removes the stopword `not` from "not good". Its features become identical to those of "good": a smaller vocabulary has cost the one distinction the classifier needed. Similarly, lowercasing or dropping punctuation can erase a name an extractor must find.




For a pre-trained model, follow its documented preprocessing; some Arabic models expect segmented text and others expect raw text. In every case, keep an untouched copy, compare policies on held-out data, and test dialect and code-switched samples separately rather than reporting one pooled score.

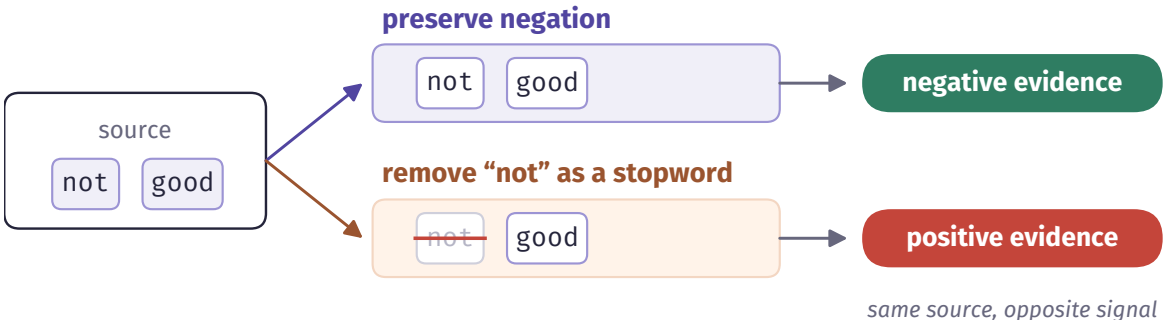
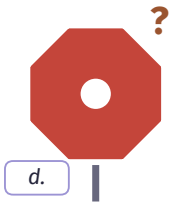



**Figure 4.10** A transformation can erase task-relevant information. The original text and the declared policy belong in the audit trail.

### 4.2.1 Sentence segmentation




Many pipelines need sentences as well as tokens: a classifier labels each request, a summariser picks sentences, a language model marks where they start and end. **Sentence segmentation** looks easy: split after a full stop, a question mark, or an exclamation mark. The full stop is the hard case, because it also ends abbreviations and sits inside numbers:

- د. abbreviates دكتور (*duktur*, Dr.), أ. أستاذ (*ustadh*, Prof.), م. مهندس (*muhandis*, Eng.), and ج.م the Egyptian pound;
- times and amounts such as *10.30* contain a dot with no space after it;
- Arabic has its own question mark (U+061F); its comma (U+060C) and semicolon (U+061B) separate clauses but do not end a sentence;
- chat messages often use line breaks instead of punctuation, or no punctuation at all.

Figure 4.11 shows the abbreviation problem in one message.

**Listing 4.1** A rule-based sentence splitter with an abbreviation check (`examples/ch04/sentence_split.py`)

```
BOUNDARY = re.compile(r"(?<=[.!?\u061f])\s+|\n+")  # mark+space or newline
ABBREVIATION = re.compile(       # d. (Dr.), a. (Prof.), m. (Eng.), g.m.
    r"(?:^|\s)(?:\u062f|\u0623|\u0645|Dr|Prof)\.$|\u062c\.\u0645\.?$")

def split_sentences(text):
    sentences, current = [], ""
```

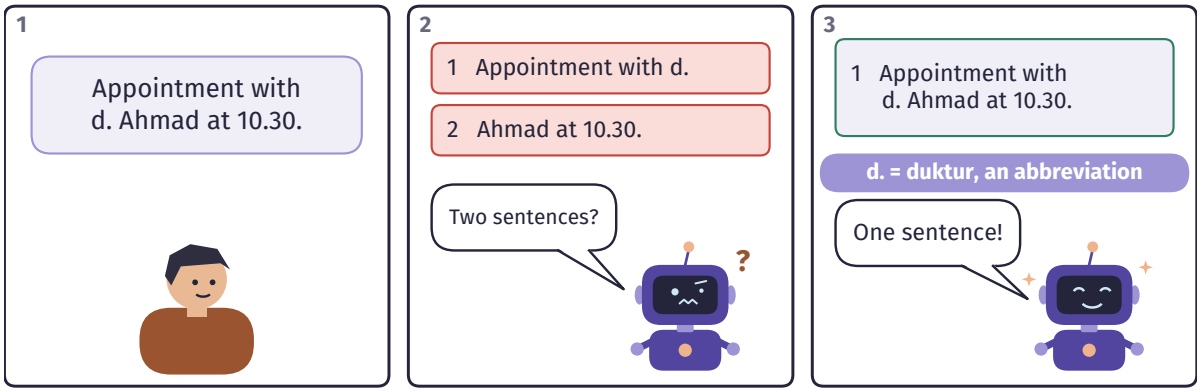


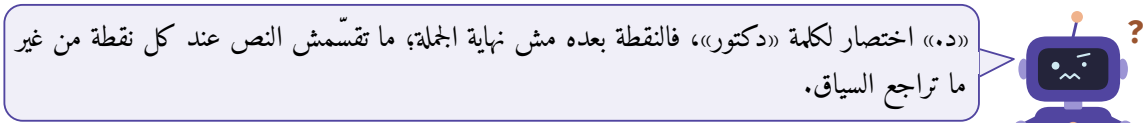

**Figure 4.11** Splitting at every full stop cuts Nour's message after the abbreviation *d.*; an abbreviation check keeps it one sentence.

```
    for piece in BOUNDARY.split(text):
        current = current + " " + piece if current else piece
        if not ABBREVIATION.search(current):     # a title is not the end
            of a sentence
            sentences.append(current.strip())
            current = ""
    if current:
        sentences.append(current.strip())
    return [s for s in sentences if s]
```

Listing 4.1 treats an end mark followed by whitespace, or a line break, as a candidate boundary, and rejects the candidate when the text before it ends with a known abbreviation. The script asserts five correct decisions:

- a three-sentence complaint ending in a full stop, an Arabic question mark, and an exclamation mark is split into three;
- "an appointment with د. أحمد (Dr Ahmad) at 10.30" stays one sentence;
- an English message with *Dr.* is split into two;
- a line break counts as a boundary;
- a sentence with an Arabic comma and semicolon stays whole.

It also records two known failures. An unpunctuated chat message with two requests stays one sentence. A fee ending in *g.m.* at the end of a sentence is merged with the next one, because one dot ends both the abbreviation and the sentence. English splitters often use a capital letter after the dot as evidence, but Arabic script has no capitals. Fixing either failure needs context or a trained model; NLTK's Punkt, for example, learns abbreviations from unannotated text. Figure 4.12 summarises the rule and its failures.



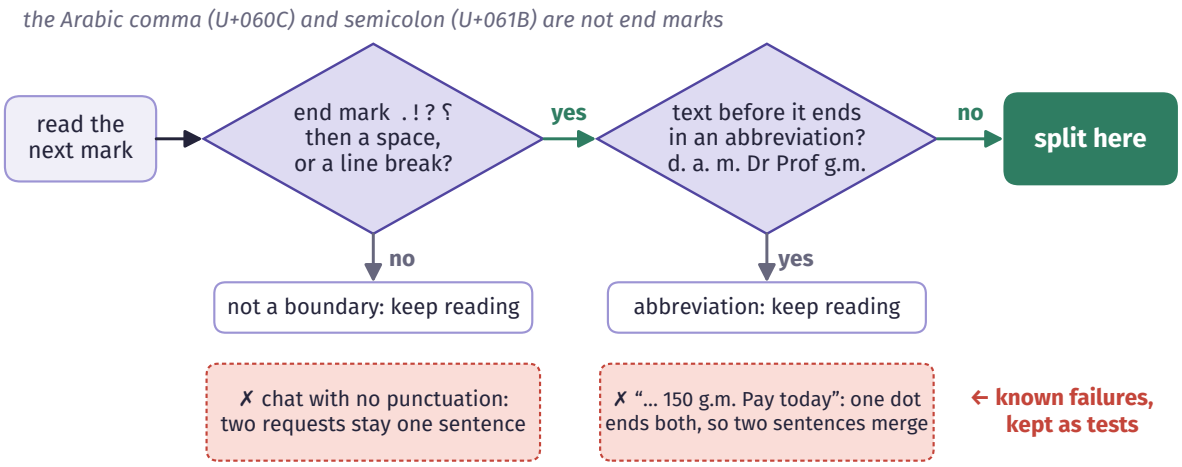



**Figure 4.12** The splitter of Listing 4.1 as a decision flow.

## 4.3 An explicit English–Arabic preprocessing policy

Seven common operations are lowercasing, punctuation removal, diacritic removal, tatweel removal, alef normalisation, stopword removal, and light stemming. Switching all of them on is not "clean text"; it is seven decisions. For each one, name the requirement it serves.

| Written example | Form to compare | What the distinction illustrates |
|---|---|---|
| وسيكتبونها | و + س + يكتبون + ها | Attached conjunction, future marker, verb, and object pronoun: "and they will write it". |
| علم | عِلْم / عَلَم / عَلِمَ | The same unvocalised letters can correspond to science, flag, or "he knew". Context and diacritics distinguish readings. |
| يَدرسونَ | يدرسون | Removing diacritics changes pronunciation evidence without changing these base letters. |
| الطبيعيــة | الطبيعية | Removing tatweel removes a presentation stretch. It is different from dropping a letter. |
| أ / إ / آ | ا | Alef normalisation merges distinct written forms. Its usefulness depends on the task. |
| والطلاب | طلاب | An illustrative affix-removal policy strips the conjunction and definite article. It is not a universal morphological analysis. |

**Figure 4.13** Arabic forms with script preserved. Each transformation illustrates a stated policy rather than a universal tokenizer or analyser.

### 4.3.1 Choose the order and preserve an audit trail

This subsection makes two points: run the steps in one fixed order, and keep a record of what they did.

**Each step changes something different.** Lowercasing touches only English letters. Diacritic removal and tatweel removal delete different Arabic marks. Alef normalisation goes further: it changes the spelling itself. Removing punctuation can erase a sentence boundary, and removing stopwords can delete a word the task needs, such as a negation.

**Order changes the result.** A stopword list written for normalised spellings misses a word that has not been normalised yet. A stemmer sees different input before and after punctuation is split off. So fix the order and use the same order in training and in serving; otherwise the model receives inputs it never saw.

**Keep an audit trail.** For every message, save four things: the original text, the transformed text, the tokens, and the policy version. Then any output can be traced back to the input that produced it.

One caution: the stemmers in the companion lecture slides' interactive widget are toy rules that strip a few affixes. They are not full Porter stemming or Arabic morphological analysis, so they may disagree with a published table. When you report a stemmed output, name the tool that produced it.

**Example 4.7**

**Search and pronunciation need different information.** A course search wants vocalised and plain spellings to match; a speech synthesiser needs the vowel marks to pronounce the word. Removing diacritics helps the first and hurts the second. Keep the source text even when a search index uses normalised forms. Figure 4.13 shows the same unvocalised letters supporting different readings.

Table 4.7 applies this to the one vocalised request in the Lab 3 corpus, المُحاضَرَةُ أُلْغِيَتْ اليوم (*al-muhadaratu ulghiyat al-yawm*, "the lecture was cancelled today"). Under every policy the message keeps three tokens and the corpus keeps $N = 185$; only which spellings count as the same word changes. Search wants the lower rows, where the vocalised spelling joins the plain ones; a speech synthesiser needs the first row. The last row is the 105-to-102 drop of Figure 3.6(b).

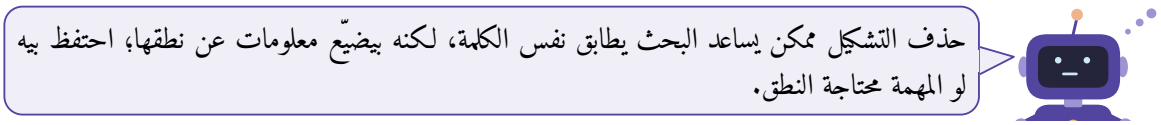


**Table 4.7** One vocalised Lab 3 request under cumulative normalisation policies

| Policy | The request becomes | Corpus $V$ | What merges |
|---|---|---|---|
| None | Vocalised الْمُحَاضَرَةُ | 107 | Nothing: the vocalised form is a type seen once. |
| N1 | Marks removed: bare المحاضرة | 106 | It joins two plain spellings (count 3). |
| N1–N2 | Unchanged | 105 | A tatweel-stretched spelling joins too (count 4). |
| N1–N4 | Hamza dropped from ألغيت; final taa marbuta written as haa | 102 | Three more pairs: أريد, الاختبار, and المنصة. |

### 4.3.2 Use tests that reflect the intended distinctions

A preprocessing test should check both what changes and what must stay:

- removing a tatweel keeps the letters on either side;
- the chosen alef mapping is applied;
- a negation token survives for sentiment;
- a course code stays exact for extraction;
- an empty result is not treated as a normal sentence.

Include mixed-script text and a word unseen when the vocabulary was fitted.

A step can do exactly what it promises and still hurt the model. Compare policies on the same held-out split with a task metric, and look at the cases where merging forms raised recall but lowered precision by joining different names or meanings.

For a pre-trained model, check its own preprocessing and tokenizer. AraBERT shows why both "always leave Arabic unchanged" and "always segment Arabic" are too broad: versions of one model family expect different input. Match the checkpoint you actually use, and record it with the experiment.

**KEY POINT**

Text preprocessing is a modelling choice with a traceable purpose. The right pipeline preserves the distinctions the task needs, supplies the units the model expects, and behaves identically during training, evaluation, and serving.

## 4.4 From IDs back to text

To inspect what a tokenizer preserves, we also need the return path. **Detokenization** maps IDs back through the vocabulary and rebuilds readable text by the tokenizer's rules: a leading-space marker shows where a space goes, and special markers such as the end marker are not printed. A *round trip*, text to IDs and back, is a simple test of a

policy: inspect every difference and decide whether it is an intended transformation, an unacceptable loss, or an implementation fault. Figure 4.14 shows a round trip that quietly breaks extraction. Chapter 7 connects the round trip to a model that predicts the next token.

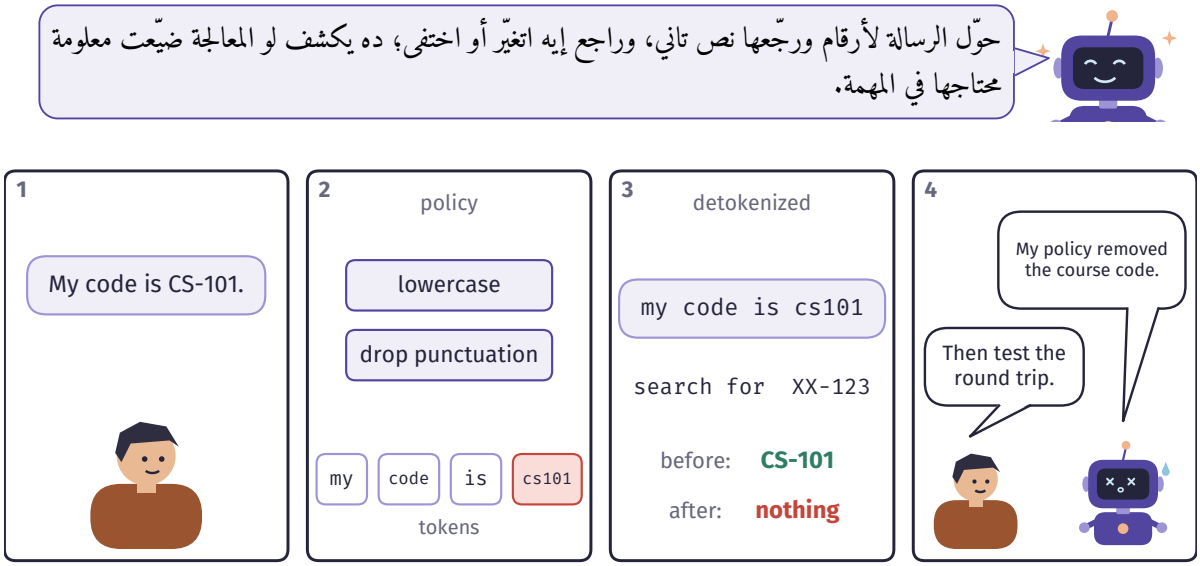



**Figure 4.14** A round trip through a lowercase-and-strip-punctuation policy. The detokenized text looks harmless, but the course-code pattern no longer matches: the policy, not a model, removed the code. The script asserts both searches.

**Example 4.8**

**What comes back, and what does not.** Take a declared word-level policy: lowercase; words and single punctuation marks are tokens; a closed vocabulary of nine words and marks plus `<unk>`; detokenization joins tokens with spaces and removes the space before a question mark. Send this message, which ends with متى الاختبار؟ (*mata al-ikhtibar?*, "When is the exam?"), through it and back:

| | |
|---|---|
| In | When is the exma for CS120? متى الاختبار؟ |
| Tokens | `when is the exma for cs120 ? متى الاختبار ؟` |
| IDs | `1 2 3 0 5 0 6 7 8 9` |
| Out | when is the <unk> for <unk>? متى الاختبار؟ |

The Arabic half returns unchanged, because its three tokens are in the vocabulary. Three words do not. *When* became *when*: lowercasing was a choice. *exma* became `<unk>`: the typo is lost, so a spelling corrector can no longer see it. *CS120* became `<unk>` as well, and that is the course code the request is about. Only one change in three was intended. The round trip names all three, so you can decide which one to fix.

> **CHECK THE CODE** `examples/ch04/round_trip.py` – asserts the tokens, the IDs, and the three changed words.

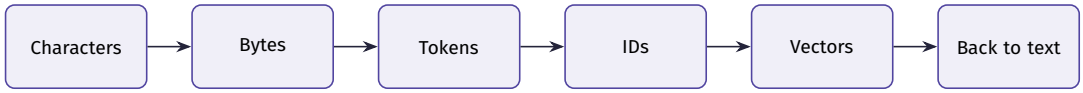
**A byte-level round trip is exact, if you decode at the end.** A byte-level BPE vocabulary has no `<unk>`: every input byte has a token, so joining the tokens' bytes and decoding them returns the input exactly. All 16 requests of Table 4.5 come back unchanged under both vocabularies. The trap is decoding too early. GPT-2 splits أريد (*urid*, "I want") into five tokens, `D8 | A3 | D8 B1 | D9 8A | D8 AF`: the first letter, alef with hamza (U+0623), is cut across two tokens. Decoded one token at a time, those two give the replacement character U+FFFD twice, and the word arrives broken. Joined first, the five tokens give أريد again. The multilingual vocabulary keeps the letter whole and needs three tokens. A program that streams generated text must therefore hold back an incomplete character until its last byte arrives (Section 4.1.4).



**CODE IT IN THE LAB MANUAL**

**Lab 3, part C** (corpus statistics under the normalisation policies of Table 4.7). **Project level 4**: write Sanad's shared tokenizer down as a declared policy and fit BPE on the training tickets; add NFC to the shared normaliser (Section 4.1.4); measure the unknown-token rate by language variety; start the keyboard service; and decide, from every service's dev scores, which tokenizer all three share.

## Chapter 4 at a glance




### Keep these ideas

- Keep the layers apart: characters, bytes, tokens, IDs, vectors; each has its own count.
- Word, character, byte, and subword tokenizers trade vocabulary size against sequence length.
- BPE merges the most frequent adjacent pair, again and again; the merge list is part of the model.
- A model only understands its own vocabulary: tokenize with the tokenizer it was trained with.
- Normalise only what the task allows you to lose.

### In one small example

Weighted BPE shares an ending across two words before it learns the beginnings (Figure 4.6).

**Common mistake.** Stripping negation words or diacritics as "noise".
**Instead.** Test each preprocessing step against the distinctions the task must keep.

## 4.5 Review and practice

These questions follow the chapter objectives in order. Explain the reasoning before checking the optional answers.

### Quick quiz

Choose one answer for each question before reading the rest of the review; the key is in the instructor edition.

**1.** How is the Arabic letter *ayn* (U+0639) stored in UTF-8?

- **A** As one byte.
- **B** As the two bytes `D8 B9`.
- **C** As four bytes.
- **D** It cannot be encoded in UTF-8.

**2.** After lowercasing, how many tokens does whitespace splitting give for “Students love NLP!”?

- **A** 4.
- **B** 18.
- **C** 3.
- **D** 2.

**3.** In the weighted corpus `low`:5, `lower`:2, `newest`:6, `widest`:3, what is the initial count of the pair `e s`?

- **A** 6.
- **B** 3.
- **C** 8.
- **D** 9.

**4.** Applying the five learned merges to the unseen word `lowest` gives:

- **A** `low est_`
- **B** `lowest_`
- **C** `l o w e s t _`
- **D** `lo west_`

**5.** A model is loaded with a different vocabulary mapping of the same size. What happens?

- **A** Nothing; only the size matters.
- **B** Row IDs now name different words, so inputs and outputs are misread.
- **C** The tokenizer corrects the difference automatically.
- **D** The model raises an error automatically.

6. Which tool can map *better* to *good*?
   - **A** A suffix-stripping stemmer.
   - **B** BPE.
   - **C** A lemmatizer using lexical knowledge.
   - **D** Lowercasing.

7. A sentiment pipeline removes the stopword `not`. What is the risk?
   - **A** None; stopwords carry no information.
   - **B** It only affects Arabic text.
   - **C** It always improves recall.
   - **D** "not good" can become identical to "good".

8. Why is "split after every full stop" a poor sentence splitter for help-desk Arabic?
   - **A** Arabic never uses the full stop.
   - **B** The dot also ends abbreviations such as *d.* and *g.m.*, and Arabic has no capital letters to signal a new sentence.
   - **C** The Arabic comma always ends a sentence.
   - **D** Full stops are removed by UTF-8 encoding.

9. The six-letter word امتحان (*imtihan*, exam) has no diacritics. How many UTF-8 bytes does it use?
   - **A** 6.
   - **B** 3.
   - **C** 12.
   - **D** 24.

10. What is padding for?
    - **A** It replaces words missing from the vocabulary.
    - **B** It marks where a sentence ends.
    - **C** It hides the next word during training.
    - **D** It fills short sequences so a batch has equal lengths.

## Check your understanding

**Q1.** Why do words, characters, bytes, and subwords produce different lengths?

**Q2.** In the weighted BPE example, why does the first merge have count nine?

**Q3.** Can a token ID be interpreted as a semantic magnitude?

**Q4.** Why is stemming inappropriate as an automatic preprocessing rule for every model?

**Q5.** Explain how NFC, NFKC, and task-specific alef normalization differ. Why must an extracted span declare its offset unit and text version?

**Q6.** In the leading-boundary example, why is the third merge count three? Why does learning a piece resembling a prefix not prove a morphological analysis?

**Q7.** Why must a byte-level tokenizer join a token sequence's bytes before decoding them?

**Q8.** What does a tokenize-then-detokenize round trip test, and what does it not test?

## Work it through

**E1.** Apply the five learned BPE merges to the unseen word lowest. Show every applicable merge and explain why the training corpus is not updated.

**E2.** Write separate preprocessing policies for Arabic document search and for reading that is sensitive to pronunciation. Include one transformation you would test for each.

**E3.** Propose one change to Listing 4.1 that fixes the ج.م failure without breaking the د. أحمد case. State the new rule, one input it now gets right, and one input it could now get wrong.

**E4.** For `"cafe\u0301 NLP"`, compute the raw and NFC code-point start indices of `NLP`. Then give the unseen `rearrange` segmentation after the four leading-boundary merges. Record which input distinctions each policy changes and one test it needs.

**E5. Count the units.** For `exam`, امتحان, and مِش (*mish*, written with one kasra), give the number of code points and of UTF-8 bytes. How many letters does a reader see in مِش? Which count would you report as "length" for a tokenizer budget, and why?

**E6. Run BPE yourself.** Learn three merges from the weighted corpus `hug`:10, `pug`:5, `pun`:12, `bun`:4, `hugs`:5, using characters, the end marker `_`, and the chapter's tie rule. Give each merge with its count, the final segmentation of `pun` and `hugs`, and the segmentation of the unseen word `bug`.

**E7. Predict the splitter.** Before running Listing 4.1, predict how many sentences it returns for each message, and mark any answer that is wrong:

(a) سألت أ. منى عن الموعد. لم ترد. (I asked Prof. Mona about the date. She did not reply.)

(b) "Is the portal down? I can't log in!"

(c) "The exam is at 10.30 tomorrow."

(d) عايز اغير الجدول الكارنيه ضاع برضه (I want to change the timetable the ID card got lost too)

**E8. Check the contract.** A model was trained with the vocabulary `[<unk>, i, love, nlp, data]` (IDs 0–4). Encode "i love nlp". A colleague later loads the alphabetical vocabulary `[<unk>, data, i, love, nlp]`. What text do the same IDs decode to? Write one test that would catch this before deployment.

### Read the picture

**V1.** In Figure 4.6, why are the merge counts weighted occurrences rather than numbers of distinct words?

**V2.** Figure 4.3 shows one text under three policies. Why can a model trained with one policy not simply be given IDs from another?

### Check your progress

Use the objective identifiers from the opening list. Quiz items check recall; Q items ask for explanation; E items, labs, and assignments ask you to calculate, implement, or report.

**Table 4.8** From Chapter 4 objectives to evidence of learning

| Goal | Preparation | Your evidence |
|---|---|---|
| O1 | Examples 4.1 and 4.2, Section 4.1.4, Section 4.4. | Quiz 1, 9; Q1, Q5, Q7–Q8; E4–E5; Worksheet 6. |
| O2 | Section 4.1: policies on paper, and Section 4.1.2. | Quiz 2; Q1; Worksheets 6 and 7. |
| O3 | Sections 4.1.7 and 4.1.8. | Quiz 3–5; Q2, Q6; E1, E4, E6: learn and apply fixed merges; V1. |
| O4 | Section 4.1.10. | Quiz 5, 10; Q3; E8; V2; Worksheet 8. |
| O5 | Sections 4.2, 4.2.1, and 4.3. | Quiz 6–8; Q4; E2: two task-specific policies; E3, E7: sentence boundaries; Worksheet 7. |

### Further reading

The Sennrich, Haddow, and Birch BPE paper develops subword units for rare words. The SentencePiece paper describes language-independent subword tokenization.

Unicode's normalization specification and text segmentation specification define the forms and boundary algorithms. The Python regex documentation defines engine-specific classes and flags.

CHAPTER 5

# N-gram Language Models

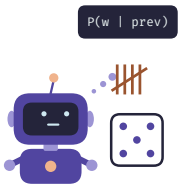
IN THIS CHAPTER


5.1 N-grams and count-based prediction 108
5.2 Evaluating a language model 112
5.3 Sparsity and smoothing 114
5.4 Sampling and Arabic n-grams 118
5.5 Review and practice 121


**LEARNING OBJECTIVES**

After working through this chapter, you should be able to:

- **O1.** construct sentence-bounded unigram, bigram, and trigram windows;
- **O2.** derive conditional probabilities from outgoing transition counts and the chain rule;
- **O3.** score a sentence with log probability and perplexity, and say which targets were counted;
- **O4.** explain sparsity, and repair it with add-one smoothing and an unknown-word policy;
- **O5.** combine orders by interpolation or normalised backoff, with weights chosen on development data;
- **O6.** sample a sentence from counts, and apply the same counting to Arabic text.




Chapter 3 counted single words and Chapter 4 fixed what a token is. Now we count *pairs* of tokens and use them to predict the next word. A model that gives every possible next word a probability is a language model; this chapter builds the simplest kind, from counts alone. Every model here is small enough to check by hand: you can see which transitions have evidence, what happens when evidence is missing, and how to measure the result honestly.

**The goal of this chapter** is to predict the next word from counts and to measure that prediction honestly. Three stops get there: count transitions and turn them into probabilities; measure how surprised the model is by held-out text; and repair the zeros

that unseen transitions cause. Figure 5.1 shows the route. Chapter 6 reuses the same counts to compare and classify whole messages.

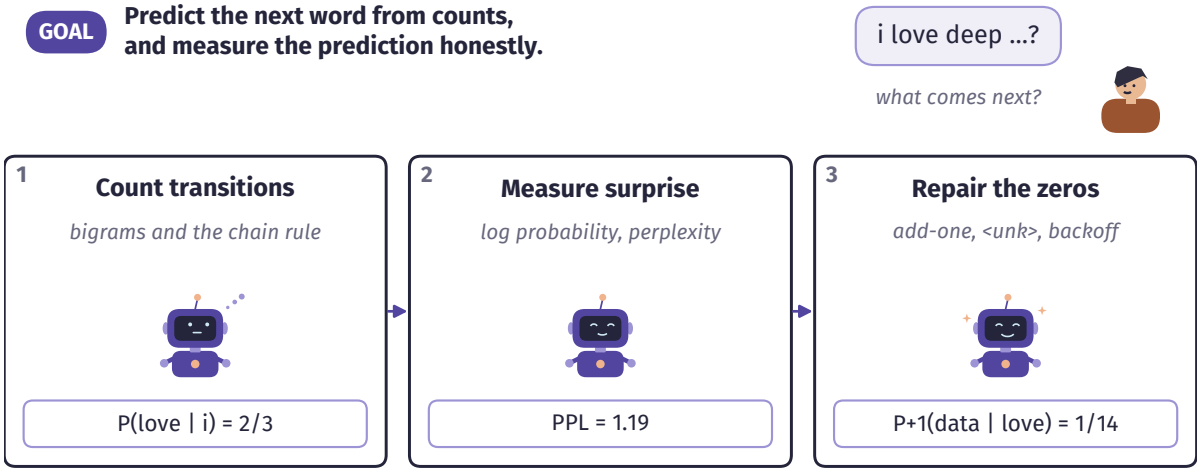



**Figure 5.1** Chapter 5's goal and route: counts become next-word probabilities (Section 5.1), perplexity measures how surprised the model is by held-out text (Section 5.2), and smoothing repairs the zeros of unseen transitions (Section 5.3). Each stop shows a number computed in the chapter.

**Listing 5.1** The running six-sentence corpus, shared with Chapter 7 (`examples/ch07/neural_bigram.py`)

```
corpus = [
    "I love natural language processing",
    "I love deep learning",
    "deep learning needs data",
    "natural language processing needs data",
    "students love deep learning",
    "I study natural language processing",
]
```

Lowercase each sentence and split it on spaces: the corpus has 27 word tokens and 11 word types. Every count credited to "the running corpus" can be checked against Listing 5.1. Other examples use small corpora of their own, named where they appear and listed in *How to use this book*.

## 5.1 N-grams and count-based prediction

An **$n$-gram** is a window of $n$ consecutive tokens. Without padding, a sequence of length $L$ has $\max(0, L - n + 1)$ of them: "i love natural language processing" has five unigrams, four bigrams, and three trigrams. A bigram model predicts from one previous token; a trigram model from two.

Any sentence probability can be split into one prediction per word. For "i love learning",

$$P(\texttt{i})\,P(\texttt{love} \mid \texttt{i})\,P(\texttt{learning} \mid \texttt{i}, \texttt{love}).$$

Each factor conditions on everything before it, so nothing has been assumed yet. In general this **chain rule** is exact:

$$P(w_1, \ldots, w_m) = P(w_1) \prod_{t=2}^{m} P(w_t \mid w_1, \ldots, w_{t-1}).$$

A bigram model makes one approximation: it replaces each long history by the previous word alone, $P(w_t \mid w_{t-1})$. This is the **Markov assumption**. To score whole sentences it also needs a start and an end policy (below); until then, the running corpus is counted without one.

The price of a short history is easy to see. In "The soups that I made from a new cookbook were delicious", the word before *were* is *cookbook*, but the verb must agree with *soups*, eight words back (Figure 5.2). A larger $n$ keeps more history but sees each window less often. Neural models share evidence across longer contexts, yet they too must be tested for agreement on held-out text.

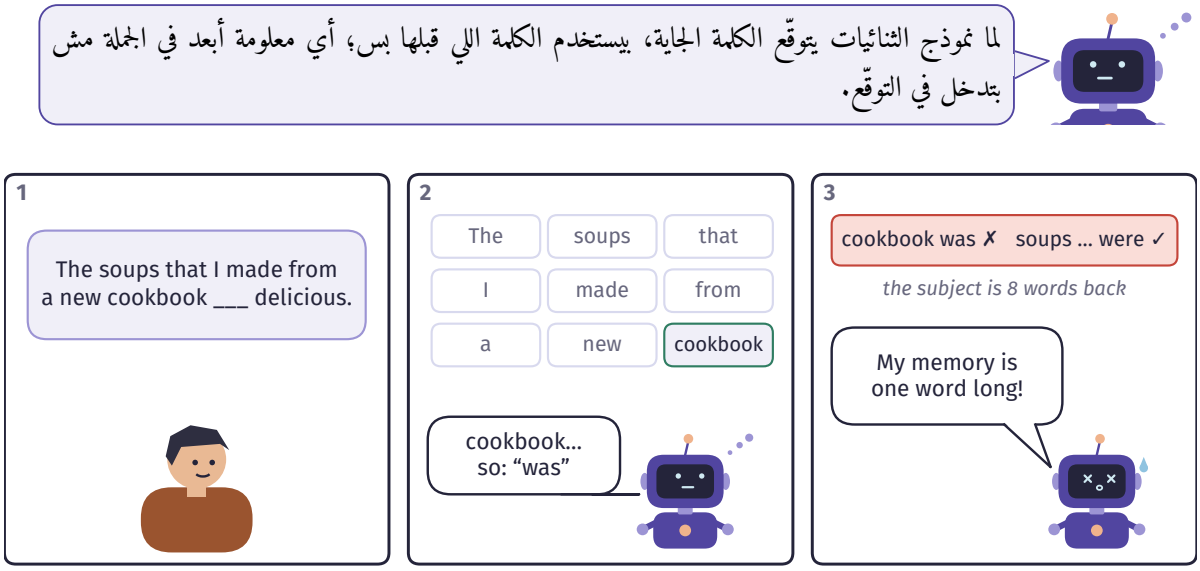



**Figure 5.2** A bigram model sees only the previous word. After *cookbook* it prefers *was*, but the verb must agree with *soups*, eight words back.

For counts $C(a, b)$ of training transitions from $a$ to $b$, let $C_{\text{out}}(a) = \sum_b C(a, b)$. Then

$$\widehat{P}(b \mid a) = \frac{C(a, b)}{C_{\text{out}}(a)}.$$

These normalised counts are the **maximum-likelihood estimate**: no other row of probabilities makes the training transitions more likely. That says nothing about performance on another domain.

The denominator counts *outgoing transitions*, not every occurrence of $a$. A sentence-final word has no outgoing transition unless an end marker follows it.

المقام بيعدّ المرات اللي للكلمة فيها تكملة، مش كل مرات ظهورها؛ ظهورها في آخر الجملة يتحسب بس لو بعده علامة نهاية.



**Example 5.1**

In the running corpus, `i` is followed by `love` twice and `study` once, so $P(\texttt{love} \mid \texttt{i}) = 2/3$ and $P(\texttt{study} \mid \texttt{i}) = 1/3$. Likewise `love` precedes `deep` twice and `natural` once, and all three occurrences of `deep` precede `learning`. These counts are the baseline that neural training must beat.

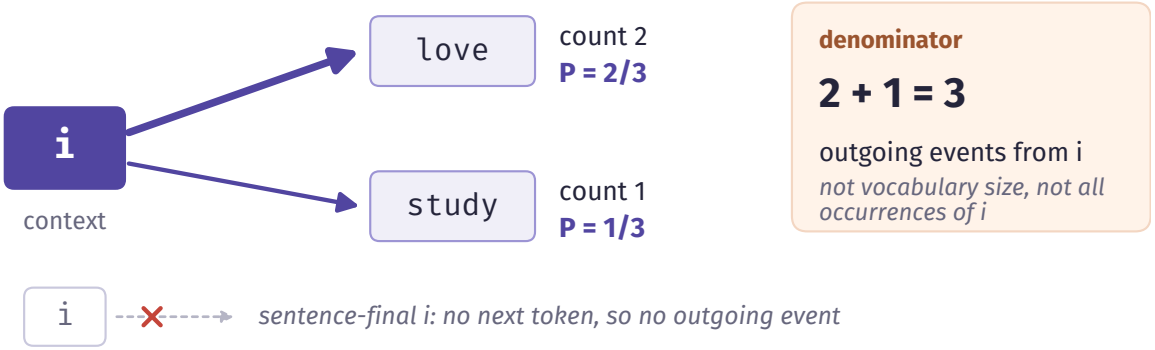



**Figure 5.3** Conditionals use outgoing transitions. In the unpadded running corpus, the context `i` has three observed targets; sentence-final appearances supply no additional outgoing event.

### 5.1.1 Compute conditional probabilities from a corpus

Compute three conditional probabilities from this marked corpus, keeping its casing:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

- Two of three sentences start with `I`: $P(\texttt{I} \mid \texttt{<s>}) = 2/3$.
- The three outgoing `I` transitions go to `am`, `am`, `do`: $P(\texttt{am} \mid \texttt{I}) = 2/3$.
- The two outgoing `am` transitions go to `Sam` and `</s>`: $P(\texttt{Sam} \mid \texttt{am}) = 1/2$. Without the end marker, `am` would have only one outgoing transition and this would wrongly be 1.

With $P(\texttt{</s>} \mid \texttt{Sam}) = 1/2$ as well, the unsmoothed probability of "I am Sam" is

$$(2/3)(2/3)(1/2)(1/2) = 1/9.$$

### 5.1.2 Boundary markers change the experiment

An n-gram pipeline can pad a sentence with $n-1$ start markers and one end marker. For "i love learning" the padded bigrams are

$$(\texttt{<s>}, \texttt{i}), \quad (\texttt{i}, \texttt{love}), \quad (\texttt{love}, \texttt{learning}), \quad (\texttt{learning}, \texttt{</s>}).$$

For trigrams the sequence is `<s> <s> i love learning </s>`, with four windows: two start markers predict `i`, `<s> i` predicts `love`, `i love` predicts `learning`, and `love learning` predicts the end marker. With padding, both orders give $L+1$ prediction windows for an $L$-word sentence; without it, $L-1$ and $L-2$. (The interactive window widget in the companion lecture slides leaves unigrams unpadded.)

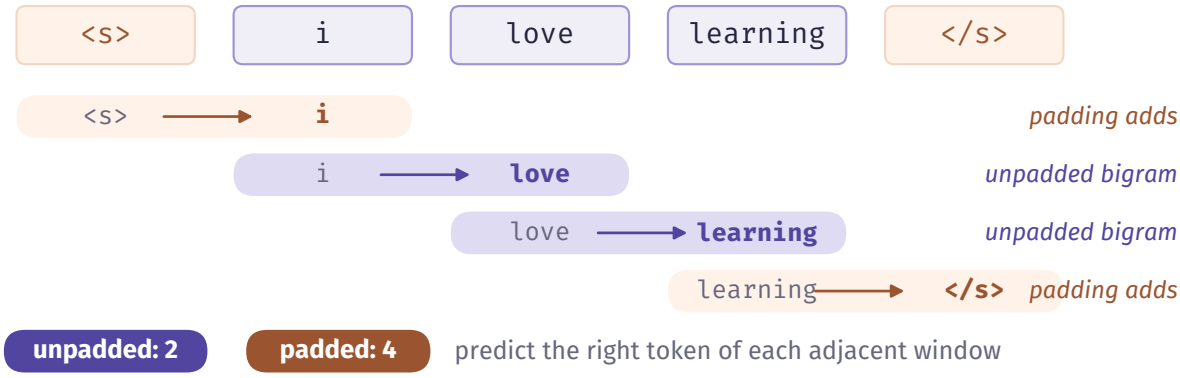



**Figure 5.4** Boundary padding creates explicit first-word and ending predictions. The unpadded sequence has two bigrams; adding start and end markers produces four.

A start marker is only context; an end marker is something the model predicts. So the *target support*, the set of symbols the model may output, includes `</s>` but not `<s>`. Fix the support before computing any smoothing denominator.

> **Example 5.2**
>
> Train on "i love learning" and "i love data" with one start and one end marker. Then $P(\texttt{i} \mid \texttt{<s>}) = 1$, $P(\texttt{love} \mid \texttt{i}) = 1$, $P(\texttt{learning} \mid \texttt{love}) = 1/2$, and each final word predicts `</s>` with probability one. The first sentence, including its ending, has probability $1 \cdot 1 \cdot (1/2) \cdot 1 = 1/2$. Without such a start and end policy, the unpadded running corpus cannot give a whole-sentence probability.

Multiplying many small probabilities can underflow to zero, so add their logarithms instead:

$$\ln P(w_{1:m}, \texttt{</s>}) = \sum_{t=1}^{m+1} \ln P(w_t \mid w_{t-1}),$$

where $w_0 = \texttt{<s>}$ and $w_{m+1} = \texttt{</s>}$.

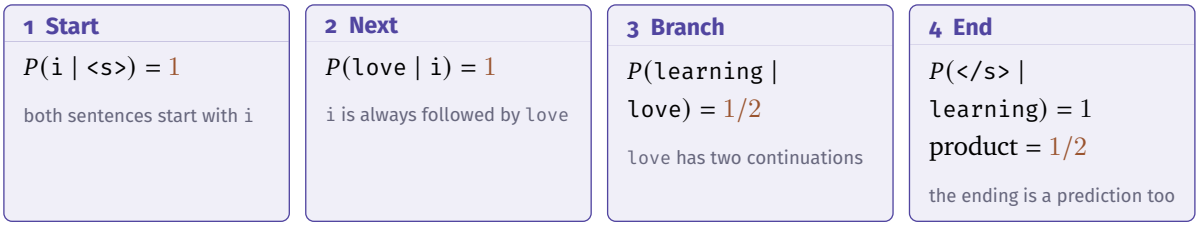



**Figure 5.5** Example 5.2 by the chain rule, one factor per frame. Amber marks the new factor.

## 5.2 Evaluating a language model

A language model is useful only if it predicts text it has not seen. This section turns the probability of held-out text into one number per model, the perplexity, and says what that number does and does not show.

### 5.2.1 Average surprise: loss and perplexity

Extending a fixed prefix multiplies its probability by a number between zero and one, so its probability cannot increase. This creates a length bias; it does not mean every long sentence is less likely than every short one. To compare average prediction difficulty, measure the model's surprise *per target* instead:

1. For each predicted word (a *target*), compute its surprise $-\ln P$. A likely word ($P$ near 1) gives a surprise near 0; an unlikely word gives a large one.
2. Average the surprise over all $N_{\text{tgt}}$ targets. This average is the loss $\mathcal{L}$.
3. Undo the logarithm with $\exp$. The result is the perplexity.

$$\mathcal{L} = -\frac{1}{N_{\text{tgt}}} \sum_{\text{targets}} \ln P(w_t \mid w_{t-1}), \qquad \text{PPL} = \exp(\mathcal{L}).$$

$\mathcal{L}$ is the **mean negative log probability** (the average loss), and PPL is the **perplexity**. Read perplexity as "how many equally likely options the model is choosing between at each step". If every target gets probability $1/4$, then $\mathcal{L} = \ln 4$ and $\text{PPL} = 4$: the model is as unsure as someone picking one of four options at random. Lower is better, but only when two models are scored on the same targets (Chapter 8).

**Which targets count?** A sentence of $m$ words gives $m + 1$ targets: the $m$ words plus `</s>`, because the model must also predict where the sentence ends. `<s>` is given, not predicted, so it never counts.

**A zero breaks everything.** If one target has probability zero, $\ln 0 = -\infty$, so the loss and the perplexity become infinite. Report this rather than hiding it: the model has called something that actually happened impossible. Smoothing (Section 5.3) is the cure.

**Example 5.3**

In Example 5.2, "i love learning" has probability $1/2$. Its four targets are `i`, `love`, `learning`, and `</s>`. So $\mathcal{L} = \frac{1}{4}\ln 2 \approx .173$ and $\mathrm{PPL} = 2^{1/4} \approx 1.19$. If you forget the end marker you count only three targets and get $2^{1/3} \approx 1.26$, a different number for the same model. Always say which targets you counted.

الحيرة مش عدد كلمات القاموس؛ لو قيمتها أربعة، فصعوبة التوقّع في المتوسط زي الاختيار بين أربع بدائل احتمالهم متساوي.



### 5.2.2 Prediction quality and application quality

**Intrinsic evaluation** measures the predictor itself, for example by held-out perplexity. **Extrinsic evaluation** measures what it adds to a complete task, such as word error rate in speech recognition or correctly answered help-desk tickets. Swapping only the language model while keeping every other component fixed gives a controlled comparison. A lower perplexity alone does not prove that more tickets are resolved (Figure 5.6).

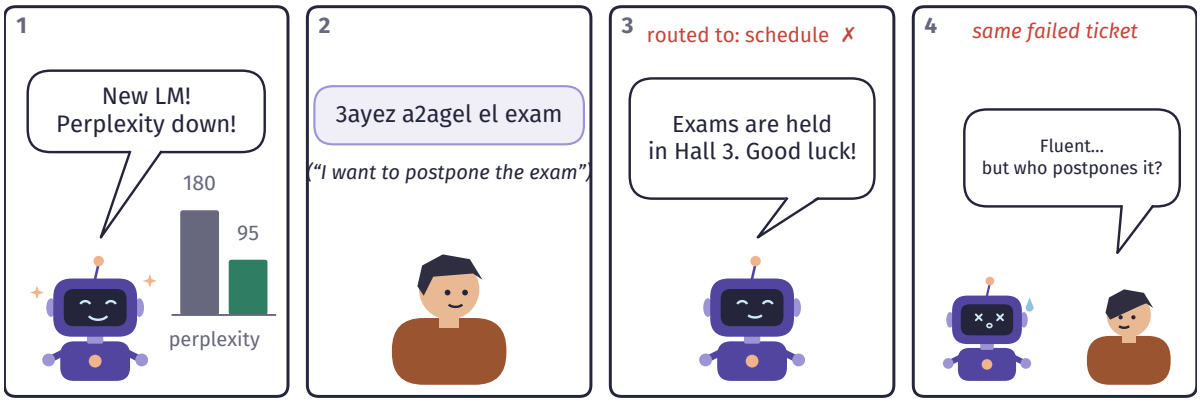



**Figure 5.6** Sanad's new language model lowers perplexity, yet Nour's request to postpone an exam is still answered with a fluent schedule reply and routed to the wrong team. An intrinsic gain is not evidence of task completion. The perplexity values are illustrative.

Perplexity behaves like an *effective* number of choices, not the number of possible words. Take five unpadded targets `red red red red blue`. A uniform model over `red`, `blue`, `green` gives each target $1/3$, so the perplexity is 3. A model giving $(.8, .1, .1)$ instead gives

$$P(T) = .8^4(.1) = .04096, \qquad \mathrm{PPL}(T) = .04096^{-1/5} \approx 1.895,$$

although three words are still possible. (This example has no end marker, unlike the sentence model; compare models only on the same targets.)

Counts also describe *where* the text came from. A preference in one corpus does not show popularity in another population, and a zero count does not prove a sentence is ungrammatical. Plan for shifts in domain, author, and dialect, and keep the corpus statement from Chapter 3.

## 5.3 Sparsity and smoothing

A vocabulary of size $V$ allows $V^n$ ordered $n$-grams, but a corpus contains only a limited number of windows, many repeated. A larger $n$ captures more context and leaves more combinations unseen. One unseen transition gives probability zero to every sentence that contains it.

For $V = 10{,}000$ there are $10^8$ possible bigrams and $10^{12}$ possible trigrams. A corpus of one million windows can fill at most 1% of the bigram space and 0.0001% of the trigram space; repeated windows fill even less. Language is not spread evenly over these combinations, and useful models exploit that.

### 5.3.1 Zero counts and add-one smoothing

> **Example 5.4**
>
> **Every word is known, yet the sentence is "impossible".** In the running corpus, `love` has three outgoing transitions: `deep` twice and `natural` once. It is never followed by `data`, so $P(\texttt{data} \mid \texttt{love}) = 0$ and "i love data" gets probability zero, although `i`, `love`, and `data` all occur in training (Figure 5.7).

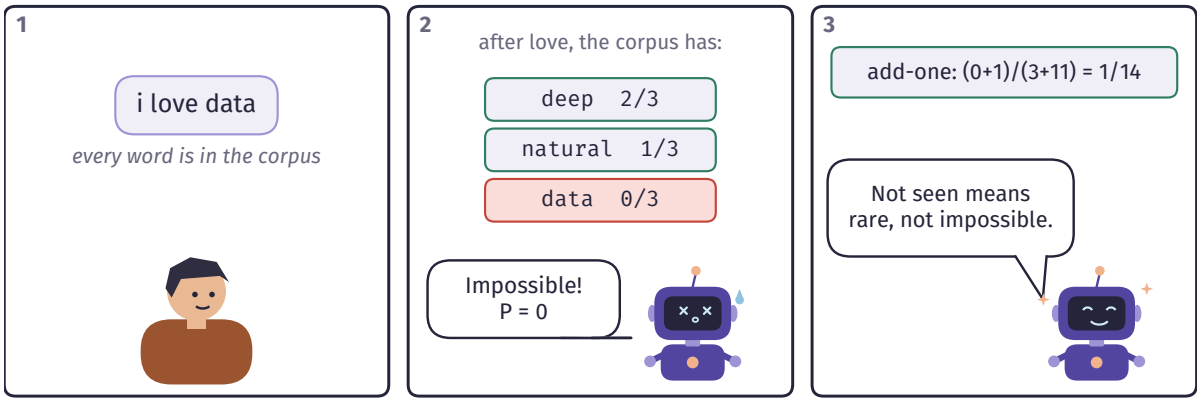



**Figure 5.7** Every word of "i love data" occurs in the running corpus, but `love` is never followed by `data`, so the unsmoothed model calls the sentence impossible. Add-one smoothing gives the transition $1/14$: rare, not impossible.

**Add-one smoothing** adds one to every count in a row:

$$\widehat{P}_{+1}(b \mid a) = \frac{C(a,b) + 1}{C_{\text{out}}(a) + V}.$$

The row still sums to one over the $V$ targets, and an unseen context gets a uniform row. For Example 5.4, $\widehat{P}_{+1}(\texttt{data} \mid \texttt{love}) = (0+1)/(3+11) = 1/14$: rare, no longer impossible. Smoothing reserves probability for unseen *transitions* between known symbols; it does not create evidence that any one of them is likely.

في تمهيد إضافة الواحد، بندّي احتمال حتى لانتقال ما ظهرش في التدريب؛ عدم ظهوره مش معناه إنه مستحيل، ولا التمهيد دليل إنه شائع.

**Example 5.5**

Use the eleven-word vocabulary and context `i`, with three outgoing transitions. Add-one gives `love` $3/14$, `study` $2/14$, and each of the other nine words $1/14$; the sum is $(3 + 2 + 9)/14 = 1$. That is a big change: the observed favourite falls from $2/3$ to $3/14$. *Add-α* smoothing adds $\alpha > 0$ instead of one, with denominator $3 + 11\alpha$; choose $\alpha$ on validation data.

**Pause, predict, then explore**

Use two new sentences: "i see bird" and "i see fish". Add one start and one end marker to each. Draw the two outgoing arrows from `see`.

---

Calculate $P(\texttt{bird} \mid \texttt{see})$ without smoothing, then with add-one smoothing over exactly `i`, `see`, `bird`, `fish`, and `</s>`. The start marker is context only.

---

**Figure 5.8** Count targets, then check the support used by smoothing.

### 5.3.2 Fix the support before scoring held-out text

Smoothing over a fixed vocabulary does not add a *word* outside that vocabulary: the row has no entry for it. Returning a positive value for an extra word without reallocating the existing mass would make that row sum to more than one, and adding held-out words to $V$ after looking at them would let test data change the model.

The standard repair is decided before any scoring:

1. Fit the vocabulary on training text alone.
2. Reserve one **unknown symbol** `<unk>` in the target support, so $V$ counts the training words, `</s>`, and `<unk>`.
3. Map every held-out word outside the fitted vocabulary to `<unk>`, both as a target and as a context.
4. State where `<unk>` gets its probability. Under add-one alone it gets only the added count. A common alternative maps rare training words, such as hapaxes, to `<unk>` so that it has real evidence.

Report the **unknown-target rate** beside the perplexity. A model that maps most test words to `<unk>` solves a much easier problem, so its low perplexity says little about the real words.

**Example 5.6**

**Scoring a sentence with an unknown word.** Train on "i love learning" and "i love data" (Example 5.2). The support is `i`, `love`, `learning`, `data`, `</s>`, `<unk>`, so $V = 6$. Score the held-out "i love music": *music* becomes `<unk>`, giving the targets `i love <unk> </s>`. With add-one:

$$\frac{2+1}{2+6} \cdot \frac{2+1}{2+6} \cdot \frac{0+1}{2+6} \cdot \frac{1}{6} = \frac{3}{8} \cdot \frac{3}{8} \cdot \frac{1}{8} \cdot \frac{1}{6} = \frac{9}{3072} \approx .0029.$$

The last factor is $1/6$ because `<unk>` never occurred as a context, so its row is uniform. One of the four targets is unknown: report an unknown-target rate of $1/4$ beside the score.

**CHECK THE CODE** `examples/ch05/review_checks.py` – asserts this score, the zero-count example, and the new review numbers.

كوّن القاموس من بيانات التدريب وثبّته، ومثّل أي كلمة برّه القاموس بنفس رمز المجهول؛ ما تضيفش له كلمات من بيانات الاختبار.



### 5.3.3 Backoff and interpolation

*Backoff* uses a shorter context when a longer one has too little evidence. *Interpolation* mixes the two, for example

$$P(b \mid a, c) = \lambda P_3(b \mid a, c) + (1 - \lambda) P_2(b \mid c), \qquad 0 \le \lambda \le 1.$$

Both parts must already be proper distributions, including for unseen contexts.

In general, a three-order mixture is

$$P(w \mid a, b) = \lambda_3 P_3(w \mid a, b) + \lambda_2 P_2(w \mid b) + \lambda_1 P_1(w), \quad \lambda_i \ge 0, \quad \sum_i \lambda_i = 1.$$

If every part uses the same target support and handles missing contexts, the mixture sums to one. Weights must be fixed per context, not chosen separately for each candidate word.

**Example 5.7**

**Where does the missing mass go?** Take fixed rows over three targets A, B, C: high-order $(.75, .25, 0)$ and low-order $(.5, .25, .25)$. A half-and-half mixture is $(.625, .25, .125)$, which sums to one. A naive fallback that keeps the high-order A and B and copies the low-order value for unseen C gives $(.75, .25, .25)$, which sums to $1.25$.

Normalized backoff first discounts the observed values to $.6$ and $.2$, leaving $.2$ spare. It shares that spare mass among the unseen targets in proportion to their low-order

probabilities. Only C is unseen, so it gets all .2: the row is $(.6, .2, .2)$. Figure 5.9 compares the three rows. These discounts are chosen for illustration; they are not Katz or Kneser–Ney estimates.

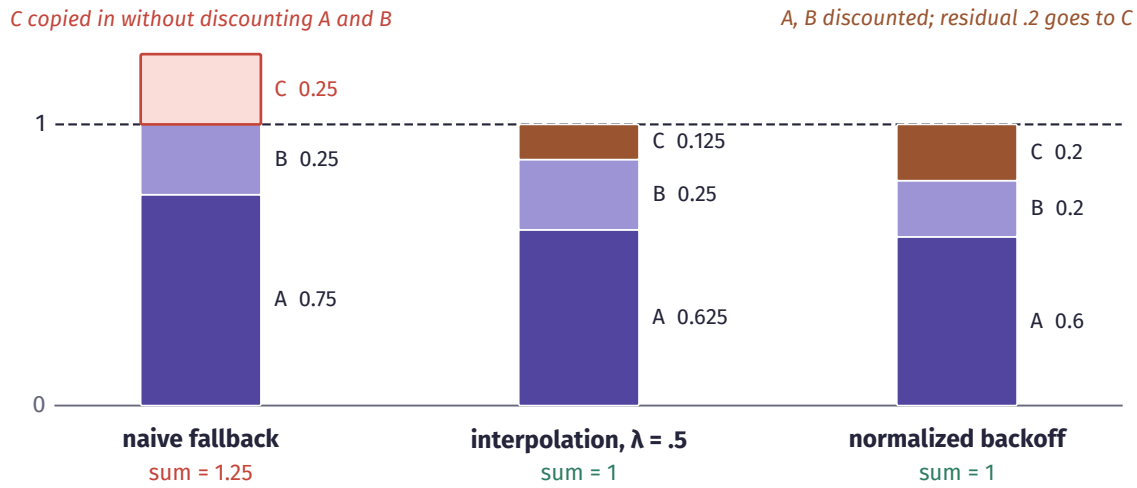



**Figure 5.9** Where the missing mass goes, for the rows of Example 5.7. Copying the low-order value for unseen C on top of undiscounted A and B gives a row summing to 1.25. Interpolation and normalized backoff both sum to one.

لو مجموع الاحتمالات واحد وعايز تدّي فرصة لكلمات ما ظهرتش بعد السياق ده، لازم تقلّل نصيب الكلمات اللي ظهرت عشان المجموع يفضل واحد.



In general, if $S$ is the set of observed targets with discounted probabilities $q(v)$, each unseen target gets

$$P_{\text{bo}}(w) = \frac{1 - \sum_{v \in S} q(v)}{\sum_{v \notin S} P_{\text{low}}(v)} P_{\text{low}}(w), \qquad w \notin S.$$

This needs non-negative spare mass and, when it is positive, some low-order mass on the unseen targets. If no target is unseen, reserve nothing.

**Stupid Backoff** uses scaled lower-order scores without normalising each row. Its scores can rank candidates well, but they are not probabilities, so they cannot enter the perplexity formula above.

### 5.3.4 Choose mixture weights on development data

Freeze the count models fitted on training data. Then choose the mixture weights that give the lowest mean loss on *development* data. Do not refit counts on development text, and keep the test set out of weight selection.

For the two rows of Example 5.7, write $P_\lambda = \lambda P_{\text{high}} + (1 - \lambda) P_{\text{low}}$. A development set with targets A, A, A, C gives:

| $\lambda$ | $P_\lambda(\mathrm{C})$ | **Mean dev loss (nats)** |
|---|---|---|
| 0 | .25 | .8664 |
| .25 | .1875 | .8500 |
| .5 | .125 | .8724 |
| .75 | .0625 | .9742 |
| 1 | 0 | $\infty$ |

The best grid weight is .25, giving $(.5625, .25, .1875)$. It wins on this tiny set, not on language in general. Freeze it before testing.

> **CHECK THE CODE** `examples/ch05/interpolation_backoff.py` – checks the grid, the mass accounting, and the colour perplexity example.

**Effective counts expose the size of a change.** Multiply a smoothed row by the context's outgoing count to see it on the original count scale: $C^*(w) = C_{\text{out}}\widehat{P}(w)$. In Example 5.5, `love`'s effective count is $3(3/14) = 9/14 \approx .643$, although its raw count is 2. The eleven effective counts still sum to 3; they show how much add-one moved, not new observations.

## 5.4 Sampling and Arabic n-grams

Two more uses of the same counts close the chapter: generating text by sampling, and counting Arabic, where the choice of unit changes every row.

### 5.4.1 Sampling a sentence from counts

Sampling draws the next word from its distribution instead of always taking the most likely one. Put the targets in a fixed order, give each an interval of $[0, 1)$ as long as its probability, and draw $u$ uniformly. For the row `data` .5, `learning` .5, any $u < .5$ selects `data` and any $u \geq .5$ selects `learning`. Report the order and the seed so that the run can be repeated.

Start from `<s>`, draw a word, append it, and use it as the next context. Stop at `</s>` or at a declared length budget, and record which one stopped the run: a model with a cycle can run forever even when every row sums to one.

> **Example 5.8**
>
> **A complete sample trace.** Fit the two-sentence corpus of Example 5.2. The `<s>` and `i` rows always give `i` and `love`. The `love` row chooses between `data` and `learning`, and either word then predicts `</s>`. With targets ordered alphabetically and Python's seed 7, the sampler produces `[i, love, learning, </s>]`. With a budget of two targets it stops after `[i, love]` with status `budget`. Figure 5.10 draws the trace.

> **CHECK THE CODE** `examples/ch05/bigram_sampling.py` – reproduces this sample trace.

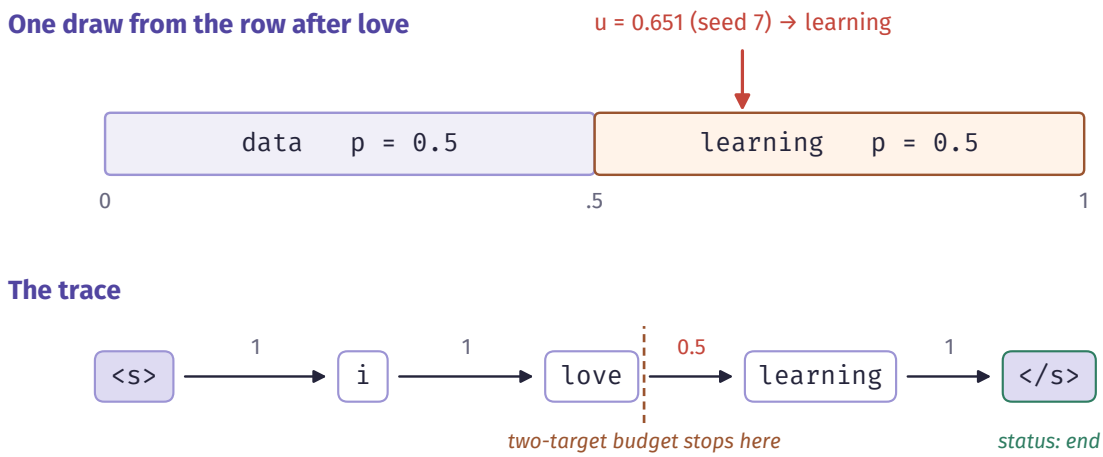



**Figure 5.10** Sampling from the two-sentence bigram model. Top: the row after `love` as cumulative intervals; seed 7 draws $u \approx .651$, which falls in the `learning` interval. Bottom: the complete trace, with each transition's probability; a two-target budget would stop after `love`.

A fluent sample may copy training text or join pieces inappropriately, and it is not a held-out score. Temperature and truncation (Chapter 7) change how we sample, not the fitted counts.

### 5.4.2 N-grams on Arabic text

The counting rules are the same in every language; the units are not. Figure 5.11 pads three short Arabic requests: "I want to postpone the exam", "I want to postpone the submission", and "when is the exam". Arabic reads right to left, so the start marker stands at the right, but the model still conditions on the *preceding* token in reading order. The target support has $V = 6$ symbols: five words and the end marker.

> **CHECK THE CODE** `examples/ch05/arabic_bigram.py` – reproduces every count and probability.

The requests arrive already split into sentences and tokens by the policy of Chapter 4 (Sections 4.2.1 and 4.3). This section changes only the counting; a segmentation or normalisation error made there reaches every row here.

**Example 5.9**

Unsmoothed, the first request, أريد تأجيل الاختبار (*urid ta2jil al-ikhtibar*, "I want to postpone the exam"), has probability (in transliteration, so the formula reads left to right)

$$P(\textit{urid} \mid \texttt{<s>})\, P(\textit{ta2jil} \mid \textit{urid}) \cdot P(\textit{al-ikhtibar} \mid \textit{ta2jil})\, P(\texttt{</s>} \mid \textit{al-ikhtibar}) = \tfrac{2}{3} \cdot 1 \cdot \tfrac{1}{2} \cdot 1 = \tfrac{1}{3},$$

and a perplexity of $3^{1/4} \approx 1.32$ over its four targets (Section 5.2.1). The new request "when …postponing the exam" needs the transition from متى (*mata*, when) to تأجيل (*ta2jil*, postponing), which never occurs, so its unsmoothed probability is zero.

**(a) Padded requests** (read right to left)

| end | Request | start |
|---|---|---|
| </s> | أريد تأجيل الاختبار | <s> |
| </s> | أريد تأجيل التسليم | <s> |
| </s> | متى الاختبار | <s> |

أريد I want تأجيل postponing
الاختبار the exam التسليم the submission
متى when

**(b) Transitions, MLE and add-one ($V = 6$ targets)**

| Context $a$ | Next $b$ | $C(a,b)$ | $C_{\text{out}}(a)$ | MLE | Add-one |
|---|---|---|---|---|---|
| <s> | أريد | 2 | 3 | 2/3 | 3/9 |
| <s> | متى | 1 | 3 | 1/3 | 2/9 |
| أريد | تأجيل | 2 | 2 | 1 | 3/8 |
| تأجيل | الاختبار | 1 | 2 | 1/2 | 2/8 |
| تأجيل | التسليم | 1 | 2 | 1/2 | 2/8 |
| متى | الاختبار | 1 | 1 | 1 | 2/7 |
| الاختبار | </s> | 2 | 2 | 1 | 3/8 |
| التسليم | </s> | 1 | 1 | 1 | 2/7 |
| متى | تأجيل | 0 | 1 | 0 | 1/7 |

**Figure 5.11** A three-request Arabic corpus with boundary markers, and every observed bigram with its unsmoothed and add-one estimate. The shaded row is an unseen transition.

> With add-one the four factors are $2/9$, $1/7$, $2/8$, and $3/8$, with product $1/336 \approx .0030$ (natural log $\approx -5.82$). Smoothing makes the request possible, but on one invented count, not on evidence.

Arabic morphology changes what a word bigram is. Without segmentation, الامتحان (*al-imtihan*, the exam), والامتحان (and the exam), and للامتحان (for the exam) are three unrelated types, each with its own sparse row. Splitting off clitics lets them share counts, but lengthens sequences: a bigram then often conditions on و or ال instead of the previous content word. Neither choice is neutral; compare them on held-out text with the same targets (Chapter 8).

فصل «الامتحان» و«والامتحان» بخلّيهم يشتركوا في «امتحان»، بس الجزء اللي قبله مباشرة هيبقى «ال»؛ نموذج الثنائيات مش هيشوف أبعد من كده.



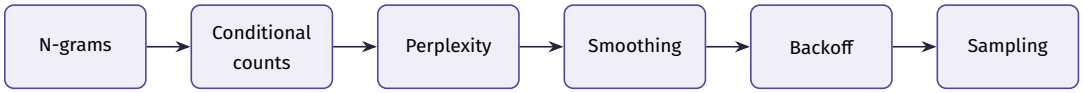
Character n-grams can identify a language variety without segmenting words. Egyptian negation wraps a verb as ما…ش (*ma-…-sh*), so ماعرفتش (*ma3raftish*, I didn't know) is probably an unseen word type, yet its final ش recurs across many negated verbs. Short, frequent markers such as مش (*mish*, not) and عايز (*3ayiz*, want) suit character trigrams. A model trained only on Modern Standard Arabic may give such requests low probability; measure the gap on held-out data before blaming the variety.



**CODE IT IN THE LAB MANUAL**

**Lab 3, part G**: a bigram model and sampling (G1–G6) on the core route, and interpolation (G7) as a challenge CHALLENGE. **Sanad**: no new project level yet. Level 5, after Chapter 6, puts this chapter's interpolated bigram model behind the keyboard, next to that chapter's search and router.

**Chapter 5 at a glance**




**Keep these ideas**

- A bigram model estimates $P(w \mid \text{previous})$ from outgoing transition counts; the chain rule multiplies them.
- Perplexity is the inverse geometric mean of the target probabilities: lower is better, on the same targets only.
- Unseen transitions need smoothing or a suitable backoff rule; words outside the vocabulary also need an unknown-word policy.
- Fix the vocabulary and `<unk>` on training text before scoring anything held out.
- Choose interpolation weights on development data, with the counts frozen.

**In one small example**

After "i love learning" and "i love data", the sentence "i love learning" with its end marker has probability $1 \cdot 1 \cdot \frac{1}{2} \cdot 1 = \frac{1}{2}$ (Figure 5.5).

**Common mistake.** Counting a sentence-final word as having an outgoing transition, or ignoring start and end markers.
**Instead.** State the boundary policy first; then count only the transitions it defines.

## 5.5 Review and practice

These questions follow the chapter objectives in order. Explain the reasoning before checking the optional answers.

### Quick quiz

Choose one answer for each question before reading the rest of the review; the key is in the instructor edition.

**1.** How many unpadded trigram windows does a five-token sentence contain?

- **A** 5.
- **B** 4.
- **C** 3.
- **D** 2.

**2.** In the `I am Sam` corpus, what is $P(\texttt{am} \mid \texttt{I})$?

- **A** $2/3$.
- **B** $1/3$.
- **C** $1/2$.
- **D** $1$.

**3.** A six-word sentence is scored by a bigram model with one start and one end marker. How many targets enter its perplexity?

- **A** 5.
- **B** 6.
- **C** 7.
- **D** 8.

**4.** A model gives every target probability $1/4$. What is its perplexity?

- **A** $1/4$.
- **B** $4$.
- **C** $\ln 4$.
- **D** It depends on the number of targets.

**5.** Context `i` has three outgoing transitions, two to `love`, and $V = 11$. What is the add-one estimate of $P(\texttt{love} \mid \texttt{i})$?

- **A** $2/3$.
- **B** $3/14$.
- **C** $2/14$.
- **D** $1/11$.

**6.** For $V = 10{,}000$, how many ordered bigrams are possible?

- **A** $10^4$.
- **B** $2 \times 10^4$.
- **C** $10^{12}$.
- **D** $10^8$.

**7.** A held-out sentence contains a word that never occurred in training. What does the chapter's policy do with it?

- **A** Adds it to the vocabulary and refits the counts.
- **B** Skips it when computing perplexity.
- **C** Maps it to `<unk>`, fixed in the support before scoring, and reports the unknown-target rate.
- **D** Gives it probability zero and reports infinite perplexity as the result.

**8.** A fallback keeps the high-order row $(.75, .25, 0)$ for A and B and copies the low-order $.25$ for unseen C. What is wrong with the result?

- **A** It sums to $1.25$, so it is not a probability distribution.
- **B** Nothing; every target now has a positive probability.
- **C** C should get the high-order value, $0$.
- **D** It sums to less than one.

**9.** Where should the weights of an interpolated model be chosen?

- **A** On the test set, to report the best score.
- **B** On the training data, refitting the counts for each weight.
- **C** By reading generated samples and keeping the most fluent.
- **D** On development data, with the training counts frozen.

**10.** What is the tradeoff of segmenting Arabic clitics before counting bigrams?

- **A** There is none; sparsity always falls.
- **B** Forms share counts, but a bigram often conditions on a clitic instead of a content word.
- **C** It increases the number of types.
- **D** It removes the need for smoothing.

## Check your understanding

**Q1.** Why does the conditional denominator count outgoing transitions?

**Q2.** Why does the target support include `</s>` but not `<s>`?

**Q3.** In "The soups that I made from a new cookbook were delicious", why does a bigram model have no evidence for *were* over *was*? Would a trigram model fix it?

**Q4.** Why can two perplexities be compared only when both models score the same targets?

**Q5.** What does add-one smoothing do for an unseen context?

**Q6.** Why can a naive fallback row sum above one? How do normalized backoff and interpolation avoid that problem?

**Q7.** Why should interpolation weights be chosen on development data, and why is a fluent generated sample insufficient evidence of application quality?

**Q8.** Why can segmenting Arabic clitics both reduce and worsen sparsity for a bigram model?

## Work it through

**E1. List the windows.** Pad "i love deep learning" for a trigram model, list every prediction window, and check the count against $L + 1$. Which window lets the model predict the end of the sentence?

**E2.** For the two-sentence boundary example, calculate the unsmoothed probability of "i love data" including its ending. Identify one unseen transition that would make another sentence have probability zero.

**E3.** Using Figure 5.11, compute the unsmoothed and add-one probabilities of the second request, "I want to postpone the submission", including its end marker. Which is larger, and why?

**E4.** A model assigns the targets of one held-out sentence, including `</s>`, the probabilities $0.5$, $0.25$, $0.25$, and $0.125$. Compute the mean negative log probability and the perplexity. What happens if one more target receives probability zero?

**E5. Score an unknown word.** Use the model of Example 5.6 ($V = 6$, add-one, `<unk>` in the support). Score the held-out "i love data" and the held-out "i like data", including the end marker. Give each product and each unknown-target rate. Which sentence scores higher, and why should you not conclude that the model "understands" it better?

**E6.** For high-order probabilities $(.75, .25, 0)$ and low-order probabilities $(.5, .25, .25)$ over A, B, C, calculate the mixture with high-order weight $.25$. Then retain discounted A/B probabilities $.6$ and $.2$, allocate residual mass to unseen C, and compare with naive fallback. Which rows can enter the chapter's perplexity calculation?

## Read the picture

**V1.** Redraw Figure 5.5 for "i love data". What is its probability, and what do the two sentence probabilities add up to?

**V2.** In Figure 5.3, why does a sentence-final appearance of a word add no outgoing event when the corpus is not padded?

## Check your progress

Use the objective identifiers from the opening list. Quiz items check recall; Q items ask for explanation; E items, labs, and assignments ask you to calculate, implement, or report.

**Table 5.1** From Chapter 5 objectives to evidence of learning

| Goal | Preparation | Your evidence |
|---|---|---|
| O1 | Boundary markers and Figure 5.11. | Quiz 1; Q2; E1; Lab 3, G1. |
| O2 | Examples 5.1–5.2, Section 5.1.1. | Quiz 2; Q1, Q3; E2–E3; V1–V2; Lab 3, G2–G3. |
| O3 | Section 5.2. | Quiz 3–4; Q4; E4; Lab 3, G4–G5. |
| O4 | Sections 5.3.1 and 5.3.2. | Quiz 5–7; Q5; E5. |
| O5 | Sections 5.3.3 and 5.3.4. | Quiz 8–9; Q6–Q7. CHALLENGE E6 and Lab 3, G7 (interpolation). |
| O6 | Section 5.4. | Quiz 10; Q7–Q8; E3; Lab 3, G6. |

## Further reading

Jurafsky and Martin's Speech and Language Processing, chapter "N-gram Language Models", covers the same ground with larger corpora and Kneser–Ney smoothing.

Brants et al., Large Language Models in Machine Translation (2007), introduces Stupid Backoff scores. Heafield, KenLM: Faster and Smaller Language Model Queries (2011), studies efficient queries and storage for estimated n-gram models. These complement the chapter's teaching estimates; neither is implemented by the small examples here.